



УНИВЕРЗИТЕТ У НОВОМ САДУ
ФАКУЛТЕТ ТЕХНИЧКИХ
НАУКА У НОВОМ САДУ



Лука Бурсаћ

**Управљање датотекама,
трансакцијама и
интерпретација упита у
релационим базама података**

Нови Сад, 2026

	УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА 21000 НОВИ САД, Трг Доситеја Обрадовића 6	Број:
	ЗАДАТАК ЗА ЗАВРШНИ РАД	Датум:

(Податке уноси предметни наставник - менџор)

Студијски програм:	Софтверско инжењерство и информационе технологије		
Студент:	Лука Бурсаћ	Број индекса:	SV 22/2022
Степен и врста студија:	Основне академске студије		
Област:	Електротехничко и рачунарско инжењерство		
Ментор:	Бранко Милосављевић		
НА ОСНОВУ ПОДНЕТЕ ПРИЈАВЕ, ПРИЛОЖЕНЕ ДОКУМЕНТАЦИЈЕ И ОДРЕДБИ СТАТУТА ФАКУЛТЕТА ИЗДАЈЕ СЕ ЗАДАТАК ЗА ЗАВРШНИ РАД, СА СЛЕДЕЋИМ ЕЛЕМЕНТИМА: - проблем – тема рада; - начин решавања проблема и начин практичне провере резултата рада, ако је таква провера неопходна;			

НАСЛОВ ЗАВРШНОГ РАДА:

Управљање датотекама, трансакцијама и интерпретација упита у релационим базама података
--

ТЕКСТ ЗАДАТКА:

Пројектовати и имплементирати систем за управљање релационим базама података у програмском језику Java. Систем треба да обухвати управљање датотекама и блоковима на диску, баферовање страница у радној меморији, трансакционе механизме са гарантовањем ACID особина, управљање метаподацима, као и парсирање, планирање и извршавање подскупа SQL језика кроз стабло релационих оператора. Систем реализовати у клијентско-серверској архитектури са сопственим протоколом за мрежну комуникацију. Исправност решења проверити одговарајућим скупом аутоматизованих тестова.
--

Руководилац студијског програма:	Ментор рада:

Примерак за: ☐ - Студента; ☐ - Ментора
--



УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА
21000 НОВИ САД, Трг Доситеја Обрадовића 6

КЉУЧНА ДОКУМЕНТАЦИЈСКА ИНФОРМАЦИЈА

Редни број, РБР :	
Идентификациони број, ИБР :	
Тип документације, ТД :	Монографска документација
Тип записа, ТЗ :	Текстуални штампани материјал
Врста рада, ВР :	Дипломски - бечелор рад
Аутор, АУ :	Лука Бурсаћ
Ментор, МН :	Др Бранко Милосављевић, редовни професор
Наслов рада, НР :	Управљање датотекама, трансакцијама и интерпретација упита у релационим базама података
Језик публикације, ЈП :	српски/ћирилица
Језик извода, ЈИ :	српски/енглески
Земља публиковања, ЗП :	Република Србија
Уже географско подручје, УГП :	Војводина
Година, ГО :	2026
Издавач, ИЗ :	Ауторски репринт
Место и адреса, МА :	Нови Сад, трг Доситеја Обрадовића 6
Физички опис рада, ФО : (поглавља/страна/ цитата/табела/слика/графика/прилога)	11/83/12/11/48/0/2
Научна област, НО :	Електротехничко и рачунарско инжењерство
Научна дисциплина, НД :	Примењене рачунарске науке и информатика
Предметна одредница/Кључне речи, ПО :	Релационе базе података, трансакције, датотеке, слогови, упити
УДК	
Чува се, ЧУ :	У библиотеци Факултета техничких наука, Нови Сад
Важна напомена, ВН :	
Извод, ИЗ :	Имплементација једног система за управљање релационим базама података у програмском језику Java
Датум прихватања теме, ДП :	
Датум одбране, ДО :	17.07.2026
Чланови комисије, КО :	Председник: Др Петар Петровић, ванредни професор
	Члан: Др Марко Марковић, доцент
	Члан:
	Члан, ментор: Др Бранко Милосављевић, редовни професор
	Потпис ментора

	UNIVERSITY OF NOVI SAD • FACULTY OF TECHNICAL SCIENCES 21000 NOVI SAD, Trg Dositeja Obradovića 6	
	KEY WORDS DOCUMENTATION	
Accession number, ANO :		
Identification number, INO :		
Document type, DT :	Monographic publication	
Type of record, TR :	Textual printed material	
Contents code, CC :		
Author, AU :	Luka Bursać	
Mentor, MN :	Branko Milosavljević, Phd., full professor	
Title, Ti :	Management of files, transactions and query interpretation in relational databases	
Language of text, LT :	Serbian	
Language of abstract, LA :	Serbian/English	
Country of publication, CP :	Republic of Serbia	
Locality of publication, LP :	Vojvodina	
Publication year, PY :	2026	
Publisher, PB :	Author's reprint	
Publication place, PP :	Novi Sad, Dositeja Obradovica sq. 6	
Physical description, PD : (chapters/pages/ref./tables/pictures/graphs/appendixes)	11/83/12/11/48/0/2	
Scientific field, SF :	Electrical and Computer Engineering	
Scientific discipline, SD :	Applied computer science and informatics	
Subject/Key words, S/KW :	Relational databases, transactions, files, records, queries	
UC		
Holding data, HD :	The Library of Faculty of Technical Sciences, Novi Sad	
Note, N :		
Abstract, AB :	Implementation of a system for relational database management written in Java	
Accepted by the Scientific Board on, ASB :		
Defended on, DE :	17.07.2026	
Defended Board, DB :	President:	Petar Petrović, Phd., assoc. professor
	Member:	Marko Marković, Phd., asist. professor
	Member:	
	Member, Mentor:	Branko Milosavljević, Phd., full professor
		Menthor's sign



УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА
21000 НОВИ САД, Трг Доситеја Обрадовића 6

ИЗЈАВА О НЕПОСТОЈАЊУ СУКОБА ИНТЕРЕСА

Изјављујем да нисам у сукобу интереса у односу ментор – кандидат и да нисам члан породице (супружник или ванбрачни партнер, родитељ или усвојитељ, дете или усвојеник), повезано лице (крвни сродник ментора/кандидата у правој линији, односно у побочној линији закључно са другим степеном сродства, као ни физичко лице које се према другим основама и околностима може оправдано сматрати интересно повезаним са ментором или кандидатом), односно да нисам зависан/на од ментора/кандидата, да не постоје околности које би могле да утичу на моју непристрасност, нити да стичем било какве користи или погодности за себе или друго лице било позитивним или негативним исходом, као и да немам приватни интерес који утиче, може да утиче или изгледа као да утиче на однос ментор-кандидат.

У Новом Саду, дана _____

Ментор

Кандидат

*Hvala mojim roditeljima, mom kumu i svim prijateljima koji su mi
pružili neverovatne životne prilike i učinili me onim što jesam*

Sadržaj

1	Uvod	1
1.1	O sistemu	1
1.2	Klijentsko-serverska arhitektura	2
1.3	Sistem za obradu upita	2
2	Upravljanje datotekama	3
2.1	Upravljanje blokovima	3
2.1.1	Interfejs ka <i>file</i> sistemu operativnog sistema	3
2.1.2	Stranice	3
2.1.3	Upravljanje <i>log</i> datotekom	4
2.2	Upravljanje baferima	5
2.2.1	Algoritmi izbora bafera za smenu	6
2.2.1.1	<i>Naive</i>	6
2.2.1.2	<i>FIFO</i>	6
2.2.1.3	<i>LRU</i>	6
2.2.1.4	<i>Clock</i>	6
2.2.1.5	<i>First unmodified</i>	6
2.2.1.6	<i>LRM</i>	6
2.3	Slogovi	6
2.3.1	Mehanizmi definisanja strukture sloga	7
2.3.1.1	Šema	7
2.3.1.2	Raspored polja	7
2.4	Primena strukture slogova na blokove	8
2.4.1	Stranica slogova	8
3	Upravljanje transakcijama	9
3.1	Realizacija transakcionih mehanizama u sistemu	9
3.1.1	Transakcije kao glavno mesto pristupa vrednostima	9
3.1.2	Sistem za oporavak	10
3.1.2.1	Sistem <i>log</i> -ovanja	10
3.1.2.2	Algoritmi oporavka sistema	12
3.1.2.3	Algoritam poništavanja transakcije	13
3.1.3	Bezbedan višenitni pristup	13
3.1.3.1	Izolacioni nivoi transakcija	14
3.1.3.2	Katalog katanaca	15
3.1.3.3	Upravljanje katancima na nivou individualnih transakcija . . .	16
3.2	Transakcije i klijenti	16

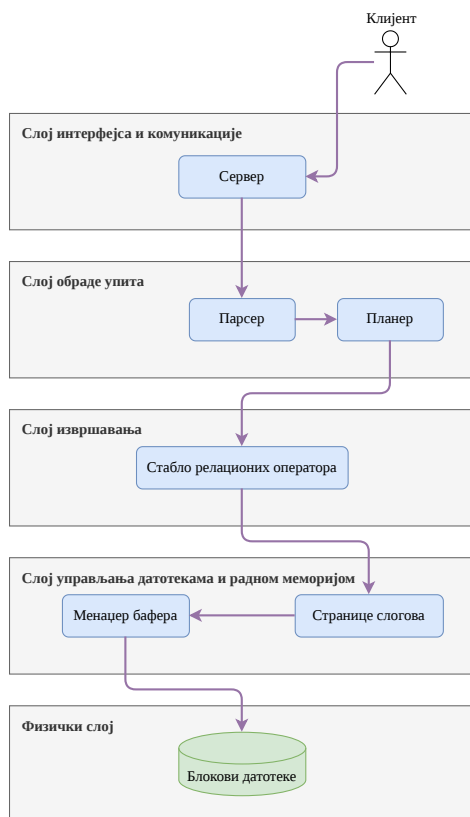
3.2.1	Algoritam zapisa mirne kontrolne tačke	16
4	Metapodaci	17
4.1	Kataloške tabele	17
4.2	Statistički podaci	18
4.2.1	Računanje statističkih podataka	18
4.3	Pristup metapodacima	18
5	Relacioni operatori	19
5.1	Virtuelna mašina	19
5.1.1	Konstante	19
5.1.2	Izrazi	20
5.1.3	Članovi	21
5.1.4	Predikati	21
5.1.5	Generalizacija evaluacije	21
5.2	Struktura relacionih operatora u sistemu	22
5.2.1	Protočna obrada	22
5.2.2	Hijerarhija implementacije relacionih operatora	22
5.2.2.1	Scan	23
5.2.2.2	UpdateScan	24
5.2.2.3	UnaryScan	24
5.2.2.4	UnaryUpdateScan	24
5.2.2.5	BinaryScan	24
5.2.2.6	DiffSchemaJoinContextScan	24
5.2.2.7	TableScan	24
5.2.2.8	DummyTableScan	24
5.2.2.9	SelectScan	25
5.2.2.10	ExtendProjectScan	25
5.2.2.11	RenameScan	25
5.2.2.12	ProductScan	25
5.2.2.13	UnionAllScan	25
5.2.3	Primeri	26
5.2.3.1	Primer izgleda komponenti virtuelne mašine	26
5.2.3.2	Primer poziva <code>getValue()</code> metode	27
5.2.3.3	Primer poziva <code>next()</code> metode	28
6	Parsiranje	29
6.1	Tokenizator	29
6.2	Parser	30
6.2.1	Iskazi	30
6.2.2	Sintaksičke kategorije <i>SQL</i> operacija	30
6.2.2.1	Parse	30

6.2.2.2	ParseUpdate	31
6.2.2.3	ParseSelect	31
6.2.2.4	ParseInsert	32
6.2.2.5	ParseDelete	32
6.2.2.6	ParseCreateTable	33
6.2.2.7	ParsePredicate	33
6.2.2.8	ParseExpression	34
6.2.2.9	ParseEvaluable	36
7	Planiranje	37
7.1	Struktura klasa planova u sistemu	37
7.1.1	Hijerarhija implementacije planova	37
7.1.1.1	Plan	39
7.1.1.2	TablePlan	39
7.1.1.3	DummyTablePlan	39
7.1.1.4	SelectPlan	39
7.1.1.5	ExtendProjectPlan	41
7.1.1.6	RenamePlan	42
7.1.1.7	ProductPlan	42
7.1.1.8	UnionAllPlan	43
7.2	Planer	44
7.2.1	Evaluacija izraza tokom planiranja	44
7.2.2	Ulazna tačka kreiranja i izvršavanja planova	44
7.2.3	Planiranje <i>read-only</i> naredbi	46
7.2.3.1	Algoritam planiranja SELECT naredbi	46
7.2.3.2	Automatsko generisanje opisa planova	50
7.2.4	Planiranje modifikacionih naredbi	51
7.2.4.1	Algoritam planiranja INSERT naredbe	51
7.2.4.2	Algoritam planiranja UPDATE naredbe	52
7.2.4.3	Algoritam planiranja DELETE naredbe	52
7.2.4.4	Algoritam planiranja CREATE TABLE naredbe	52
8	Klijentsko-serverska arhitektura	53
8.1	Komunikacija sa klijentima	53
8.1.1	Odgovori sistema	53
8.1.2	Protokol serijalizacije	54
8.2	Serverski sloj	55
8.2.1	Rukovođenje klijentima	55
8.2.1.1	Obrada jednog klijenta	56
8.2.2	Pisanje kontrolnih tačaka	57
8.2.3	Gašenje sistema	57

8.3	Klijentski sloj	58
8.3.1	Štampanje tabela	58
8.3.2	Masovno pokretanje naredbi	58
9	Testovi	59
9.1	Organizacija testova	59
9.1.1	Testno okruženje	59
9.1.2	Sistem izolacije direktorijuma na disku	60
9.1.3	Sistem izolacije direktorijuma u radnoj memoriji	60
9.2	Funkcionalni testovi	60
9.3	Testiranje performansi	61
9.3.1	Testovi performanse algoritama smene bafera	61
9.3.2	Testovi performanse <i>RBO</i> tehnike u planiranju <i>SELECT</i> naredbi	64
10	Pregled sistema	65
10.1	Izgradnja i pokretanje	65
10.1.1	Rukovođenje zavisnostima	65
10.1.1.1	Zavisnosti servera	65
10.1.1.2	Zavisnosti klijenta	66
10.1.1.3	Zavisnosti testnog okruženja	66
10.2	Sistemska konfiguracija	66
10.3	Integracija sa <i>GitHub</i> platformom	67
10.4	Primer funkcionisanja celokupnog sistema	67
11	Zaključak	69
11.1	Motivacija	69
11.2	Ograničenja sistema	69
11.2.1	Implementiran je samo podskup <i>SQL</i> -a	69
11.2.2	Ograničenje na slogove fiksne dužine	70
11.2.3	Sporo računanje statističkih metapodataka	71
11.2.4	Nedostajuća implementacija indeksnih struktura podataka	71
11.2.5	Neoptimalni planer	71
11.2.6	Ulančavanje članova predikata je moguće samo konjunkcijom	72
11.3	Razlike između osnovne implementacije i <i>LBDB</i> sistema	72
	Spisak slika	73
	Spisak listinga	75
	Spisak tabela	77
	Spisak korišćenih skraćenica	79
	Spisak korišćenih pojmova	81
	Литература	83

1.1 O sistemu

LBDB je višekorisniški sistem sa transakcijama za upravljanje relacionim bazama podataka. Njegova svrha je da prima naredbe dobijene od klijenata, interpretira ih po standardu *SQL* programskog jezika i da vrati rezultat tim klijentima. Rezultat može biti broj pogodjenih slogova za slučaj modifikacionih operacija ili rezultujuća tabela za slučaj upita. Zbog efikasnog interpretiranja, potrebno je obezbediti algoritme i strukture podataka za sledeće module: upravljanje datotekama, rad sa transakcijama, rad sa metapodacima, parsiranje upita, pravljenje planova izvršavanja upita, izvršavanje upita i za mrežnu komunikaciju. Moduli ovog sistema su raspoređeni tako da se svaki brine o jednoj grupi algoritama i struktura podataka kroz koju upit prolazi.



Слика 1: Упрошћени дијаграм слојевите архитектуре система

Svaki sloj iz uprošćene arhitekture sistema je posebno detaljno objašnjen u nastavku, a uz te slojeve se dodatno objašnjavaju i transakcije, koje su isprepletene kroz ceo sistem. Sličan dijagram koji opisuje celu arhitekturu sistema, ali mnogo detaljnije, se može pronaći na slici 33.

Osnovna struktura i algoritmi su izvedeni iz knjige *Database Design And Implementation* [1], a njihova unapređenja su deo ovog rada. Knjiga definiše zadatke na kraju svakog modula i ti zadaci su osnova za unapređivanje sistema. Pregled znatnih razlika između osnovne implementacije i *LBDB* sistema se može pronaći u sekciji 11.3, a detaljan spisak urađenih zadataka i njihovih beleški se može pronaći u okviru repozitorijuma¹.

1.2 Klijentsko-serverska arhitektura

LBDB sistem se pokreće kao server i njemu se pristupa preko mreže i specijalnog protokola. Postoji implementacija klijentske aplikacije koja implementira ovaj protokol. Detalji se mogu pronaći u poglavlju 8 koje opisuje klijentsko-serversku arhitekturu sistema.

1.3 Sistem za obradu upita

LBDB klasa predstavlja najviši apstrakcioni nivo sistema obrade upita na koji se server oslanja. Služi za orkestraciju upravljača glavnih podsistema: menadžer transakcija (sekcija 3.2), menadžer metapodataka (sekcija 4.3) i planer (sekcija 7.2.2). Takođe, klasa *LBDB* je odgovorna za logiku inicijalizacije i oporavljanja sistema od neočekivanog gašenja, za određeni direktorijum baze podataka.

¹<https://github.com/lukascobbler/lbdb>

Глава 2

Upravljanje datotekama

Upravljanje datotekama se vrši kroz više slojeva u sistemu, gde je svaki sloj odgovoran za organizaciju datoteka na različitom apstrakcionom nivou. Osnovna premisa je da sve operacije sa datotekama, to jest diskom moraju da se izvršavaju u jedinicama blokova, jer je operativni sistem, a i hardver (disk) optimizovan za rad sa njima [1]. Zbog toga što je blok najmanja jedinica interakcije sa diskom i datotekama, sva čitanja, pisanja i modifikacije podataka se rade zajedno sa celim blokom gde se ti podaci nalaze, a ne direktno.

2.1 Upravljanje blokovima

Sistem za upravljanje blokovima je primarno zadužen za dobavljanje bloka sa diska gde se nalaze traženi podaci i za korektno zapisivanje *log* podataka. Svaki blok ima svoj unikatni identifikator, koji je predstavljen *Java record* strukturom. Svaki blok je vezan za datoteku i ima svoju blok poziciju u toj datoteci.

2.1.1 Interfejs ka *file* sistemu operativnog sistema

Najniži nivo apstrakcije predstavlja menadžer datoteka (*FileManager* klasa) koji ima funkciju interfejsa ka *file* sistemu operativnog sistema i nema predstavu šta se nalazi u samim datotekama *LBDB* sistema. Menadžer datoteka čuva pokazivače na sve datoteke kojima sistem upravlja i omogućava višenitni bezbedan pristup istim, upotrebom *Java synchronized* ključne reči. Višenitni bezbedan pristup omogućava sistemu da podrži više različitih klijenata u isto vreme, ali nije dovoljan samo na ovom sloju, već je detaljno obrađen u okviru transakcija (sekcija 3.1.3).

Moguće je podesiti sistem da koristi proizvoljnu veličinu jednog bloka, zavisno od prirode podataka kojima će baza podataka biti popunjena i to je glavni parametar menadžera datoteka.

2.1.2 Stranice

Stranica (eng. *page*) predstavlja sirove bajtove jednog bloka učitane u radnu memoriju. Iako nije nužno potrebno, sve vrednosti iz stranice zajedno sa njihovim tipom se pretvaraju u ekvivalentne *Java* objekte (koristeći *ByteBuffer* standardnu *Java* klasu) da bi se omogućio pristup operacijama iz *Java* standardne biblioteke za te tipove. Kada menadžer datoteka dobavlja određeni blok, bajtovi tog bloka se smeštaju u radnu memoriju u objekat stranice. Stranice su na apstrakcionom nivou ispod sistema baferovanja i koriste se kao potpora tog sistema.

Sistem podržava sledeće proste i kompozitne tipove podataka: *String*, *Boolean* i *Integer*. Kod prostih tipova, iz stranice se samo čitaju njihovi bajtovi i pretvaraju u određeni *Java* objekat tog tipa, dok kod kompozitnih tipova kao što je *String* potrebno je znati dužinu, same bajtove i kodiranje da bi se konstruisao *Java* objekat *String* tipa.

2.1.3 Upravljanje log datotekom

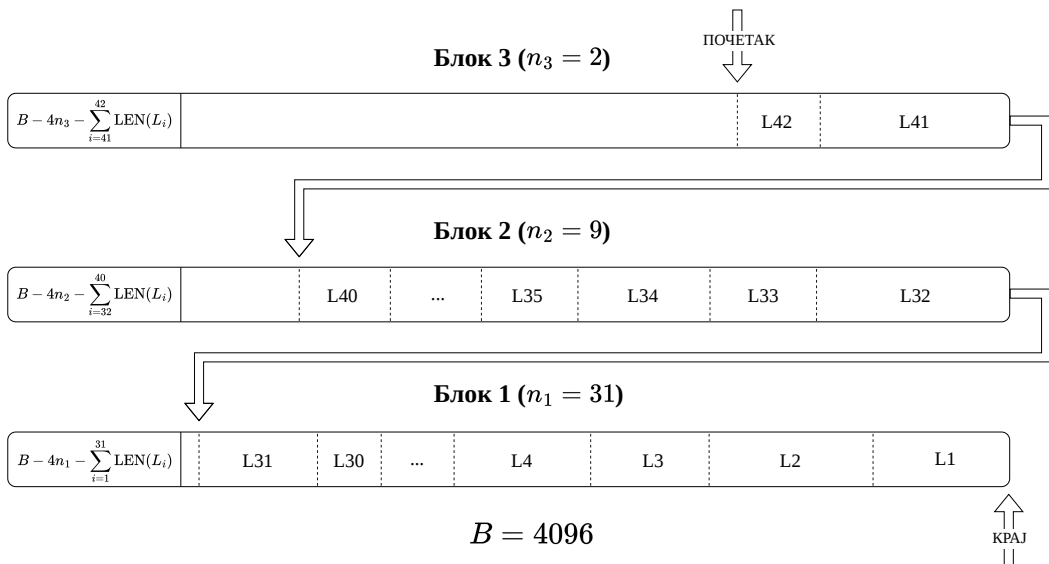
Jedan *log* je niz bajtova čije interpretiranje ukazuje na to kako se promenila neka vrednost u nekom bloku.

Log datoteka je specijalna vrsta datoteke u kojoj se čuva niz *log*-ova. Na apstrakcionom nivou upravljanja blokovima, nije bitan sadržaj ove datoteke, već samo algoritmi neophodni za korektno prolaženje kroz nju i algoritmi za dodavanje novih *log*-ova. Ovi algoritmi se nalaze u menadžeru *log*-ova (*LogManager* klasa) i *log* iteratoru (*LogIterator* klasa).

Log datoteka je organizovana tako da se noviji *log*-ovi nalaze u blokovima bližim kraju datoteke, a unutar jednog bloka *log* datoteke noviji *log*-ovi se nalaze pri početku bloka. Ako nema mesta da se upiše novi *log*, prelazi se u sledeći blok *log* datoteke.

Menadžer *log*-ova obezbeđuje algoritme upravljanja *log* datotekom tako što čuva stranicu poslednjeg bloka *log* datoteke. Upravljanje *log* datotekom podrazumeva dodavanje novih *log*-ova, upis *log*-ova na disk i arhiviranje *log* datoteke.

Iterator *log*-ova obezbeđuje čitanje *log* datoteke u korektnom redosledu i implementiran je pomoću *Java* *Iterator* interfejsa.



Слика 2: Izgled *log* datoteke i smer iteracije

Svaki blok *log* datoteke na nultoj poziciji sadrži četvorobajtni broj koji predstavlja lokaciju prvog *log*-a u tom bloku. Svaki *log* se sastoji iz sadržaja i svoje dužine koja je isto četvorobajtni broj.

Prilikom dodavanja novog *log*-a, sistem izračunava *log* sekvencu novog *log*-a (eng. *log sequence number*) koja unikatno identifikuje zapisan *log* i može se koristiti u daljim podsistemima za forsiranje pisanja prethodnih *log*-ova u *log* datoteku i time garantovati redosled operacija.

2.2 Upravljanje baferima

Čišćenje stranice iz memorije nakon završetka operacije koja ju je koristila može biti jako neefikasno jer se za ponovni pristup tom bloku mora odlaziti do diska, pogotovo u višekorisničkim kontekstima i situacijama kada se istim blokovima često pristupa. Zbog toga se uvodi koncept bafera (eng. *buffer*) koji, uz algoritme u menadžeru bafera (*BufferManager* klasa), omogućava stranicama da ostanu u memoriji prilagodljivu količinu vremena. Posledica ovog sistema je da direktan pristup podacima sa diska više nije moguć, već se sve operacije obavljaju kroz bafere.

Buffer klasa enkapsulira učitane stranice i sadrži dodatne podatke koji se mogu iskoristiti za implementaciju raznih algoritama ubrzanja sistema:

- kada je stranica učitana
- kada je bilo poslednje pristupanje stranici
- koji je redni broj bafera u listi bafera
- koja transakcija je poslednja modifikovala tu stranicu i *log* sekvenca poslednje operacije
- koliko transakcija trenutno upotrebljava tu stranicu, kraće rečeno broj pinova bafera

Pošto je količina radne memorije ograničena, i količina bafera u sistemu isto mora biti ograničena. Menadžer bafera ima listu bafera sa kojima raspolaže i potrebno je da poveća stepen iskorišćenosti bafera iz te liste što je više moguće. U idealnom, hipotetičkom scenariju, menadžer bafera bi predvideo budućnost i znao kojim baferima bi se sledeće pristupalo i izbacio iz memorije one koji su vremenski najdalje od pristupa. Ovaj scenario je očigledno nemoguć, pa je potrebno iskoristiti algoritme koji najbolje “predviđaju budućnost” na osnovu realnih podataka. Kada se bafer izbaci iz memorije, njegov sadržaj se piše u blok za koji je vezan i na taj način se podaci perzistiraju. Postoje i druge situacije (sekcija 3.1.2.2.2) kada se sadržaj bafera odmah zapisuje na disk.

Da bi se bafer izbacio iz memorije, ne sme da bude deo ni jedne aktuelne transakcije, to jest broj pinova mu mora biti nula. Postoje dva scenarija kada stranica koja do sada nije bila u memoriji treba da se mapira na bafer:

- U slučaju da ne postoji nijedan bafer sa nula pinova, nova stranica čeka određeni vremenski period da se oslobodi neki bafer i ako se nijedan bafer ne oslobodi, vraća se greška klijentu (detaljnije u sekciji 3.1.3.2).

- U slučaju da postoji više bafera sa nula pinova, potrebno je izabrati koji će biti izbačen iz radne memorije pomoću algoritma izbora.

2.2.1 Algoritmi izbora bafera za smenu

U optičaju je nekoliko algoritama [1] za izbor bafera koji će biti smenjen. Potvrda svih tvrdnji o performansama se može pronaći u [1] i u sekciji 9.3.1.

2.2.1.1 *Naive*

Naivni algoritam, kako mu i ime kaže, nema nikakvu logiku izbora bafera kojeg će smeniti već samo uzima prvi bafer koji ima nula pinova. Očekivane su loše performanse jer je velika šansa da će se smeniti bafer koji će se uskoro opet koristiti.

2.2.1.2 *FIFO*

FIFO (eng. *first in first out*) algoritam smenjuje bafer koji je najranije ušao u listu bafera. Performanse su bolje od naivnog algoritma ali *FIFO* algoritam će smeniti i jako često korišćene bafere iako su najranije ušli u sistem, na primer baferi gde se čuvaju blokovi metapodataka sistema.

2.2.1.3 *LRU*

LRU (eng. *least recently used*) algoritam smenjuje bafer koji je najdavnije korišćen. Odlične performanse jer ako bafer dugo nije korišćen, verovatno se neće još dugo ni koristiti [1].

2.2.1.4 *Clock*

Clock algoritam pretpostavlja da baferi imaju fiksne pozicije u listi bafera i smenjuje prvi bafer koji ima nula pinova, ali pretragu počinje od prethodnog smenjenog bafera. Performanse su odlične jer je šansa da je bitan bafer pinovan velika [1].

2.2.1.5 *First unmodified*

First unmodified algoritam smenjuje prvi bafer koji pronađe da nije modifikovan i da ima nula pinova, ili ako je svaki modifikovan prvi koji ima nula pinova. Performansa može biti bolja od naivnog algoritma, ali može se desiti da modifikovan bafer neće dugo biti korišćen što znači da nije izabran optimalan bafer za smenu.

2.2.1.6 *LRM*

LRM (eng. *least recently modified*) algoritam smenjuje bafer koji ima nula pinova i koji je poslednje izmenjen, to jest bafer sa najmanjim brojem *log* sekvence. Pretpostavka je da modifikovani bafer neće ponovo biti korišćen neko vreme jer je transakcija već završila. Može biti bolji od naivnog algoritma u specifičnim situacijama, a često je lošiji od njega.

2.3 Slogovi

Podsistem za upravljanje baferima je generalan i ne pruža nikakvu strukturu podataka unutar samih bafera, odnosno blokova. Na nivou relacione baze podataka, najmanja jedinica

interakcije nije jedan blok, već jedan slog neke tabele i potrebno je blokove organizovati tako da se to omogući. Jedan slog neke tabele je isto što i jedan red te tabele što je objašnjeno detaljnije u sekciji 5.

2.3.1 Mehanizmi definisanja strukture sloga

Da bi se podržalo kreiranje perzistentne strukture jednog sloga nove tabele, potrebno je definisati mehanizme kojima će klijenti opisivati tu strukturu. Na apstrakcionom nivou upravljanja datotekama, sistem se samo brine o tome da je teoretska i fizička struktura ispoštovana, a perzistiranje i samo kreiranje strukture je zadatak viših podsistema.

2.3.1.1 Šema

Svaki slog jedne tabele se sastoji od istih metapodataka, to jest istih kolona. Svaka kolona se opisuje svojim tipom, svojom dužinom na disku i tome da li može sadržati *NULL* vrednosti.

Svaki od tipova je definisan u *SQL Java* standardnoj biblioteci, ali korišćenje tih vrednosti direktno može dovesti do nekompletnosti na raznim mestima gde su tipovi korišćeni u sistemu, pa je zbog toga uvedena enumeracija koja striktno definiše podržane tipove, zajedno sa njihovom podrazumevanom dužinom u bajtovima. Tip *VARCHAR*, to jest *String* nema podrazumevanu dužinu jer je različita za svako polje. Dodatno postoji i *NULL* tip koji označava odsustvo vrednosti za to polje.

```
import java.sql.Types;
public enum DatabaseType {
    INT(Types.INTEGER, 4), BOOLEAN(Types.BOOLEAN, 1),
    VARCHAR(Types.VARCHAR, -1), NULL(Types.NULL, 0);
    ...
}
```

Листинг 1: Podržani tipovi kolona

Šema jedne tabele je skup podataka o kolonama te tabele zajedno sa imenima tih kolona. Bitno je napomenuti da tabele u svojoj osnovnoj definiciji predstavljaju podatke koje se nalaze u datotekama, ali to nije uvek slučaj. Postoje i virtuelne tabele koje su rezultati upita i mogu, ali ne moraju da se direktno mapiraju na tabele koje se nalaze u datotekama. Moguće je kombinovati više tabela u jednu tabelu i dodati virtuelne kolone (sekcija 5.2.2.10) koje se ne nalaze u datoteci već su njihove vrednosti izvedene na osnovu neke kalkulacije.

2.3.1.2 Raspored polja

Šema predstavlja teoretski izgled jedne tabele, ali to nije dovoljno da bi se taj izgled perzistirao i mogao ponovo rekreirati. Zbog toga je potrebno uvesti mehanizam pamćenja i fizičkih karakteristika kolona tabele (postoji samo za nevirtuelne tabele). Taj mehanizam se realizuje preko rasporeda polja (eng. *layout*).

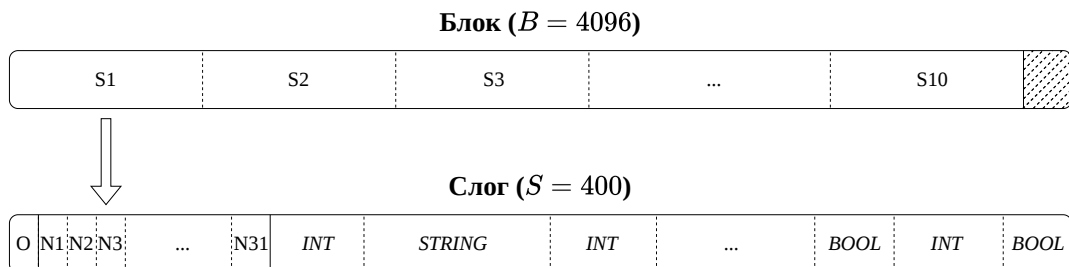
Za svaku kolonu pamte se sledeće fizičke karakteristike: pozicija početka vrednosti te kolone, maksimalna dužina vrednosti te kolone i pozicija te kolone u šemi. Takođe, pamti se i celokupna dužina celog sloga. Kolone se identifikuju pomoću njihovog naziva.

2.4 Primena strukture slogova na blokove

Nakon definisanja fizičke strukture sloga tabele, potrebno je primeniti tu fizičku strukturu na blokove datoteka. U *LBDB* sistemu, jedan blok sadrži fiksni broj slogova koji su svi iz iste tabele i ne postoje vrednosti promenjive dužine (više o ovom ograničenju u sekciji 11.2.2).

Pošto su svi slogovi iste dužine, $\frac{B}{S}$ slogova staje u jedan blok, gde B predstavlja dužinu bloka u sistemu, S predstavlja dužinu jednog sloga te tabele, a $B - S * \lfloor \frac{B}{S} \rfloor$ prostora ostaje neiskorišćeno (sve vrednosti su u bajtovima). Slogovi u blokovima čuvaju samo vrednosti kolona, ali ne i metapodatke tih kolona. Podsystem upravljanja datotekama se ne brine o metapodacima kolona, već za to postoji poseban podsystem (sekcija 4).

Ipak, u okviru jednog sloga se čuvaju metapodaci o tome koje vrednosti nisu prisutne, to jest imaju *NULL* vrednost i to da li je slog obrisan. Rezerviše se četvorobajtno zaglavlje na početku svakog sloga i njegovi bitovi predstavljaju ove metapodatke. Da li je slog označen kao obrisan se predstavlja prvim bitom (O), dok ostalih 31 bitova (N_i) označavaju da li polje na toj poziciji ima *NULL* vrednost. Zbog ovoga postoji ograničenje na broj polja po tabeli, maksimalno 31 polje.



Слика 3: Izgled jednog bloka popunjenog slogovima

2.4.1 Stranica slogova

RecordPage klasa enkapsulira svu logiku održavanja strukture individualnog bloka tako što pruža interfejs za postavljanje vrednosti samo na osnovu imena kolone i broja sloga u tom bloku. Takođe, pruža interfejs za postavljanje *NULL* vrednosti i pretragu slobodnih ili zauzetih slogova u bloku za koji je povezana. Za pristup svim blokovima jedne tabele, potrebno je sukcesivno konstruisati objekte *RecordPage* klase, što je posao podsystema relacionih operatora (sekcija 5.2.2.7).

Bitno je napomenuti da logika stranice slogova za postavljanje vrednosti ne radi samo postavljanje vrednosti, već računa gde ta vrednost treba biti postavljena. Postavljanje vrednosti delegira sistemu transakcija.

Глава 3

Upravljanje transakcijama

Transakcije su osnovni mehanizam za garantovanje različitih osobina otpornosti sistema za upravljanje relacionim bazama podataka. Omogućavaju sistemu da vrši atomične operacije, da se uvek održi u konzistentnom stanju, da izoluje nezavisne konkurentne operacije i da garantuje perzistentnost podataka. Generalno, ove osobine se zovu *ACID* osobine (eng. *atomicity, consistency, isolation, durability*) [2].

Svaka operacija u sistemu mora biti izvršena u okviru jedne transakcije, ali se jedna transakcija može sastojati i od više operacija koje se sve moraju ili uspešno izvršiti ili se moraju sve poništiti. Svaka transakcija ima početak i kraj. Kraj može biti potvrda (eng. *commit*) ili poništavanje (eng. *rollback*). Ako je transakcija uspešno potvrđena, od tog trenutka pa na dalje sve izvršene promene moraju biti odmah vidljive drugim transakcijama.

3.1 Realizacija transakcionih mehanizama u sistemu

Vrednost je niz bajtova (određenog tipa), koja dobija semantički značaj tek na apstrakcionim nivoima iznad nivoa transakcija, naime na nivou organizovanja blokova u slogove. Na nivou transakcija, vrednosti nemaju semantičko značenje, ali da bi sistem obezbedio visok stepen usklađenosti sa *ACID* osobinama, operacije nad vrednostima se obavljaju isključivo kroz transakcije koje enkapsuliraju svu potrebnu logiku tih osobina.

3.1.1 Transakcije kao glavno mesto pristupa vrednostima

Na nivou slogova, operacije se izvršavaju u celinama jednog sloga, ali gradivni elementi tih slogova su ipak vrednosti. Stranica slogova upravlja pozicijama vrednosti, a sam pristup njima delegira transakciji za koju je vezana.

Pošto blokovi gde se vrednosti nalaze moraju biti u memoriji da bi se njima pristupalo, potrebno je učitati ih u sistemske bafere. Podsystem za upravljanje baferima je svestan samo načina mapiranja blokova na bafere, ali nema nikakve garancije o redosledu pristupa, ne čuva od konfliktujućih operacija i ne prati istoriju izmena. To je posao transakcija.

Svi baferi u radnoj memoriji sistema se grupišu u transakcije, tako što svaka transakcija čuva listu svojih bafera i radi operacije samo nad njima. Bafer ulazi u listu neke transakcije tako što ga transakcija pinuje, i time označava da se sadržaj tog bafera ne sme vratiti na disk dok transakcija ne završi sa njim (dok ne prekine baferov pin). Stranica slogova i transakcija rade u paru: stranica slogova zna kojem bloku želi da pristupi, a transakcija zna kako da obezbedi da operacije nad baferom tog bloka ispoštuju *ACID* osobine.

3.1.2 Sistem za oporavak

Sistem za oporavak je podsistem u okviru transakcija koji sadrži prvi deo logike zbog kojeg se sav pristup vrednostima radi kroz transakcije. Primarno se brine o oporavku sistema, ali se brine i o poništavanju operacija jedne transakcije (eng. *rollback*). Obuhvata **ACID** (eng. *atomicity, consistency, durability*) osobine.

LBDB sistem je softversko rešenje koje radi u kontekstu nekog hardvera i operativnog sistema. Svaki od slojeva na koje se *LBDB* sistem oslanja, ima mogućnost da zakaže zbog nekog faktora koji je izvan kontrole *LBDB* sistema. Primeri su nestajanje struje, ubijanje *LBDB* serverskog procesa ili neuspešno pisanje na disk. Ako se u trenutku zakazivanja izvršava neka operacija koja menja podatke u sistemu, ti podaci će biti izgubljeni, a sistem će biti ostavljen u nekonzistentnom stanju. Korišćenje sistema koji je u nekonzistentnom stanju može dovesti do lošeg tumačenja podataka, ali može biti i opasno u pojedinim situacijama. Zbog ovoga se uvodi podsistem koji oporavlja sistem od nekonzistentnog stanja.

Konzistentno stanje se može definisati sa sledeće dve osobine [1], [2]:

- sve nedovršene transakcije su poništene,
- sve modifikacije potvrđenih (eng. *committed*) transakcija moraju biti na disku

3.1.2.1 Sistem *log*-ovanja

Log podaci su u *log* datoteci uvek poređani redom kojim su se izvršili u okviru jedne transakcije. Pisanjem *log* podataka se postiže visoko granulirana istorija izmena. Osnova svakog algoritma oporavka je prolaženje kroz istoriju izmena počevši od poslednje izmene, da bi se te izmene poništile ili ponovo primenile tačnim redosledom.

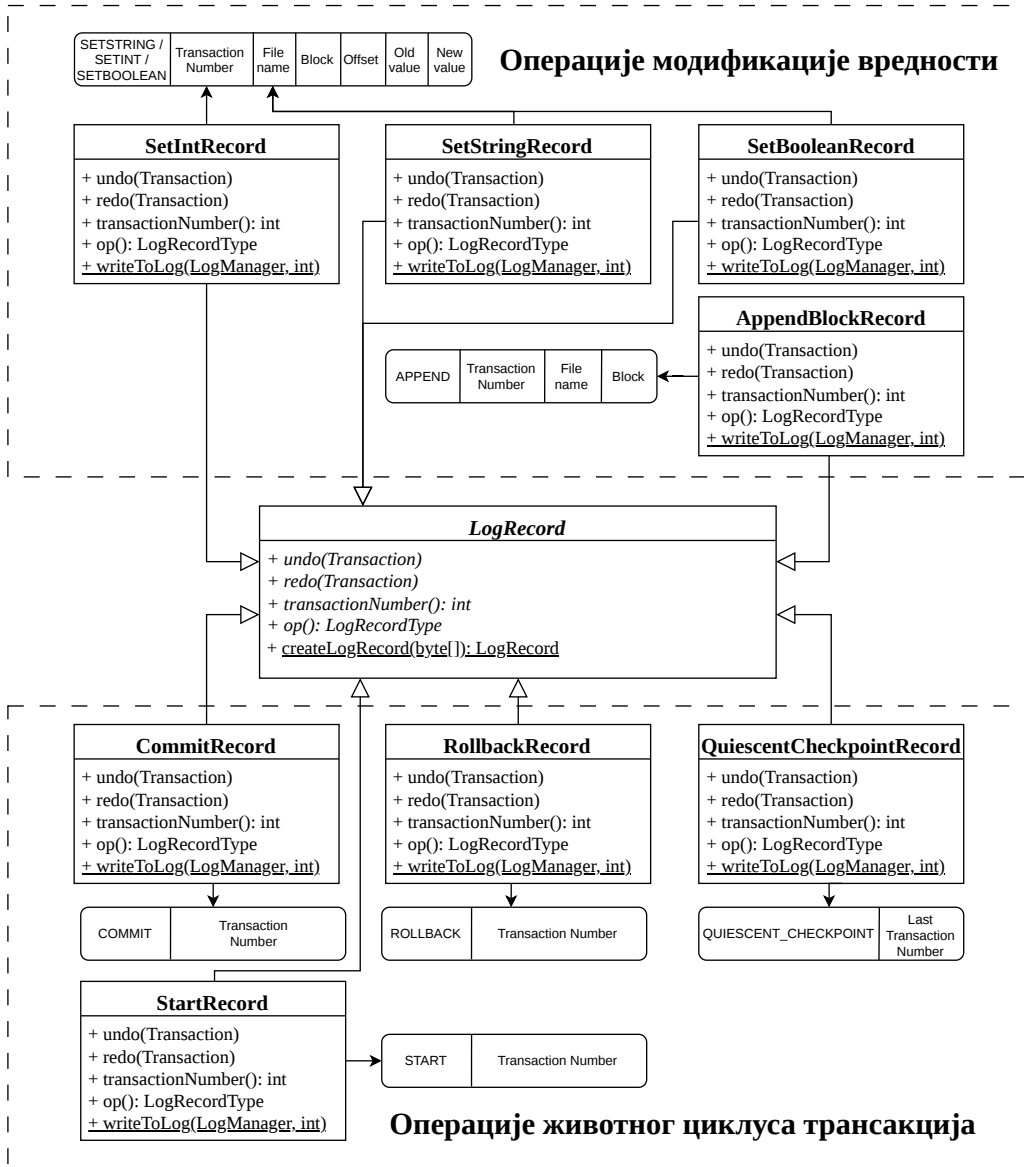
U sistemu postoji dve glavne grupe operacija: operacije modifikacije vrednosti i operacije životnog ciklusa transakcija. Za svaku operaciju se definiše struktura *log*-a koja opisuje tu operaciju. Za operacije modifikacije vrednosti, u *log*-u stoje svi podaci neophodni da se ta operacija poništi ili ponovo primeni, dok u *log*-ovima operacija životnog ciklusa transakcija stoje svi neophodni podaci za praćenje rada transakcija.

Pošto u *log*-ovima operacija modifikacije vrednosti uvek stoje i stara i nova vrednost, poništavanje ili ponovna primena operacije je trivijalna. Specijalna operacija modifikacije je operacija umetanja novog bloka na kraju datoteke, koja ne menja nikakvu vrednost ali je i dalje potrebno pratiti njene pozive zbog održavanja korektne veličine datoteka. Algoritam poništavanja umetanja novog bloka nije trivijalan jer nije samo zamena vrednosti, već je potrebno označiti bafer tog bloka kao nemodifikovan i skratiti datoteku za jedan blok. U *log* datoteku se uvek zapisuje novi *log* za pozvanu operaciju modifikacije pre same obrade te operacije.

Operacije životnog ciklusa transakcija je potrebno pratiti da bi sistem oporavka znao koje transakcije su se uspešno i neuspešno izvršile i na osnovu toga reagovao na operacije

modifikacije. Marker kontrolne tačke je specijalni zapis koji označava kada nije potrebno dalje prolaziti kroz *log* datoteku.

Hijerarhija *log* zapisa počinje od glavnog interfejsa *LogRecord* koji definiše neophodne operacije koje će algoritmi oporavka sistema zvati. Operacije životnog ciklusa transakcija ostavljaju praznu implementaciju *undo* i *redo* metoda jer ne modifikuju podatke. Dodatno, svaki tip *log* zapisa ima prateću statičku metodu *writeToLog* koja enkapsulira logiku pisanja same strukture konkretnog *log*-a preko menadžera *log*-ova za neki identifikator transakcije.



Слика 4: Operacije, njihovi *log* tipovi i struktura *log* tipova

3.1.2.2 Algoritmi oporavka sistema

Algoritam oporavka sistema je procedura koja prati korake definisane strategijom oporavka koju sistem koristi i zajedno sa podacima iz *log* datoteke vraća sistem u konzistentno stanje. Svaki algoritam oporavka mora biti idempotentan, jer sistem može naglo prestati sa radom i dok je u procesu oporavljanja [1]. Proces oporavka se vrši u okviru podizanja sistema.

Postoje tri generalna algoritma oporavka sistema [1]:

- ponovna primena uspešnih transakcija i poništavanje neuspešnih transakcija (eng. *undo redo recovery*),
- samo poništavanje neuspešnih transakcija (eng. *undo only recovery*),
- samo ponovna primena uspešnih transakcija (eng. *redo only recovery*).

Izbor algoritma oporavka utiče na to kada će sadržaj bafera biti upisan na disk.

U *LBDB* sistemu, implementirani su *undo redo* i *undo only* algoritmi oporavke i moguće je postaviti koji će sistem koristiti u okviru sistemskih podešavanja (listing 15).

3.1.2.2.1 Undo redo recovery

Algoritam se deli na dva koraka: faza poništavanja i faza ponovne primene.

Faza poništavanja (eng. *undo*):

- Za svaki log zapis (čitajući unazad od kraja *log-a*):
 - ▶ Ako je tekući zapis *commit* zapis, dodaj tu transakciju u listu potvrđenih transakcija.
 - ▶ Ako je tekući zapis *rollback* zapis, dodaj tu transakciju u listu poništenih transakcija.
 - ▶ Ako je tekući zapis *update* zapis (modifikaciona operacija), i transakcija nije ni u listi potvrđenih ni u listi poništenih, vrati staru vrednost na zadatoj lokaciji.

Faza ponovne primene (eng. *redo*):

- Za svaki log zapis (čitajući unapred od početka *log-a*):
 - ▶ Ako je tekući zapis *update* zapis i transakcija je u listi potvrđenih transakcija, upiši novu vrednost na zadatoj lokaciji.

Glavna prednost ovog algoritma je to što ne utiče na redosled pisanja bafera na disk, ali mana je to što je proces oporavka sporiji. Ako se pretpostavi da su iznenadna gašenja sistema retka, prednosti ovog algoritma pobeđuju ostale algoritme.

3.1.2.2.2 Undo only recovery

Undo only recovery algoritam radi samo fazu poništavanja jer je siguran da su svi baferi *committed* transakcija već zapisani na disku. Ovo se postiže modifikacijom *commit* algoritma tako da se prvo upisuje sadržaj bafera na disk pre pisanja *commit log-a*. Glavna prednost ovog algoritma je brzina, jer se kroz *log* datoteku prolazi samo jednom, ali mana je to što *commit* operacija postaje mnogo sporija.

3.1.2.2.3 *Redo only recovery*

Redo only recovery algoritam radi samo fazu ponovne primene jer je siguran da baferi transakcija koje nisu ni potvrđene ni poništene sigurno nisu zapisani na disk. Ovo se postiže modifikacijom transakcija tako da su baferi u njihovim listama pinovani dokle god se transakcija ne završi. Glavna prednost ovog algoritma je brzina jer se kroz *log* datoteku prolazi samo jednom, ali mana je to što baferi ostaju pinovani mnogo duže, drastično usporavajući sistem.

3.1.2.2.4 *Mirna kontrolna tačka*

Što se sistem duže koristi, to će njegov *log* datoteka postajati obimniji jer uvek sadrži kompletnu istoriju izmena podataka. Prolazak kroz celu *log* datoteku prilikom svakog pokretanja sistema može trošiti nepotrebno mnogo resursa, a u jednom trenutku će i biti nemoguće.

Trenutak kada algoritam oporavka ne mora da čita *log* datoteku dublje se može okarakterisati sa dve osobine [1]:

- svi prethodni *log* zapisi su napisani od završenih transakcija (*undo* faza algoritma)
- baferi tih transakcija su napisani na disk (*redo* faza algoritma)

Kontrolna tačka predstavlja zapis u *log* datoteci posle kojeg se ne mora čitati dalje jer je sistem provereno potvrdio da obe osobine važe. Sistem trivijalno ovo može da obezbedi nakon što je završio sa oporavkom, a algoritam koji ovo obezbeđuje u ostalim situacijama se može pronaći u sekciji 3.2.1. *LBDB* implementira samo logiku za mirnu kontrolnu tačku (eng. *quiescent checkpoint*), ali postoji i nemirna kontrolna tačka (eng. *nonquiescent checkpoint*) koja je fleksibilnija, ali i algoritamski zahtevnija.

Nakon oporavka, umesto pisanja kontrolne tačke, sistem radi arhiviranje *log* datoteke. Arhiviranje je ekvivalentna operacija, a omogućava preglednije održavanje sistema tako što se *log* datoteke kojima nije potreban dalji pristup pomeraju u zaseban direktorijum.

3.1.2.3 *Algoritam poništavanja transakcije*

Sistem radi poništavanje transakcije tako što prolazi kroz *log* datoteku od kraja ka početku i poništava svaku modifikacionu operaciju te transakcije, a staje sa prolaskom *log* datoteke kada naiđe na operaciju koja označava početak te transakcije.

3.1.3 *Bezbedan višenitni pristup*

Sistem za bezbedan višenitni pristup je podsistem u okviru transakcija koji sadrži drugi deo logike zbog kojeg se sav pristup vrednostima radi kroz transakcije. Primarno se brine o rešavanju konflikta konkurentnog pristupa istim blokovima. Obuhvata *ACID* osobinu (eng. *isolation*).

Priroda višenitnih pristupa uvodi nedeterminističan redosled operacija koji implicira konflikte. Konflikt je kada rezultat dve operacije zavisi od njihovog redosleda izvršavanja.

Postoje dve vrste konflikta: *write-write* konflikt i *read-write* konflikt. Kod *write-write* konflikta, jedna operacija modifikuje neku vrednost, pa druga modifikuje istu vrednost. Kod *read-write* konflikta, jedna operacija čita neku vrednost, a druga modifikuje istu tu vrednost. Konflikti ne mogu da se dese kod *read-read* operacija ili ako operacije rade nad vrednostima koje se nalaze u različitim blokovima [1].

Sprečavanje konfliktujućih operacija u sistemu se postiže preko sistema katanaca, koji omogućavaju korektan redosled pristupa vrednostima za čitanje i pisanje. Protokol zaključavanja u *LBDB* sistemu definiše sledeća pravila rada sa katancima:

- pre čitanja vrednosti iz bloka, potrebno je steći *deljeni* katanac za taj blok,
- pre pisanja vrednosti u blok, potrebno je steći *ekskluzivni* katanac za taj blok,
- nakon završetka transakcije, potrebno je pustiti sve katance za sve blokove koje je ta transakcija zaključala.

Poštovanje ovakvog protokola zaključavanja uvek garantuje tačnost rada sa vrednostima, ali znatno smanjuje konkurentnost sistema. Povećanje konkurentnosti sistema se radi izborom izolacionog nivoa individualnih transakcija. Izolacioni nivoi transakcija povećavaju konkurentnost ali žrtvuju tačnost pročitanih vrednosti tako što upravljaju životnim ciklusom *deljenih* katanca na različite načine [1].

Različiti izolacioni nivoi su opisani u sekciji 3.1.3.1 zbog šire slike razumevanja transakcija i potencijalnih proširenja, ali podsistem bezbednog višenitnog pristupa *LBDB* sistema implementira samo serijalizujući izolacioni nivo transakcija.

3.1.3.1 Izolacioni nivoi transakcija

Табела 1: Različiti izolacioni nivoi transakcija

Izolacioni nivo	Problemi	Puštanje <i>deljenih</i> katanaca	<i>EOF</i> marker
Serijalizujući (eng. <i>serializable</i>)	ne postoje	<i>deljeni</i> katanci se drže do završetka transakcije	postoji <i>deljeni</i> katanac
Ponovno čitanje (eng. <i>repeatable read</i>)	fantomske vrednosti	<i>deljeni</i> katanci se drže do završetka transakcije	ne postoji <i>deljeni</i> katanac
Čitanje samo potvrđenih vrednosti (eng. <i>read committed</i>)	fantomske vrednosti, promenjene vrednosti	<i>deljeni</i> katanci se puštaju odmah nakon čitanja	ne postoji <i>deljeni</i> katanac
Čitanje i nepotvrđenih vrednosti (eng. <i>read uncommitted</i>)	fantomske vrednosti, promenjene vrednosti, “prljava” čitanja	ne koriste se <i>deljeni</i> katanci	ne postoji <i>deljeni</i> katanac

Različiti izolacioni nivoi se mogu implementirati i preko *MVCC* (eng. *multi version concurrency control*) pristupa [3].

Izolacioni nivoi transakcija su koncipirani tako da svaki nivo izolacije rešava sve probleme nivoa ispod njega.

- Problem “prljavog” čitanja je kada transakcija *A* može da čita vrednosti koja je transakcija *B* izmenila, pre nego što je transakcija *B* završila.
- Problem promenljivih vrednosti je kada transakcija *A* pročita neku vrednost, transakcija *B* je modifikuje, transakcija *A* ponovo pročita istu vrednost i dobije vrednost koju je transakcija *B* modifikovala umesto vrednosti koju je transakcija *A* prvobitno pročitala.
- Problem fantomskih vrednosti je kada transakcija *A* pročita sve vrednosti nekog blokovskog opsega, transakcija *B* doda novu vrednost i proširi taj blokovski opseg sa novim blokom, i umesto da transakcija *A* pri ponovnom čitanju svih vrednosti dobije istu listu prvobitnih vrednosti, dobije i novu vrednost iz novog bloka koju je transakcija *B* dodala.

Pošto je nivo granularnosti zaključavanja na nivou bloka, fantomska čitanja u okviru transakcije *A* i datoteke *D* se mogu desiti samo ako transakcija *B* na kraju *D* doda novi blok pre završetka transakcije *A*. Sprečavanje ovoga se realizuje dobijanjem virtuelnog *deljenog* katanca nad *EOF* (eng. *end of file*) markerom datoteke *D* od strane transakcije koja čita podatke (transakcija *A*) i dobijanjem virtuelnog *ekskluzivnog* katanca nad *EOF* markerom datoteke *D* od strane transakcije koja dodaje novi blok (transakcija *B*). *B* neće moći da doda novi blok u *D* dok *A* drži virtuelni *deljeni* katanac nad *EOF* markerom.

Spomenuti izolacioni nivoi transakcija se odnose samo na operacije koje čitaju vrednosti. Operacije koje modifikuju vrednosti uvek moraju poštovati korektno dobijanje *ekskluzivnih* katanaca. Transakcije na individualnom nivou mogu tolerisati neprecizne podatke, ali kada bi se dobijanje *ekskluzivnih* zaobišlo, cela baza podataka bi postala neupotrebljiva [1].

3.1.3.2 Katalog katanaca

Instanca klase *LockTable* je deljena za sve transakcije u sistemu. U njoj se realizuju mehanizmi praćenja postojanja katanaca za sve blokove.

Deljeni katanac za neki blok je moguće steći bez obzira na postojanje drugih deljenih katanaca, ali *ekskluzivni* katanac za neki blok je moguće steći samo kada ne postoji nijedan drugi katanac bilo koje vrste.

Ako se desi da transakcija nije uspela da zaključa željeni blok (bilo sa *deljenim* ili *ekskluzivnim* katancem) vraća se greška klijentu. Sistem nema mehanizam za detekciju zastoja, već čeka da prođe određeni vremenski period, posle kog smatra da je nemoguće zaključati željeni blok. Takođe, sistem ne može da spreči situaciju gde transakcije čekaju jedne na druge u krugu (eng. *deadlock*).

3.1.3.3 Upravljanje katancima na nivou individualnih transakcija

Menadžer konkurentnosti (ConcurrencyManager klasa) sadrži skup jednostavnih algoritama koji interaguju sa katalogom katanca da bi transakcijama na individualnom nivou omogućili upravljanje katancima. Svaka transakcija ima svoj menadžer konkurentnosti.

3.2 Transakcije i klijenti

Pošto svaka operacija u sistemu mora biti izvršena u okviru neke transakcije, klijentima *LBDB* sistema je potrebno dodeliti korektne transakcione objekte. Menadžer transakcija upravlja životnim ciklusom transakcija i vezuje ih za klijente. Menadžer transakcija je jedan od tri glavna podsistema *LBDB* sistema obrade upita.

Nova transakcija počinje čim se prethodna transakcija za koju je klijent vezan završi. Prva transakcija se dodeljuje klijentu prilikom njegovog povezivanja na sistem. Podrazumevani režim završetaka transakcija je automatsko završavanje (eng. *autocommit*) u kom se svaka transakcija sastoji od tačno jedne operacije. Drugi režim završetaka transakcija je ručni režim, u kom se transakcija može sastojati od više operacija, i u tom slučaju kraj transakcije označava operacija potvrde ili poništavanja koja se dobija od klijenta.

Da bi se postiglo perzistiranje istog transakcionog objekta kroz više operacija, potrebno je pratiti sve aktuelne transakcione objekte u sistemu i unikatno definisati svakog klijenta. Za istog klijenta se operacije moraju izvršavati u okviru njegovog transakcionog objekta dokle god transakciji ne dođe kraj. Unikatni identifikator klijenta predstavljen je njegovom sesijom. Na apstrakcionom nivou upravljanja transakcijama, sesija nije ništa više nego broj, ali na višim slojevima je potrebno generisati tu sesiju korektno. Detalji generisanja sesije se mogu pronaći u sekciji [8.2.1.1](#).

3.2.1 Algoritam zapisa mirne kontrolne tačke

Da bi mirna kontrolna tačka bila zapisana, nijedna transakcija ne sme biti aktivna u sistemu, a ovo je evidentno zbog prve osobine mirne kontrolne tačke. Korektan zapis mirne kontrolne tačke se izvršava sledećim algoritmom [1]:

- menadžer transakcija prestaje da prihvata nove transakcije od klijenata
- menadžer transakcija čeka da svi klijenti završe sa svojom transakcijom, bilo ona ručna ili automatska
- menadžer transakcija zapisuje sve bafere na disk
- menadžer transakcija dodaje zapis mirne kontrolne tačke u *log* datoteku
- menadžer transakcija ponovo prihvata nove transakcije od klijenata

Menadžer transakcija je jedini koji je svestan činjenice da li su sve transakcije završene, jer ih prati, pa je algoritam zapisa mirne kontrolne tačke implementiran u okviru njega.

Глава 4

Metapodaci

Metapodaci su podaci koji opisuju druge podatke. Iako su podaci organizovani preko slogova (poglavlje 2), sistem im ne može pristupiti ako se ne pobrine o perzistiranju strukture tih slogova. Praćenje distribucije vrednosti je korisno prilikom pravljenja efikasnog načina dobavljanja slogova. Podaci koji definišu strukturu slogova i podaci o distribuciji vrednosti su primeri metapodataka kojima sistem barata.

4.1 Kataloške tabelle

Metapodatake koje sistem čuva da bi omogućio rad sa tabelama su podaci o postojećim tabelama i kako kolone tih tabela izgledaju. Ti podaci se čuvaju u okviru sistemskih tabela i one se nazivaju kataloške tabelle. Kataloška tabela *tablecatalog* čuva podatke o postojećim tabelama, dok kataloška tabela *fieldcatalog* čuva podatke o fizičkoj strukturi sloga neke tabelle.

Табела 2: Šema *tablecatalog* tabelle

Kolona	Tip	Opis
<i>tableid</i>	<i>Integer</i>	unikatni identifikator tabelle
<i>tablename</i>	<i>String</i>	ime tabelle
<i>slotsize</i>	<i>Integer</i>	veličina jednog sloga te tabelle u bajtovima

Табела 3: Šema *fieldcatalog* tabelle

Kolona	Tip	Opis
<i>type</i>	<i>Integer</i>	tip kolone zapisan kao numerička vrednost
<i>runtimelength</i>	<i>Integer</i>	maksimalna fizička dužina vrednosti ovog polja
<i>offset</i>	<i>Integer</i>	pozicija vrednosti te kolone
<i>tableid</i>	<i>Integer</i>	unikatni identifikator tabelle u kojoj se kolona nalazi
<i>fieldname</i>	<i>String</i>	ime kolone
<i>nullable</i>	<i>Boolean</i>	da li vrednosti kolone mogu biti <i>NULL</i> vrednost

Kataloške tabele se kreiraju prilikom inicijalizacije sistema. Bitno je napomenuti da se kataloške tabele perzistiraju na isti način kao i sve ostale tabele u sistemu, što znači da će one sadržati i slogove koje opisuju njih same. Time što se kataloške tabele perzistiraju isto kao i korisničke, sistemskim tabelama se može pristupiti putem standardnih mehanizama relacionih operatora (sekcija 5).

Svi identifikatori u sistemu (imena kolona, tabela, ...) se implicitno konvertuju tako da sadrže samo mala slova.

4.2 Statistički podaci

Pristup istim slogovima tabela se često može izvršiti na više različitih načina, ali neki načini mogu biti znatno manje efikasni od ostalih. Apstrakcioni nivo upravljanja metapodacima je dužan da obezbedi statističke metapodatke koji pomažu pri proceni vremena izvršavanja određenih načina pristupa. Sam posao konstruisanja efikasnog načina pristupa je briga podsistema planiranja (sekcija 7).

Statistički metapodaci neke tabele uključuju:

- broj blokova tabele
- broj slogova u tabeli
- broj različitih vrednosti kolona tabele
- broj *NULL* vrednosti kolona tabele

4.2.1 Računanje statističkih podataka

Prilikom inicijalizacije sistema, računaju se statistički metapodaci za svaku tabelu u sistemu, a svakih 100 poziva dobavljanja metapodataka za bilo koju tabelu se osvežavaju statistički metapodaci za sve tabele. Ovaj način osvežavanja nije idealan jer pauzira sistem dok se računanje statističkih metapodataka ne završi i ovo ograničenje je detaljnije opisano u sekciji 11.2.3.

Broj blokova tabele i broj slogova u tabeli se trivijalno dobijaju iteracijom kroz svaki slog.

Broj različitih vrednosti kolone tabele nije moguće izračunati precizno, jer je za to potrebno čuvanje svih jedinstvenih vrednosti te kolone u radnoj memoriji. Male nepreciznosti neće uticati na procenu vremena izvršavanja operacija, pa je iskorišćena probabilistička struktura podataka *HyperLogLog* [4] koja rešava *count distinct* problem i ona ne čuva sve jedinstvene vrednosti u radnoj memoriji. Jedna takva struktura se dodeljuje za svaku kolonu.

Brojanje *NULL* vrednosti kolona tabele se svodi na čuvanje prostog brojača za svaku kolonu.

4.3 Pristup metapodacima

Menadžer metapodataka je glavno mesto pristupa svim ostalim metapodacima. Sastoji se iz menadžera metapodataka tabela i menadžera statističkih metapodataka. Menadžer metapodataka je jedan od tri glavna podsistema *LBDB* sistema obrade upita.

SQL programski jezik je jezik deklarativnog tipa. To znači da se preko njega specificira šta treba da se uradi sa podacima (dobavljanje, filtriranje, modifikacija, ...), ali za razliku od proceduralnih programskih jezika, ne specificira se i kako. Most između deklarativne prirode SQL jezika i potrebe definisanja načina pristupa podacima je rešen implementacijom *relacione algebre* [5]. LBDB sistem prevodi kod SQL programskog jezika u stablo operatora *relacione algebre*.

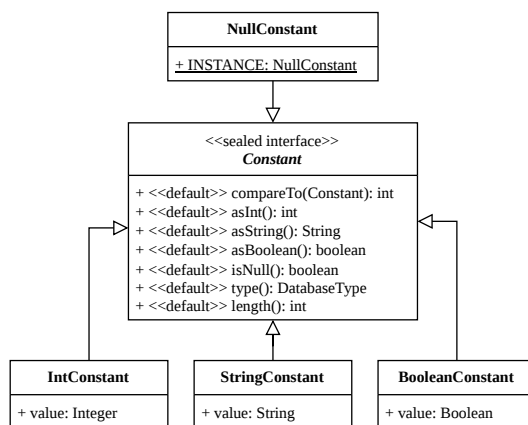
Relacija R je skup torki oblika $(d_1, d_2, \dots, d_j)$ gde za svaku komponentu d_k torke d_j važi $d_k \in D_k$, gde je D_k domen koji definiše skup svih dozvoljenih vrednosti za tu komponentu. U relacionim bazama podataka, tabela se modeluje kao relacija, dok operatori *relacione algebre* preslikavaju jednu ili više relacija u novu relaciju kao rezultat primenjene transformacije. Torke se mapiraju na slogove tabela.

5.1 Virtuelna mašina

Virtuelna mašina LBDB sistema predstavlja okruženje izvršavanja logičkih i aritmetičkih operacija i sastoji se od klasa koje modeluju komponente tih operacija.

5.1.1 Konstante

Sve vrednosti sa kojima relacioni operatori barataju predstavljene su Constant klasom. Ona omogućava sistemu da definiše generičko ponašanje za sve različite tipove podržane u sistemu. Relacioni operatori uvek vraćaju Constant objekte na svom izlazu. Implementira Comparable<Constant> interfejs zarad lakog poređenja.

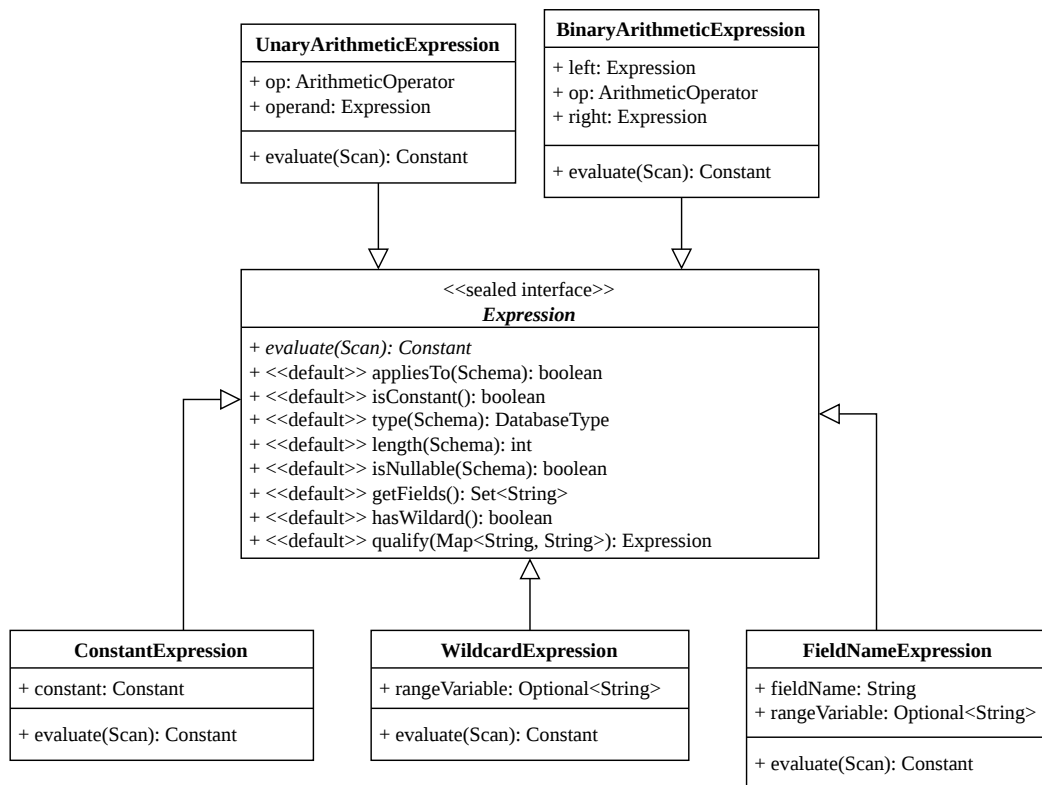


Слика 5: Хијерархија константи

Interfejs konstante je definisan *sealed interface* Java konstruktom, zbog njegove odlične kompatibilnosti sa switch sintaksom. Konkretno konstante su predstavljene *Java record* strukturom. `NullConstant` nema nikakve podatke instance, jer su sve `NULL` vrednosti identične u sistemu, pa se na svim mestima koristi ista instanca.

5.1.2 Izrazi

Sve aritmetičke operacije koje sistem evaluira su predstavljene `Expression` klasom. Evaluacija izraza uvek proizvodi `Constant` objekte. Izrazi se sastoje od proizvoljne kombinacije aritmetičkih operatora (+, -, *, /, ^), zagrada, konstanti i identifikatora kolona tabele. Interfejs izraza je definisan *sealed interface* Java konstruktom, zbog njegove odlične kompatibilnosti sa switch sintaksom. Konkretni izrazi su predstavljeni *Java record* strukturom.



Слика 6: Hijerarhija izraza

Evaluacija izraza predstavlja proces računanja konstante koja predstavlja vrednost tog izraza za neki slog tabele.

`BinaryArithmeticExpression` je izraz koji u sebi sadrži izraze i omogućava kreiranje stabla izraza, gde se prvo evaluiraju njegovi članovi, pa onda on. `UnaryArithmeticExpression` ima istu funkciju, ali za prefiksne operatore (+, -) i ima samo jedan član.

`WildcardExpression` suštinski ne predstavlja izraz, ali zbog načina parsiranja izraza (sekcija 6.2.2.8), tretira se kao jedan. Služi kao zamenski član (eng. *placeholder*) svih kolona upitanih tabela. Evaluacija je zabranjena operacija i onemogućena je izuzetkom.

`FieldNameExpression` je izraz koji identifikuje virtualnu kolonu neke tabele upita, sa opcionim preimenovanjem tabele i evaluira se na vrednost te kolone. `ConstantExpression` se evaluira direktno na konstantu za koju je vezan i nezavisan je od tabela.

5.1.3 Članovi

Član (eng. *term*) enkapsulira logiku za poređenje konstanti dva evaluirana izraza. Poređenje dve konstante zavisi od operatora poređenja koji mogu biti `=`, `!=`, `>`, `>=`, `<`, `<=`, `IS`, `IS NOT`. Razlika između `= i IS` (isto tako i između `!= i IS NOT`) je u tome kako se ponašaju sa `NULL` vrednostima. `IS` omogućava poređenje sa `NULL` vrednostima, dok `=` to ne podržava. Evaluacija člana nad nekim relacionim operatorom, za razliku od evaluacije izraza, vraća samo da li član važi ili ne.

5.1.4 Predikati

Predikati ulančavaju članove logičkim operatorima. Evaluacija predikata nad nekim relacionim operatorom funkcioniše isto kao i evaluacija jednog člana, ali između tih članova stoje različite logičke operacije. *LBDB* sistem podržava samo `AND` logičke operatore između članova i ovo ograničenje je detaljnije opisano u sekciji 11.2.6.

5.1.5 Generalizacija evaluacije

Izrazi i predikati implementiraju `Evaluatable` interfejs koji definiše sve operacije potrebne za njihovu evaluaciju i kasniju proveru validnosti. Ovaj interfejs omogućava sistemu da se ne brine o tome šta se evaluira, već samo o konstanti koju će dobiti, što dalje omogućava da se vrednosti projektovanih kolona (sekcija 5.2.2.10) dobijaju i preko evaluacije izraza i preko evaluacije predikata.

```
public interface Evaluatable {
    Constant evaluate(Scan scan);
    boolean isConstant();
    DatabaseType type(Schema schema);
    int length(Schema schema);
    boolean isNullable(Schema schema);
    Set<String> getFields();
    boolean hasWildCard();
    Evaluatable qualify(Map<String, String> aliases);
}
```

Листинг 2: `Evaluatable` interfejs

5.2 Struktura relacionih operatora u sistemu

Za izvršavanje naredbi definisanih *SQL* jezikom, često je potrebno primeniti više relacionih operatora. Primena više relacionih operatora se radi njihovim ulančavanjem u stablovsku strukturu podataka i zbog ovoga se kaže da sistem izvršava “stablo” relacionih operatora.

5.2.1 Protočna obrada

Relacioni operatori podržani u sistemu imaju dve zajedničke osobine [1]:

- generišu slogove jedan po jedan
- ne čuvaju generisane slogove i ne čuvaju nikakve međurezultate

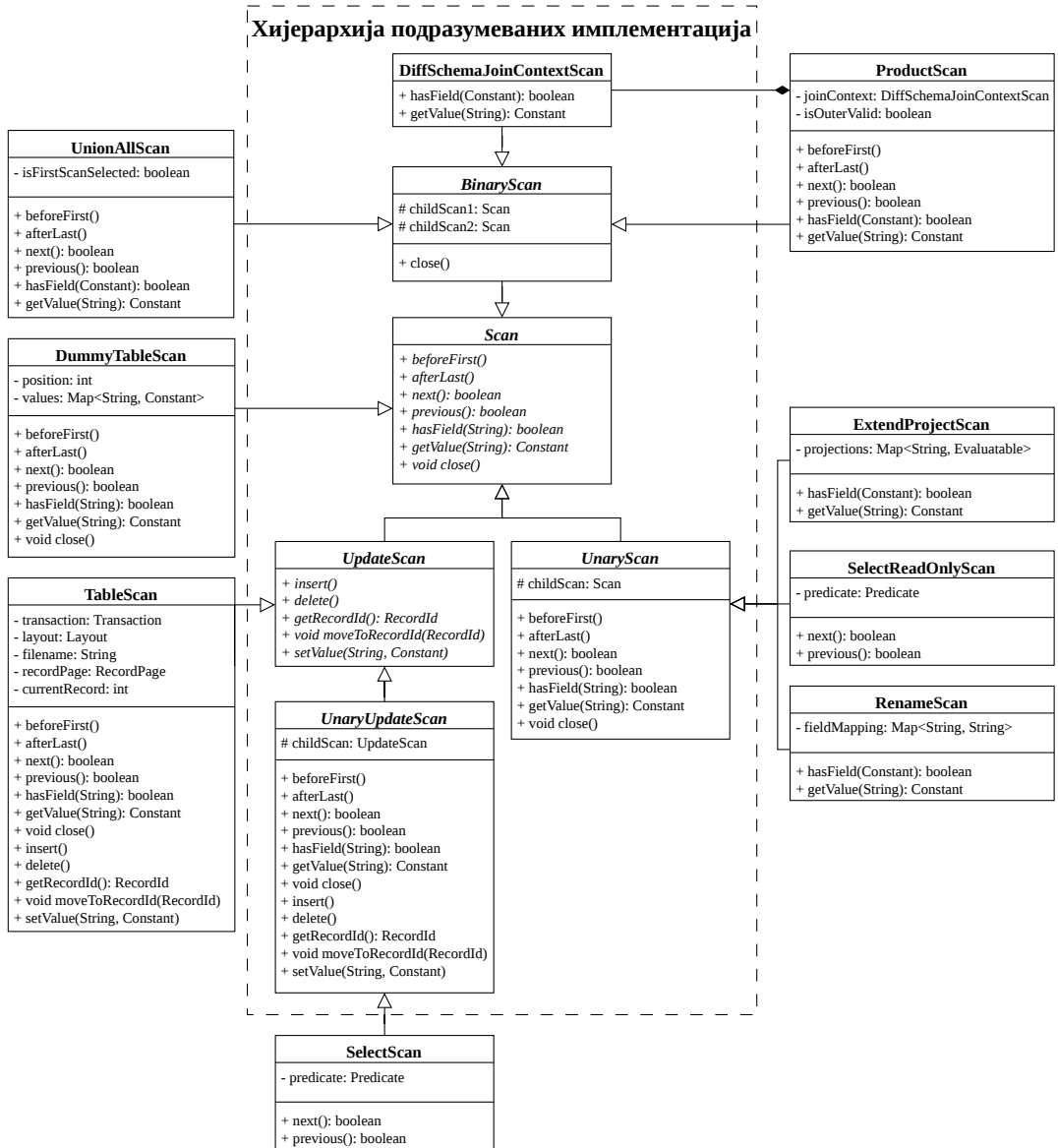
Zahtev za izvršavanje neke operacije nad stablom operatora počinje od korena stabla, koji formira rezultat uz pomoć čvorova ispod njega, ali nekad i direktno. Ovim načinom, zahtev prolazi kroz celo stablo operatora. Vraća se greška klijentu ukoliko barem jedan čvor nije uspeo da formira rezultat zbog greške prilikom izvršavanja.

Kombinacija dve navedene osobine uz delegaciju operacija se zove protočna obrada (eng. *pipelined processing*). Korišćenje protočne obrade u mnogim scenarijima ne dodaje nikakvo dodatno *U/I* opterećenje, pa ju je pogodno koristiti.

Prednost protočne obrade što ne čuva međurezultate je upravo i njena mana za određene operacije. Materijalizovana obrada (eng. *materialization, materialized processing*) omogućava i čuvanje međurezultata pa može rešiti ovu manu, ali dolazi sa svojim problemima. Takođe, neke operacije poput grupisane agregacije, vraćanje samo jedinstvenih slogova i proizvoljno sortiranje nije moguće obaviti bez materijalizovane obrade. Potrebno je koristiti oba načina obrade u sistemu za najbolje rezultate, ali *LBDB* sistem implementira samo protočnu obradu.

5.2.2 Hijerarhija implementacije relacionih operatora

Najopštija podela relacionih operatora je na one koji samo čitaju podatke (eng. *read-only*) i na one koji mogu da modifikuju podatke. Ova distinkcija je napravljena da se obezbedi sigurnost od pogrešne primene operatora u vremenu kompajliranja koda (eng. *compile time*). Hijerarhija podrazumevanih implementacija se sastoji od raznih kosturskih implementacija koje se koriste za lako definisanje novog relacionog operatora. Ostale klase van hijerarhije podrazumevanih implementacija predstavljaju različite relacione operatore podržane u sistemu i objašnjene su ispod.



Слика 7: Хијерархија имплементације relacionih operatora

5.2.2.1 Scan

Scan апстрактна класа дефинише операције неопходне за пролазак кроз све вредности резултујуће виртуелне табеле. Сваки relacioni operator имплементира бар ове операције. Враћање вредности се ради искључиво кроз Constant објекте. Scan се може, али не мора, мапирати на физичку табелу. Имплементира AutoCloseable интерфејс који омогућава *RAII* (eng. *Resource Acquisition Is Initialization*) шаблон, али не пружа имплементацију логике ослобађања ресурса.

5.2.2.2 UpdateScan

Rezultujuće virtualne tabele relacionih operatora koji dozvoljavaju modifikaciju vrednosti se moraju mapirati na fizičke tabele, jer nema smisla menjati virtualne vrednosti. To znači da za svaki virtualni slog r u stablu modifikujućih operatora, mora da postoji r' koji ima identičnu strukturu, poziciju i vrednosti u datoteci tabele. UpdateScan klasa definiše operacije promene vrednosti kolona nekog sloga, umetanja novog sloga i brisanja sloga.

5.2.2.3 UnaryScan

UnaryScan sadrži podrazumevane implementacije proizvoljnog relacionog operatora koji transformiše **jednu** relaciju, to jest jednu tabelu. Svaki poziv podrazumevane implementacije je samo prosleđen operatoru ispod. Ovakva struktura omogućava da operatori koji nasleđuju ovu klasu redefinišu samo metode koje su potrebne za funkcionisanje tog operatora i izbegava se dupliranje istog koda.

5.2.2.4 UnaryUpdateScan

UnaryUpdateScan sadrži podrazumevane implementacije proizvoljnog relacionog operatora koji može da modifikuje vrednosti fizičke tabele. Funkcioniše isto kao i UnaryScan i ima sve podrazumevane implementacije iz njega.

5.2.2.5 BinaryScan

BinaryScan sadrži podrazumevane implementacije proizvoljnog relacionog operatora koji transformiše **dve** relacije, to jest dve tabele. Pošto operatori koji rade nad dve tabele nemaju toliko međusobnih preklapanja, BinaryScan implementira samo `close()` metodu koja oslobađa resurse oba deteta.

5.2.2.6 DiffSchemaJoinContextScan

DiffSchemaJoinContextScan ima sličnu ulogu kao BinaryScan, ali samo za operatore koji vrše multiplikativno objedinjavanje dve tabele i enkapsulira korektan pristup kolonama iz dve šeme koje nemaju preklapanje.

5.2.2.7 TableScan

TableScan nije pravi relacioni operator zato što njegova implementacija ne radi transformacije tabele, već pruža logiku za dobavljanje i modifikaciju fizičkih vrednosti. Služi kao omotač oko objekata stranice slogova i zadužen je za konstruisanje novih povezanih objekata stranice slogova. Pošto pruža inicijalne vrednosti koje će dalje biti transformisane, uvek se nalazi na dnu stabla relacionih operatora.

5.2.2.8 DummyTableScan

DummyTableScan nije pravi relacioni operator zato što njegova implementacija ne radi transformacije tabele, već pruža logiku za dobavljanje i navigaciju jedinog sloga virtualne tabele koja se konstruiše u SQL upitima koji ne rade ni sa jednom fizičkom tabelom.

5.2.2.9 SelectScan

SelectScan implementira operator generalizovane selekcije $\sigma_{\varphi}(R)$ iz relacione algebre, gde je σ ime selekcionog operatora, φ je formula filtera, a R je relacija nad kojom se operator primenjuje. Redefiniše samo metode za prolazak kroz slogove, jer je to jedino neophodno da se ukinu slogovi koji ne prolaze filter. Sadrži Predicate objekat pomoću kog filtrira. Može da se koristi i u kontekstima modifikujućih stabala operatora i u kontekstima *read-only* stabala operatora i zbog toga postoji i SelectReadOnlyScan varijanta koja ima istu funkciju.

5.2.2.10 ExtendProjectScan

ExtendProjectScan implementira operator projekcije $\Pi_{a_1, \dots, a_n}(R)$ iz relacione algebre, gde je Π ime projekcionog operatora, a a_n predstavlja izraz čija evaluacija proizvodi vrednost komponente n . Operator projekcije koji klasa implementira nije striktno operator projekcije formalno definisan u relacionoj algebri, zato što dozvoljava kreiranje novih kolona sa izrazima koji će biti izračunati u trenutku izvršavanja stabla, a ne povučene direktno iz fizičke tabele. Dozvoljava i dodeljivanje proizvoljnog imena izrazima kolona, ali se to ne treba pomešati sa operatorom preimenovanja. Redefiniše metode dobavljanja vrednosti i provere postojanja kolone nekog imena.

5.2.2.11 RenameScan

RenameScan implementira operator preimenovanja $\rho_{a_n/b_n}(R)$ iz relacione algebre, gde je ρ ime operatora preimenovanja, a b_n predstavlja novo ime komponente a_n . Razlikuje se od formalne definicije operatora preimenovanja iz relacione algebre jer dozvoljava n preimenovanja odjednom da bi se izbeglo ulančavanje istih operatora. Bitno je napomenuti da operator preimenovanja omogućava pristup koloni sa imenom a kroz ime b , za razliku od operatora projekcije koji samo dodeljuje ime nekom izrazu. Redefiniše metode dobavljanja vrednosti i provere postojanja kolone nekog imena.

5.2.2.12 ProductScan

ProductScan implementira operaciju dekartovog proizvoda relacija R i S : $R \times S$. Rezultujuća relacija predstavlja virtuelnu tabelu koja ima $|R| * |S|$ torki, a svaka torka se sastoji od svih komponenti obe relacije. Operacija dekartovog proizvoda je multiplikativna, a ProductScan zahteva da tabele nemaju kolone sa istim imenom, pa se koristi DiffSchemaJoinContextScan. Iteracija kroz rezultujuću tabelu nakon ProductScan operatora se vrši tako što se za svaki slog prve tabele, prolazi kroz sve slogove druge tabele. Redefiniše metode dobavljanja vrednosti, provere postojanja kolone nekog imena i sve navigacione metode.

5.2.2.13 UnionAllScan

UnionAllScan implementira operaciju kreiranja relacije koje sadrži sve torke relacija R i S . Ne briše duplikate. Zahteva da su komponente torki obe relacije istog tipa i da obe

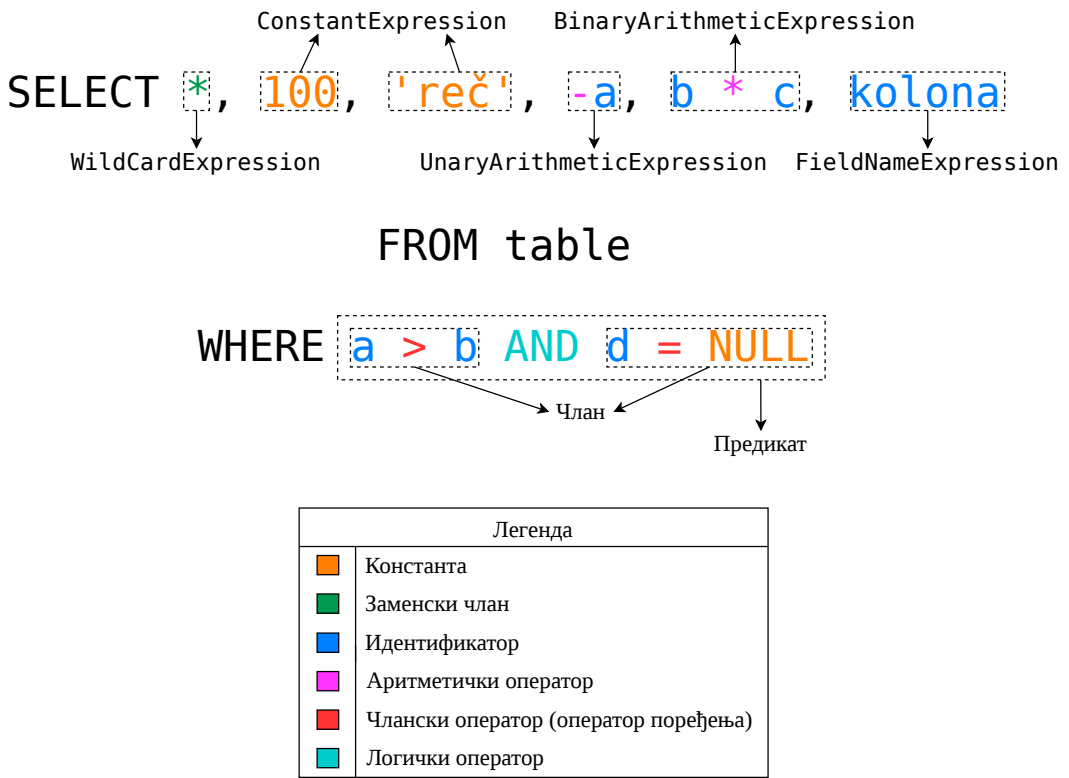
relacije imaju isti broj komponenata po torki. Po *SQL* standardu, kolonama druge tabele se pristupa po imenima prve. Operacija unije je aditivna. Iteracija kroz rezultujuću tabelu nakon *UnionAllScan* operatora se vrši tako što se prvo prolazi kroz sve slogove prve tabele, pa se prolazi kroz sve slogove druge tabele. Redefiniše metode dobavljanja vrednosti, provere postojanja kolone nekog imena i sve navigacione metode.

5.2.3 Primeri

Sledeći primeri vizuelno pokazuju različite delove stabla relacionih operatora i kako se neke funkcionalnosti ponašaju. Primeri ne oslikavaju stabla relacionih operatora koje bi *LBDB* sistem napravio, već služe da pokažu uklapanje komponenata virtuelne mašine, protočnu obradu i kako se *Scan* objekti oslanjaju jedni na druge.

5.2.3.1 Primer izgleda komponenti virtuelne mašine

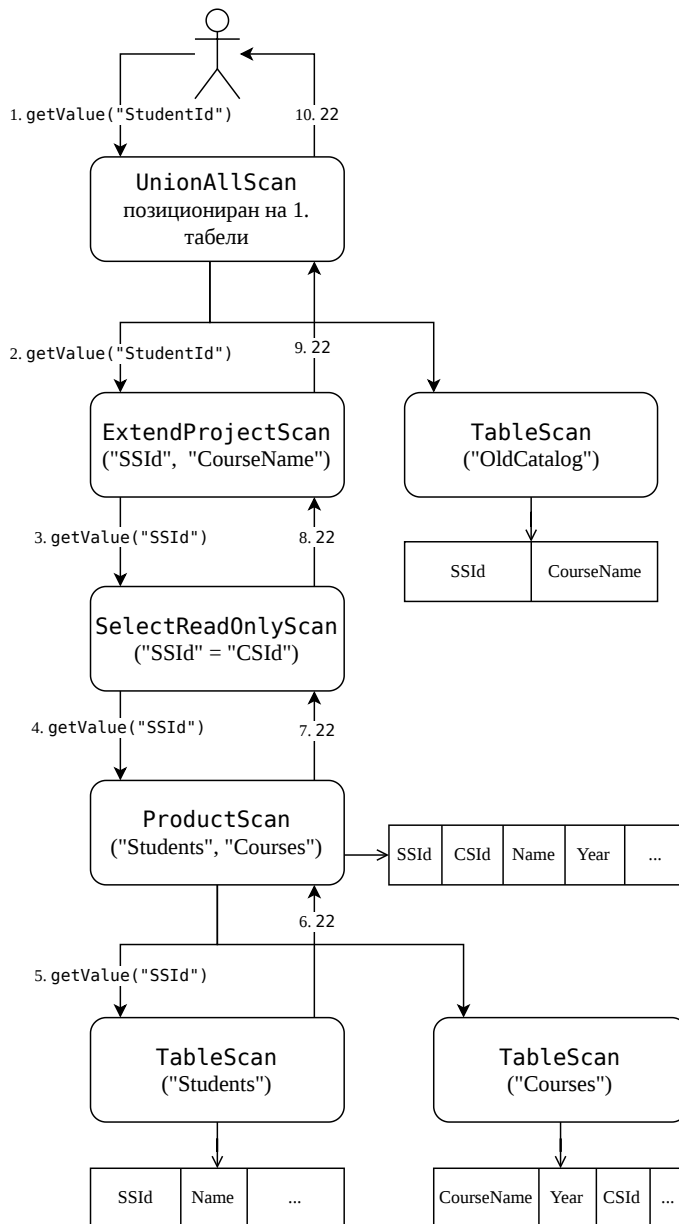
Za datu naredbu *SQL* jezika se mogu videti jednostavni primeri svake komponente virtuelne mašine.



Слика 8: Primer naredbe u *SQL* jeziku gde se pokazuju sve komponente virtuelne mašine

5.2.3.2 Primer poziva `getValue()` metode

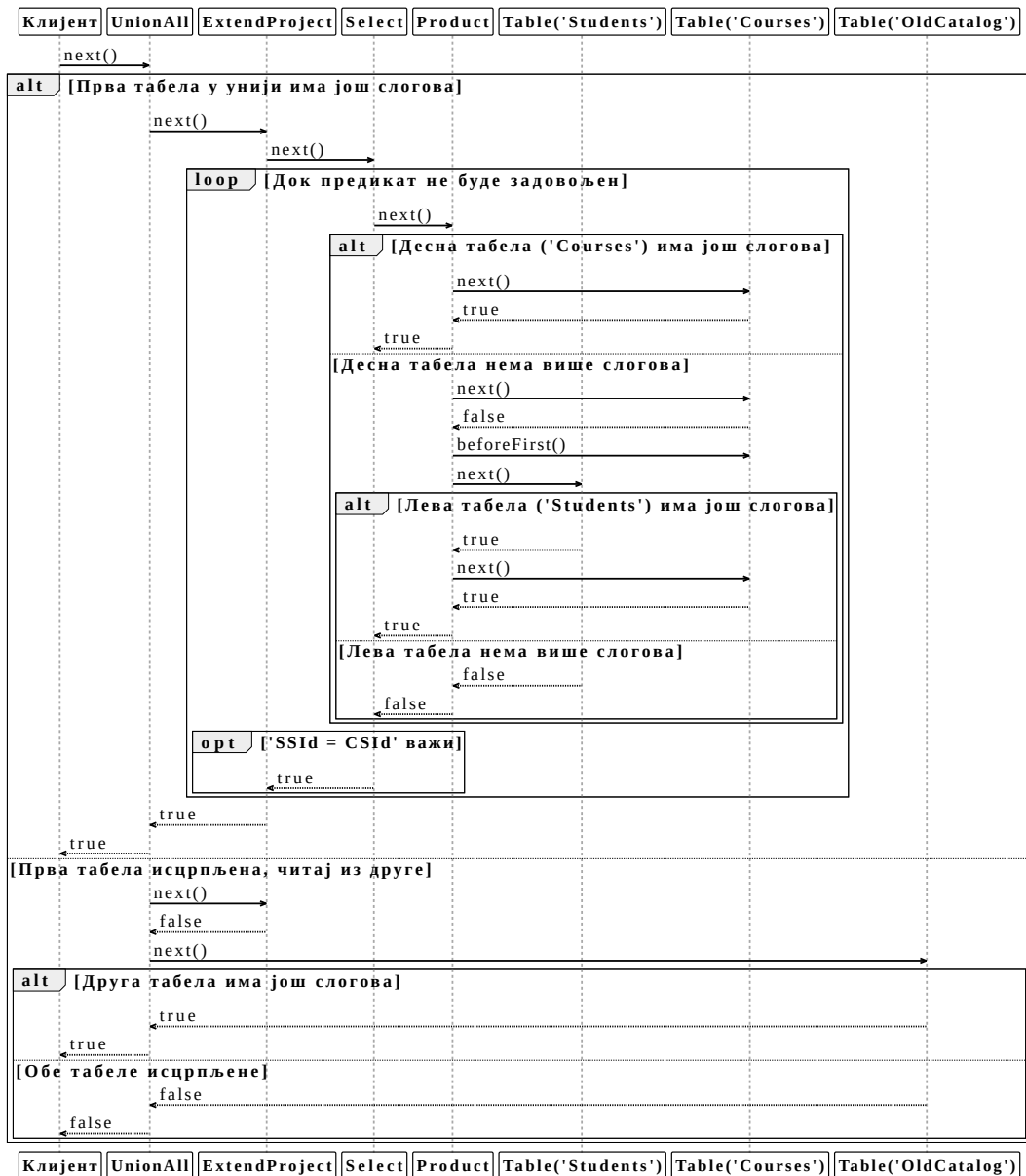
Prate se koraci poziva `getValue()` metode, za virtualnu kolonu "StudentId". Union operator, koji je skroz na vrhu, spaja dve tabele: tabelu koja modeluje zastareli način vođenja evidencije i virtualnu tabelu koja je rezultat podstabla operatora koji projektuju iste kolone kao i u tabeli zastarele evidencije. `ExtendProjectScan` je preimenovao "SSId" u "StudentId". Uz `TableScan` i `ProductScan` operatore stoji i uprošćena šema.



Слика 9: Primer poziva `getValue()` metode u konkretnom stablu relacionih operatora

5.2.3.3 Primer poziva next() metode

Ovaj primer koristi isto relaciono stablo kao i prethodni primer. Pomoću dijagrama sekvence se opisuje procedura iteracije kroz stablo sa svim vrstama operatora koji menjaju podrazumevanu iteraciju.



Слика 10: Dijagram sekvence next() metode u konkretnom stablu relacionih operatora

Klijenti šalju *SQL* naredbe u tekstualnom formatu, ali sistemu komad teksta nema nikakvo inherentno značenje. Podsystem za ekstrakciju informacija iz teksta naredbe se naziva parser.

Nije svaki komad teksta validna *SQL* naredba, ali njegova validnost se može podeliti na dva sloja [1]:

- sintaksička validnost, gde sintaksa predstavlja skup pravila koja definišu moguće operacije po nekoj gramatici,
- semantička validnost, koja je ispunjena ako je neka operacija validna u kontekstu podataka koje koristi (imena tabela, imena kolona, ...)

Parsiranje konstruiše apstraktno sintaksičko stablo (eng. *abstract syntax tree*, *AST*) koje se mapira na podržane operacije i služi za proveru **samo** sintaksičke validnosti operacije. Provera semantičke validnosti je deo planera (sekcija 7.2).

6.1 Tokenizator

Blokovi teksta se sastoje od individualnih karaktera. Većina individualnih karaktera sadrži jako malo značenja kada se obrađuju nezavisno i zbog toga se uvodi sistem koji grupiše povezane karaktere. Skup grupisanih karaktera koji zajedno imaju visok stepen značenja se zovu tokeni. Karakteri koji se obrađuju sami su isto opisani kao tokeni, jer je bitno da je svaki token imenovan.

Tokenizer klasa sadrži logiku pretvaranja bloka teksta u tokene odgovarajućeg tipa i implementirana je preko *Java Iterator* interfejsa. Token je definisan *sealed interface Java* konstruktom, zbog njegove odlične kompatibilnosti sa *switch* sintaksom. Svi tokeni u sistemu su grupisani u sledeće kategorije:

- ključne reči *SQL* jezika,
- identifikatori,
- simboli,
- brojevi,
- *String*-ovi,
- nevalidni karakteri,
- EOF token.

Svaka kategorija tokena implementira *Token* interfejs i predstavljena je *record* ili *enum Java* strukturom.

6.2 Parser

Gramatika nekog jezika predstavlja skup pravila koje opisuju sve legalne komade teksta, koje neki sistem podržava. Sintaksičke kategorije su koncepti sa kojima gramatika barata. Predstavljaju čvorove sintakstičkog stabla i sadrže se od drugih sintaksičkih kategorija i tokena.

Vrsta parsiranja koja je implementirana se zove *recursive descent* parsiranje. U *recursive descent* parsiranju, gramatika se proverava od gore ka dole. Ulazna tačka je koren sintakstičkog stabla i za svako podstablo, to jest gramatičko pravilo, postoji funkcija koja obrađuje to pravilo. Funkcije se često pozivaju rekurzivno da bi obradili veće celine, pa je tako ovaj način parsiranja i dobio ime. Svaka funkcija obrade pravila, na bilo kom nivou, se mapira na jednu sintaksičku kategoriju.

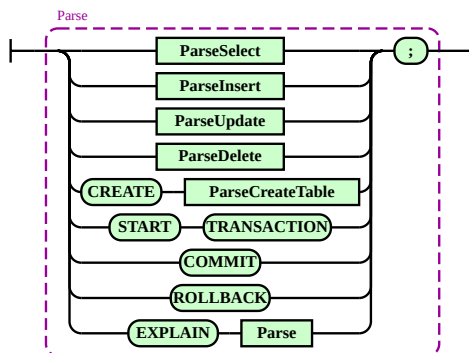
6.2.1 Iskazi

Uspešno parsiranje nekog bloka teksta koji predstavlja *SQL* operaciju rezultuje u iskazu, koji sadrži sve neophodne podatke da se ta operacija izvrši. Iskaz (Statement klasa) je definisan *sealed interface Java* konstruktom, zbog njegove odlične kompatibilnosti sa *switch* sintaksom. Svaki iskaz je predstavljen *Java record* strukturom i nasleđuje *Statement*.

6.2.2 Sintaksičke kategorije *SQL* operacija

Funkcije koje generišu iskaze predstavljaju sintaksičke kategorije najvišeg apstrakcionog nivoa i grupisane su u različite *Java* datoteke. Svaka datoteka sadrži sve sintaksičke kategorije nižeg apstrakcionog nivoa potrebne da se iskaz uspešno obradi.

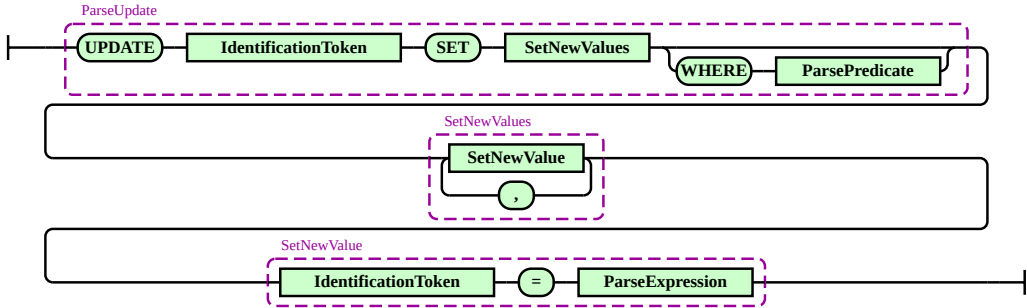
6.2.2.1 Parse



Слика 11: Gramatika Parse sintaksičke kategorije

Parse predstavlja glavnu sintatičku kategoriju i grupiše sve ostale sintaksičke kategorije. Omogućava i EXPLAIN naredbu, koja generiše opis naredbe koja će se izvršiti. Iskazi upravljanja životnim ciklusima transakcija se isto parsiraju ovde jer su previše jednostavni da bi se pravila posebna sintaksička kategorija za njih. Vraća *Statement* objekat.

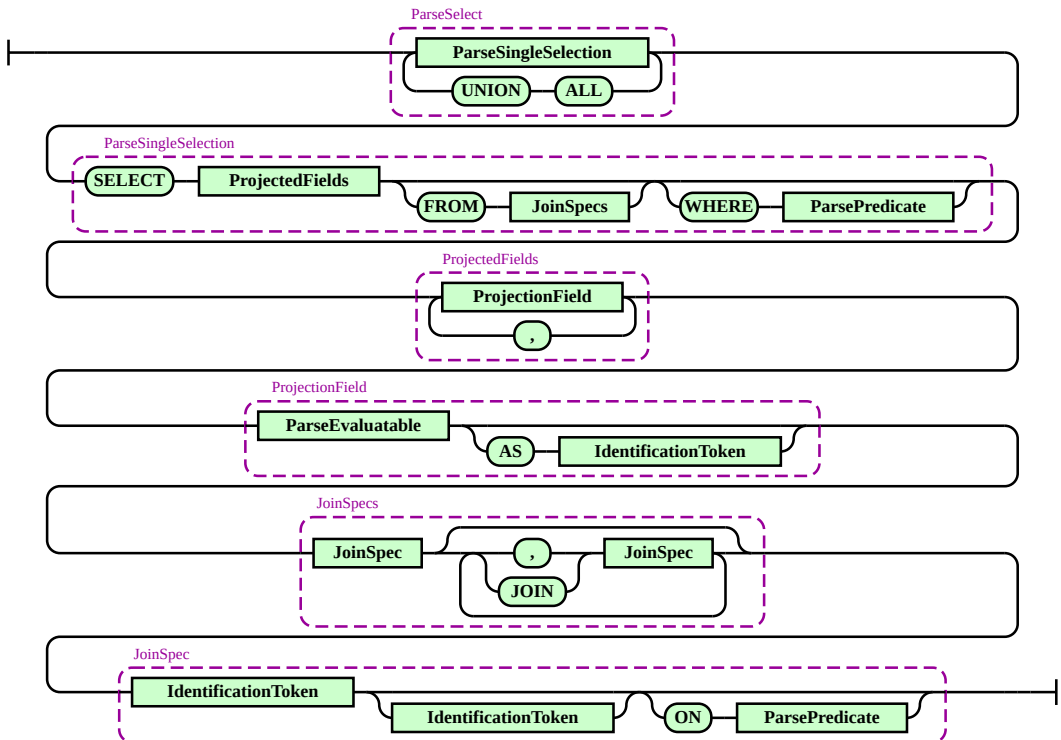
6.2.2.2 ParseUpdate



Слика 12: Граматика ParseUpdate синтаксишке категорије

ParseUpdate predstavlja naredbu modifikovanja podataka neke tabele. Може садржати произвољан број нових додела вредности, али свако поље у свим доделама вредности мора постојати у референцираној табели. Могуће је модификовати само неке слогове, а не све, тако што се дефинише услов претраге. Враћа UpdateStatement објекат.

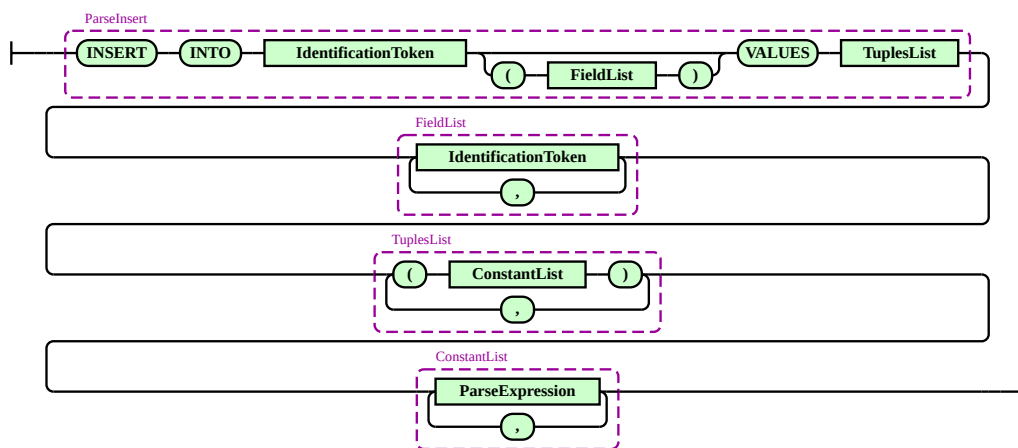
6.2.2.3 ParseSelect



Слика 13: Граматика ParseSelect синтаксишке категорије

ParseSelect predstavlja naredbu upita (eng. *query*) podataka. Može sadržati proizvoljan broj projektovanih kolona, gde je svaka kolona predstavljena kao bilo šta što može da se evaluiira i kojoj se može dodeliti novo ime (uz AS ključnu reč). Podržava filtriranje na osnovu uslova pretrage. Upit može biti nad pravim tabelama ili nad virtuelnom tabelom koja sadrži jedan slog. Ulančavanje tabela se može raditi na dva načina: samo navođenje tabela odvojene zarezom ili preko JOIN ključne reči gde se uslov ulančavanja upisuje odmah. Uslov ulančavanja napisan u JOIN sekciji se samo dodaje na uslov pretrage, umesto da predstavlja neki specijalan način ulančavanja. Vraća SelectStatement objekat.

6.2.2.4 ParseInsert



Слика 14: Gramatika ParseInsert sintaksičke kategorije

ParseInsert predstavlja naredbu umetanja novih slogova u neku tabelu. Lista polja ne mora biti definisana, uzima se podrazumevani redosled koji je napravljen tokom kreiranja te tabele. Izrazi moraju biti konstantni, to jest njihova evaluacija ne sme zavisiti od vrednosti koje ne mogu da se izračunaju bez pristupa tabelama. Vraća InsertStatement objekat.

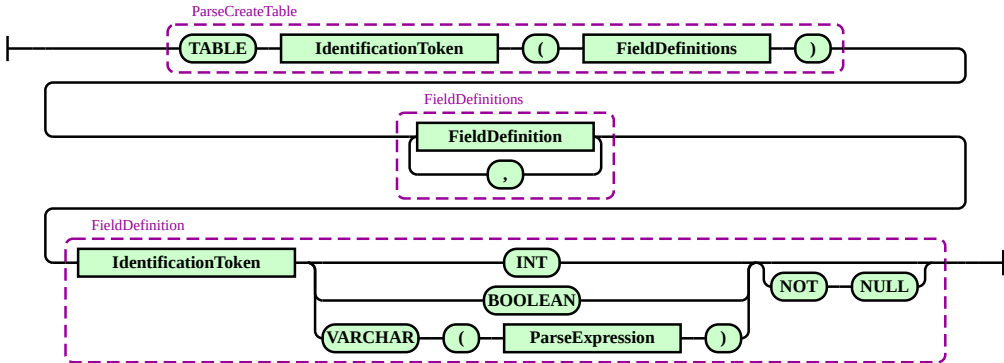
6.2.2.5 ParseDelete



Слика 15: Gramatika ParseDelete sintaksičke kategorije

ParseDelete predstavlja naredbu brisanja slogova iz neke tabele. Moguće je obrisati samo slogove koji ispunjavaju neki uslov, tako što se definiše uslov pretrage. Vraća DeleteStatement objekat.

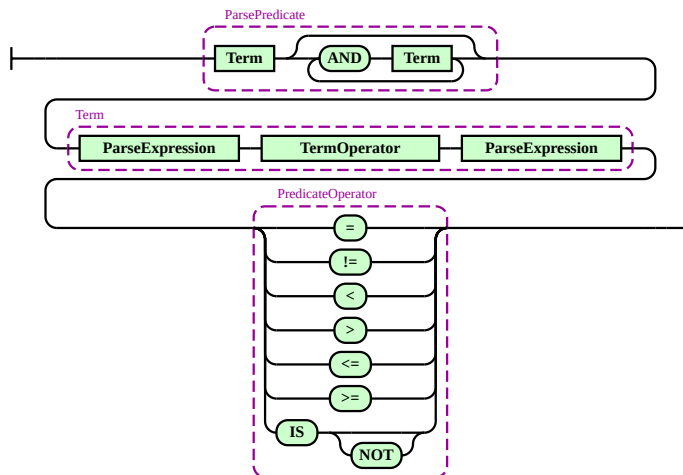
6.2.2.6 ParseCreateTable



Слика 16: Gramatika ParseCreateTable sintaksičke kategorije

ParseCreateTable predstavlja naredbu kreiranja nove tabele. Tabela može sadržati maksimalno 31 polje (sekcija 2.4). Polje može biti jedno od tipova podržanih u sistemu (listing 1), a za *String* (VARCHAR) tip se mora definisati i maksimalna dužina, koja mora biti konstantan izraz. Ograničenje da slogovi za neku kolonu ne smeju imati *NULL* vrednosti je opciono i definiše se nakon tipa kolone. Vraća CreateTableStatement objekat.

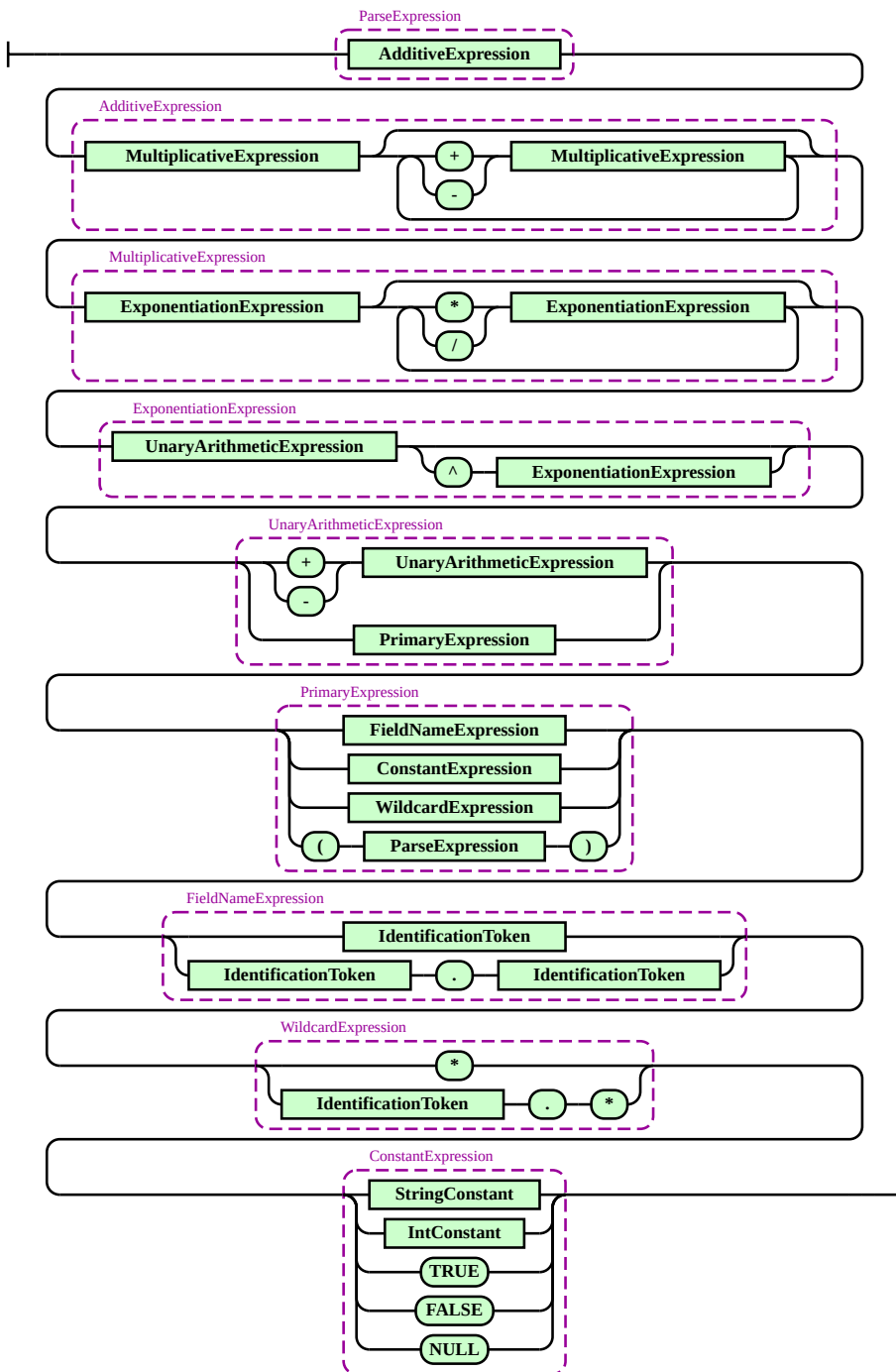
6.2.2.7 ParsePredicate



Слика 17: Gramatika ParsePredicate sintaksičke kategorije

ParsePredicate je specijalna vrsta sintaksičke kategorije koja ne proizvodi iskaz, već služi za kreiranje sintaksičkih stabala predikata. Parsiraju se izrazi i operacije poređenja od kojih se članovi sastoje, a zatim se ulančavaju članovi da bi se formirao predikat. Vraća Predicate objekat. Ne podržava članove bez operatora poređenja.

6.2.2.8 ParseExpression



Слика 18: Gramatika ParseExpression sintaksičke kategorije

`ParseExpression` je specijalna vrsta sintaksičke kategorije koja ne proizvodi iskaz, već služi za kreiranje sintaksičkih stabala izraza. Parsiranje izraza je urađeno specijalnom tehnikom *recursive descent* parsiranja koja se zove *Pratt parsing* [6]. *Pratt parsing* definiše tehnike obrade prioriteta operacija, zagrada, prepoznavanja identifikatora i zamenskih članova. Vraća `Expression` objekat.

Prvo se parsira prefiksni izraz, koji može biti literal različitog tipa, identifikator, zamenski član, izraz sa unarnom operacijom ili izraz u zagradama. Rezultat ovog koraka postaje početni levi operand. Zatim se ulazi u petlju koja se izvršava sve dok je prioritet sledećeg (infiksnog, +, −, *, /, ^) operatora strogo veći od trenutnog prioriteta.

Unutar petlje, operator se konzumira, a desni operand se dobija rekurzivnim pozivom funkcije za parsiranje izraza kojoj se prosleđuje prioritet tog novog operatora (ili prioritet umanjen za jedan, ukoliko je operacija desno-asocijativna, poput stepenovanja). Od levog operanda, operatora i desnog operanda kreira se novo stablo binarnog izraza, koje zatim postaje novi levi operand za narednu iteraciju petlje.

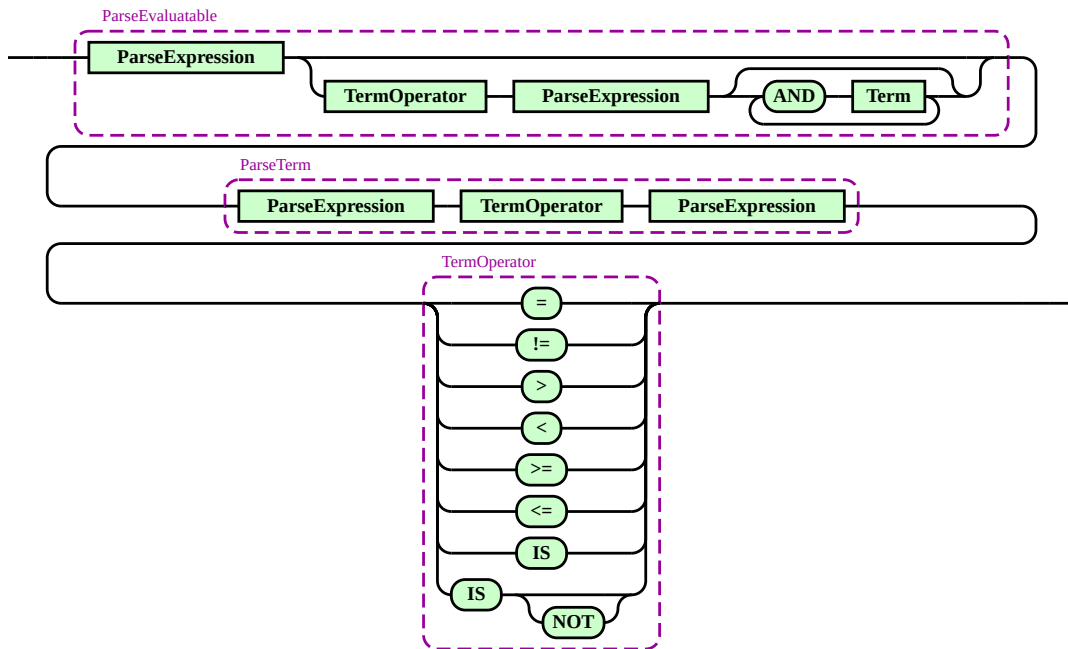
U isečku koda ispod se mogu videti različiti prioriteti operacija na osnovu tokena koji ih opisuju. Token `STAR` predstavlja i operator zamenskog člana, pa se zove `STAR` umesto `MULTIPLY`. Prefiksne operacije imaju najveći prioritet.

```
private static final int PREFIX_PRECEDENCE = 100;

private int getPrecedence(Token opToken) {
    return switch (opToken) {
        case SymbolToken.CARET -> 30;
        case SymbolToken.STAR, SymbolToken.DIVIDE -> 20;
        case SymbolToken.PLUS, SymbolToken.MINUS -> 10;
        default -> 0;
    };
}
```

Листинг 3: Prioriteti aritmetičkih operacija u sistemu

6.2.2.9 ParseEvaluatable



Слика 19: Gramatika ParseEvaluatable sintaksičke kategorije

`ParseEvaluatable` je specijalna vrsta sintaksičke kategorije koja ne proizvodi iskaz, već služi za agnostično kreiranje objekata koji se mogu evaluirati na vrednost. Liči na `ParsePredicate`, ali sa dodatnom mogućnošću da prepozna kada izraz nije deo člana ili predikata. Vraća objekat koji implementira `Evaluatable` interfejs (sekcija 5.1.5). Ne podržava članove bez operatora poređenja.

Podsistem planiranja je jedan od tri glavna podsistema *LBDB* sistema obrade upita. U okviru životnog ciklusa obrade jedne *SQL* naredbe, podsistem planiranja se oslanja na podsistem parsiranja zarad konstrukcije stabala relacionih operatora.

Podsistem planiranja se sastoji od dve celine:

- skup *Java* klasa pojedinačnih planova. Svaka klasa pojedinačnog plana modeluje primenu **jednog** relacionog operatora bez izvršavanja tog operatora u okviru virtuelne mašine
- planer, čiji je zadatak da konstruiše stablo pojedinačnih klasa planova koje se prevodi u stablo relacionih operatora; ovo stablo često ima i naziv **plan**, ali ga ne treba pomešati sa klasom plana

7.1 Struktura klasa planova u sistemu

Klasa plana relacionog operatora opisuje šemu virtuelne tabele nakon primene operatora i omogućava računanje statističkih metapodataka (sekcija 4.2) za tu virtuelnu tabelu. Ovi statistički metapodaci mogu biti iskorišćeni za efikasnije izvršavanje naredbi.

Statističke metapodatke koje klasa plana može da izračuna su isti kao i statistički metapodaci koji se prate za fizičke tabele, a oni su:

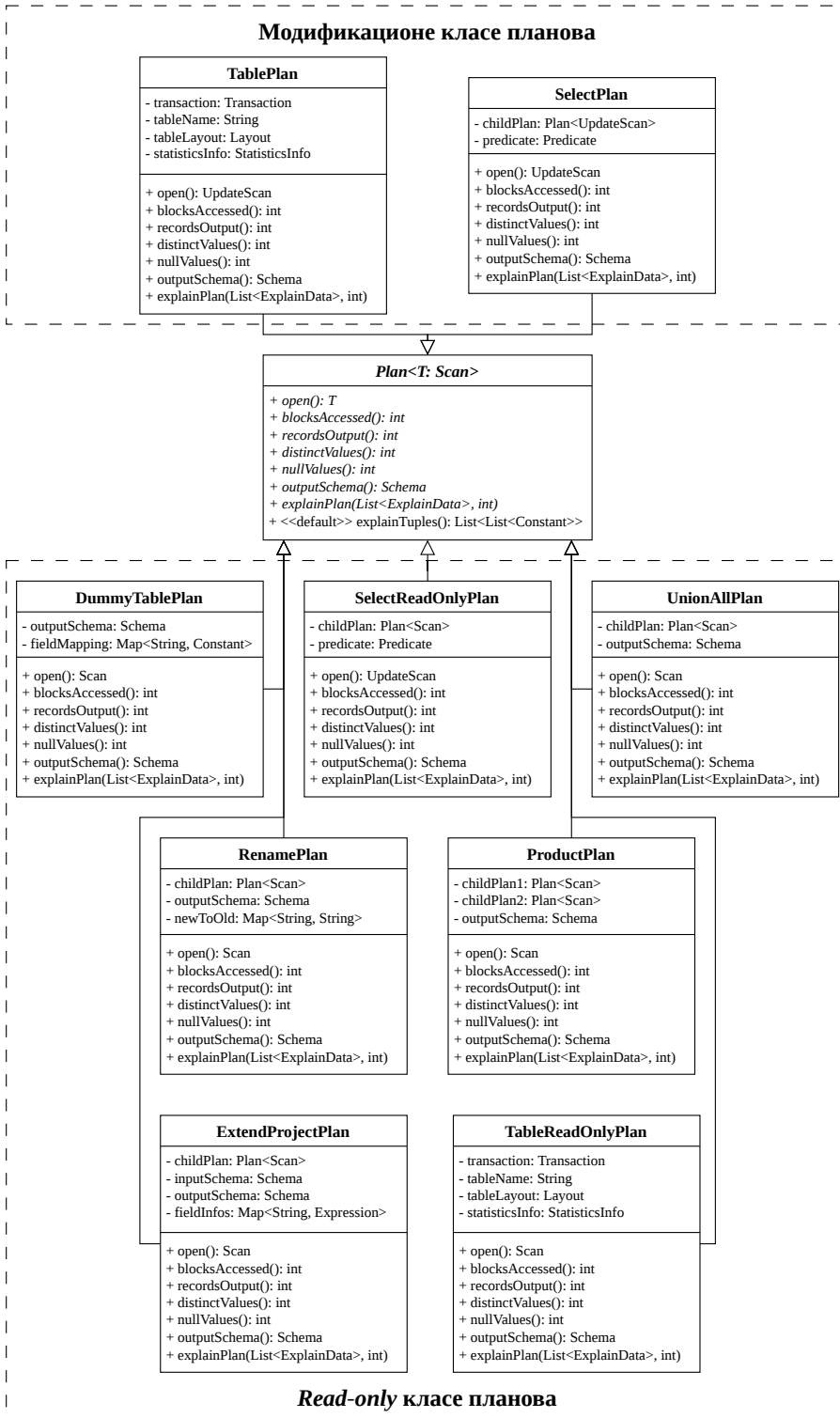
- broj blokova potrebnih za prolazak kroz sve slogove
- broj slogova
- broj jedinstvenih vrednosti za svaku kolonu
- broj *NULL* vrednosti za svaku kolonu

Postoje dva ograničenja vezana za ove statističke metapodatke:

- nisu 100% precizni, ali bez obzira na to, mogu pomoći planerima (više o ovom ograničenju u sekciji 11.2.3)
- implementacija planera *LBDB* sistema ih ne koristi u punom potencijalu (više o ovom ograničenju u sekciji 11.2.5)

7.1.1 Hijerarhija implementacije planova

Najopštija podela klasa planova je na one koji samo čitaju podatke (eng. *read-only*) i na one koji mogu da modifikuju podatke. Za razliku od hijerarhije relacionih operatora, ne postoji hijerarhija podrazumevanih implementacija jer klase planova nemaju toliko zajedničkih osobina. Podela na *read-only* i modifikacione klase planova je odrađena upotrebom *generics Java* konstrukta, umesto deljenja glavnog interfejsa na dva podtipa. Svaka klasa plana operiše nad jednim ili nad dvoje dece, a dete plana se takođe zove i prodređena klasa plana.



Слика 20: Хијерархија имплементације планова

7.1.1.1 Plan

Plan interfejs definiše operacije neophodne za računanje svih statističkih podataka, dobijanje šeme rezultujuće tabele, pretvaranje stabla planova u stablo relacionih operatora i dobijanje slogova za automatski opis plana (sekcija 7.2.3.2). Sve klase planova su modelovane preko njega.

7.1.1.2 TablePlan

TablePlan opisuje konkretnu fizičku tabelu, umesto da vrši transformacije virtuelne tabele. Izlazna šema je jednaka šemi fizičke tabele. Izvlači statističke podatke direktno iz menadžera metapodataka za tabelu za koju je vezan. Izračunati statistički metapodaci svih planova u stablu planova na kraju zavise od vrednosti statističkih metapodataka ove klase plana. Zbog implementacionih detalja Java jezika, može da se koristi samo u modifikujućim stablima operatora. Postoji i TableReadOnlyPlan varijanta koja ima istu funkciju za *read-only* stabla operatora.

7.1.1.3 DummyTablePlan

DummyTablePlan opisuje virtuelnu tabelu koja se sastoji od jednog sloga u upitima koji ne rade sa fizičkim tabelama. Izlazna šema se određuje na osnovu konstantnih vrednosti u upitu, a statistički podaci su precizni jer se lako računaju pošto je broj konkretnih vrednosti jako mali.

7.1.1.4 SelectPlan

SelectPlan opisuje virtuelnu tabelu nakon primene uslova filtriranja. Izlazna šema je jednaka šemi podređene klase plana. Broj blokova ostaje nepromenjen jer da bi znali koji sve slogovi ispunjavaju uslov filtera, potrebno je proći kroz sve slogove, pa sa time i kroz sve blokove podređenog plana.

Redukcioni faktor predstavlja koliko puta će broj slogova na izlazu biti smanjen. Računa se na osnovu uslova filtriranja, koji je predstavljen predikatom. Svaki član predikata predstavlja jedan deo filtera i ima svoj redukcioni faktor. Pošto sistem podržava samo ulančavanje članova AND logičkim operatorom, redukcioni faktor predikata se računa kao proizvod svih redukcionih faktora članova od kog se predikat sastoji.

Algoritam računanja redukcionog faktora jednog člana je predstavljen na figuri 21. Pošto se broj slogova nakon filtriranja dobija deljenjem broja slogova podređene klase plana i redukcionog faktora, specijalni slučajevi se mogu predstaviti različitim konstantama.

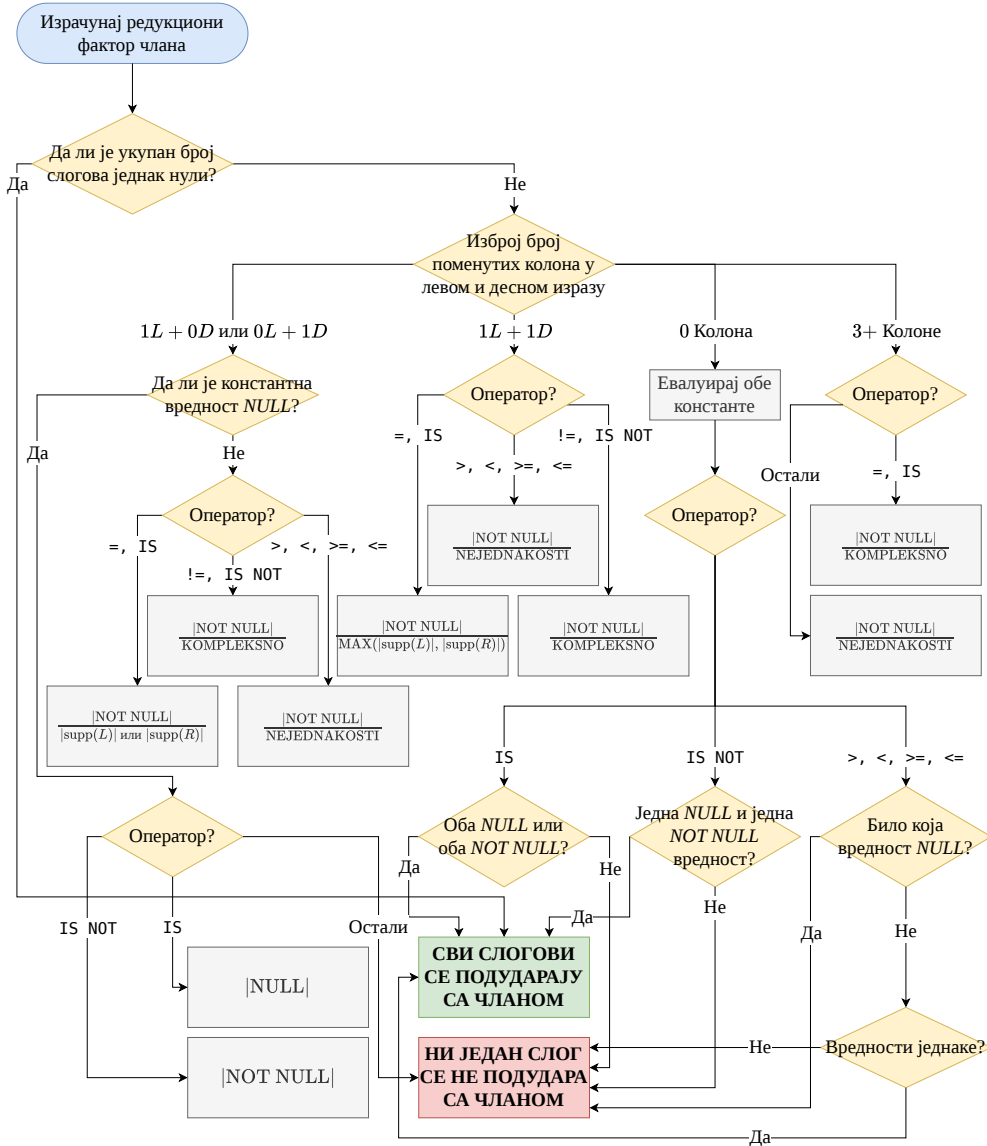
Specijalni slučaj nejednakosti je predstavljen konstantom *NEJEDNAKOSTI* koja ima vrednost 3.0 i označava procenjenu vrednost redukcije u slučaju korišćenja operacija nejednakosti.

Specijalni slučaj kada postoje suviše kompleksni izrazi je predstavljen konstantom *KOMPLEKSNO* koja ima vrednost 10.0 i označava procenjenu vrednost redukcije u slučaju

postojanja izraza koji ima ili više od dve kolone ili izraza koji kombinuje kolone sa operacijama poređenja na netrivialan način.

Slučaj kada se nijedan slog ne podudara sa članom je predstavljen konstantom maksimalne vrednosti Double tipa i označava maksimalnu redukciju. Slučaj kada svi slogovi podudaraju neki član je predstavljen konstantom 1.0 i predstavlja odsustvo redukcije.

Izbor vrednosti ovih konstanti je opisan u *SystemR* istraživačkom radu [7] o putanjama pristupa.



Слика 21: Flowchart дијаграм рачунице редукционог фактора члана

Procena broja jedinstvenih vrednosti za izlaznu kolonu vrši se analizom predikata i pronalaženjem uslova jednakosti i taj broj je jednak:

- nula – ukoliko predikat izjednačava traženu kolonu sa dve ili više različitih konstanti. U ovom slučaju, uslov je kontradiktoran i nijedan slog neće zadovoljiti filter, pa samim tim neće biti ni jedinstvenih vrednosti;
- jedan – ukoliko predikat izjednačava traženu kolonu sa tačno jednom konstantom. Svi slogovi koji prođu filter imaju istu vrednost za tu kolonu;
- minimumu između broja jedinstvenih vrednosti te kolone i svih kolona sa kojima je izjednačena, ukoliko kolona nije izjednačena ni sa jednom konstantom, ali jeste sa jednom ili više drugih kolona. U ovom slučaju, broj jedinstvenih vrednosti tražene kolone ne može biti veći od njenog originalnog broja jedinstvenih vrednosti iz podređene klase plana, ali ne može biti veći ni od broja jedinstvenih vrednosti najrestriktivnije kolone sa kojom je izjednačena.

Procena broja *NULL* vrednosti za svaku izlaznu kolonu vrši se analizom predikata i njegovog odnosa prema *NULL* konstantama i taj broj je jednak:

- ukupnom broju izlaznih slogova ovog plana, ukoliko predikat eksplicitno izjednačava traženu kolonu sa *NULL* vrednošću, a izlazna šema dozvoljava *NULL* vrednosti za tu kolonu. U ovom slučaju, svi slogovi koji prođu filter imaju *NULL* vrednost;
- nula – ukoliko predikat izjednačava traženu kolonu sa *NULL* vrednošću, ali izlazna šema ne dozvoljava *NULL* vrednosti (nije *nullable*). U ovom slučaju, uslov je nemoguće ispuniti;
- nula – ukoliko predikat sadrži uslov koji eksplicitno isključuje *NULL* vrednosti za traženu kolonu. Svi slogovi sa *NULL* vrednostima neće proći ovakav filter;
- proporciji broju *NULL* vrednosti iz podređenog plana, ukoliko predikat ne spominje *NULL* konstantu u vezi sa traženom kolonom. U ovom slučaju, pretpostavlja se da uslov filtera ravnomerno smanjuje ukupan broj slogova i broj *NULL* vrednosti, pa se broj *NULL* vrednosti iz podređene klase plana deli sa faktorom redukcije celokupnog predikata.

Zbog implementacionih detalja *Java* jezika, može da se koristi samo u modifikujućim stablima operatora. Postoji i *SelectReadOnlyPlan* varijanta koja ima istu funkciju za *read-only* stabla operatora.

7.1.1.5 ExtendProjectPlan

ExtendProjectPlan opisuje virtuelnu tabelu sa svim projektovanim kolonama. Izlazna šema ima sve dodate kolone, a iz nje su izbrisane neprojektovane kolone. S obzirom da operacija projekcije ne dodaje nove slogove, broj blokova i broj slogova ostaju nepromenjeni i direktno se preuzimaju od podređene klase plana.

Procena broja jedinstvenih vrednosti za svaku izlaznu kolonu vrši se na osnovu složenosti izraza ili predikata koji tu kolonu definiše i taj broj je jednak:

- nula – ukoliko se traži procena za kolonu koja se ne nalazi u projekciji;

- jedan – ukoliko je izraz ili predikat konstanta (ne referencira nijednu kolonu);
- dva – ukoliko je u pitanju predikat, koji će najverovatnije imati obe moguće vrednosti;
- broju jedinstvenih vrednosti te kolone iz podređene klase plana, ukoliko izraz referencira tačno jednu kolonu. Pretpostavka je da većina transformacija nad jednom kolonom (npr. aritmetičke operacije) zadržava sličnu distribuciju vrednosti;
- ukupnom broju slogova, ukoliko izraz referencira više od jedne kolone. U ovom slučaju, pretpostavlja se da kombinacija više polja rezultuje jedinstvenom vrednošću za svaki slog.

Procena broja *NULL* vrednosti za svaku izlaznu kolonu vrši se na osnovu složenosti izraza ili predikata koji tu kolonu definiše i taj broj je jednak:

- nula – ukoliko se traži procena za kolonu koja se ne nalazi u projekciji;
- nula – ukoliko je u pitanju predikat, jer se predikat može evaluirati samo na tačno i netačno;
- nula – ukoliko šema garantuje da izraz ne može imati *NULL* vrednosti (nije *nullable*);
- nula – ukoliko je izraz jednak bilo kojoj konstanti sem *NULL* konstante;
- jedan – ukoliko je izraz jednak *NULL* konstanti;
- broju jedinstvenih vrednosti te kolone iz podređene klase plana, ukoliko izraz referencira tačno jednu kolonu. Pretpostavka je da većina transformacija nad jednom kolonom zadržava sličnu distribuciju *NULL* vrednosti;
- maksimalanom broju *NULL* vrednosti među svim referenciranim kolonama iz podređene klase plana, ukoliko izraz referencira više od jedne kolone. Pretpostavlja se da će izraz biti *NULL* ukoliko je barem jedan od operanada *NULL*, pa je maksimalan broj *NULL* vrednosti pesimistična procena.

7.1.1.6 RenamePlan

RenamePlan opisuje virtuelnu tabelu sa svim primenjenim preimenovanjima kolona. Izlazna šema sadrži sve kolone sa novim imenom i nijednu kolonu sa starim imenom. S obzirom da operacija preimenovanja ne dodaje nove slogove, broj blokova i broj slogova ostaju nepromenjeni i direktno se preuzimaju od podređene klase plana. Procena broja jedinstvenih vrednosti kolone je jednaka podređenoj klasi plana ukoliko se traži novo ime stare kolone ili ukoliko je kolona nepreimenovana, a jednaka je nuli ako se traži staro ime preimenovane kolone. Procena broja *NULL* vrednosti funkcioniše isto.

7.1.1.7 ProductPlan

ProductPlan opisuje virtuelnu tabelu koja je proizvod dve tabele. Izlazna šema sadrži sve kolone od obe tabele. Za demonstraciju računice broja blokova definišemo tabele T_1 i T_2 :

Табела 4: Primer karakteristika tabela za računanje broja blokova plana proizvoda

T_i	$B(T_i)$	$R(T_i)$	$RPB(T_i)$
T_1	5	1000	$\frac{1000}{5} = 200$
T_2	100	500	$\frac{500}{100} = 5$

- $B(T_i)$ predstavlja broj blokova neke tabele,
- $R(T_i)$ predstavlja broj slogova neke tabele,
- $RPB(T_i)$ predstavlja koliko slogova može da stane po jednom bloku za neku tabelu.

Ovaj primer se odnosi na rad sa konkretnim fizičkim tabelama, ali u generalnom slučaju, tabele nisu fizičke, već su predstavljene klasama planova sa kojima klasa plana proizvoda barata.

Da bi se prošlo kroz svaki slog rezultujuće tabele, potrebno je da se za svaki slog leve tabele prođe kroz svaki slog desne tabele. Formula koja opisuje broj blokova potreban da se ovo izvrši je sledeća [1]:

$$B(T_r) = B(T_l) + (R(T_l) * B(T_d))$$

Ako stavimo konkretne vrednosti tabela T_1 i T_2 u ovu formulu, dobijamo različite rezultate u odnosu na to koja tabela je leva, a koja desna:

- $(T_l = T_1, T_d = T_2) \Rightarrow B(T_r) = 5 + (1000 * 100) = 100005$
- $(T_l = T_2, T_d = T_1) \Rightarrow B(T_r) = 100 + (500 * 5) = 2600$

Vidi se da ako stavimo da tabela T_1 bude desna, a T_2 leva, dobijamo manji broj blokova rezultujuće tabele, a sa time i efikasniju operaciju proizvoda. Ekvivalentna formula [1]:

$$B(T_r) = B(T_l) + (RPB(T_l) * B(T_l) * B(T_d))$$

daje bolji uvid zbog čega računica broja blokova rezultujuće tabele nije simetrična u odnosu na dve tabele koje učestvuju u proizvodu. Sabirak $RPB(T_l) * B(T_l) * B(T_d)$ znatno više utiče na finalni rezultat u odnosu na $B(T_l)$.

Što je slog manji, jedan blok može da ih sadrži više. U suprotnom, što je slog veći, jedan blok može da ih sadrži manje. U tabelama gde je slog veći, potrebno je pristupiti više blokova da bi se prošlo kroz isti broj slogova kao u tabelama gde je slog manji. U operacijama proizvoda bolje je staviti tabelu gde je slog veći (to jest gde je RPB manji) na levu stranu, a tabelu gde je slog manji (to jest gde je RPB veći) na desnu stranu jer se slogovima desne tabele pristupa znatno više nego slogovima leve tabele.

Broj slogova je proizvod broja slogova obe podređene klase plana, a broj jedinstvenih i broj *NULL* vrednosti se prosleđuje podređenoj klasi plana u kom se nalazi tražena kolona.

7.1.1.8 UnionAllPlan

UnionAllPlan opisuje virtuelnu tabelu koja je zbir dve tabele. Izlazna šema je jednaka izlaznoj šemi leve podređene klase plana. Broj blokova je zbir broja blokova obe podređene klase plana, a broj slogova je zbir broja slogova obe podređene klase plana. Procena broja jedinstvenih vrednosti kolone je jednaka zbiru procena jedinstvenih vrednosti obe podređene

klase plana za tu kolonu. Procena *NULL* vrednosti kolone je jednaka zbiru procena *NULL* vrednosti obe podređene klase plana.

7.2 Planer

Većina naredbi definisanih *SQL* standardom zahteva propratno stablo relacionih operatora. Konstrukcija stabla klasa planova koje se dalje prevodi u stablo relacionih operatora i proveru semantičke validnosti naredbi su poslovi planera.

Glavna podela tehnika planiranja u relacionim bazama podataka je na tehnike praćenja striktnih pravila pravljenja planova (eng. *rule-based optimisation*, *RBO*; *heuristics-based optimisation*, *HBO*) i tehnike planiranja koji rade sa cenama (eng. *cost-based optimisation*, *CBO*). Cena predstavlja kombinaciju statističkih metapodataka relacionih operatora sa hardverskim osobinama koji ti relacioni operatori koriste.

Raniji sistemi upravljanja bazama podataka poput *INGRES* sistema su koristili *RBO* tehnike planiranja [8], dok moderni sistemi koriste *CBO* tehnike planiranja^{2,3} koje su postale popularne nakon *SystemR* istraživačkog rada o putanjama pristupa [7].

Evolucija tehnika planiranja, koja se može videti kroz ovu glavnu podelu, postoji jer je kroz istoriju bilo potrebno obezbediti sve efikasnije planere koji rade sa sve većim skupovima podataka.

7.2.1 Evaluacija izraza tokom planiranja

Evaluacija izraza i predikata u stablu relacionih operatora je najskuplje mesto evaluacije, jer se operacije izvršavaju u okviru virtuelne mašine sistema, gde se ne koriste procesorske instrukcije direktno. *PartialEvaluator* pruža obradu operacija u trenutku planiranja, što znatno povećava performansu upita jer se trivijalne operacije ne izvršavaju za svaki slog.

Trivijalne operacije koje se redukuju su: aritmetičke operacije koje ne transformišu podatke, aritmetičke operacije između dve konstante, operacije poređenja koje su uvek tačne i u slučaju da postoji kontradiktorna operacija poređenja ona skraćuje (eng. *short-circuit*) ceo predikat, čineći ga uvek netačnim bez obzira na njegove ostale komponente.

7.2.2 Ulazna tačka kreiranja i izvršavanja planova

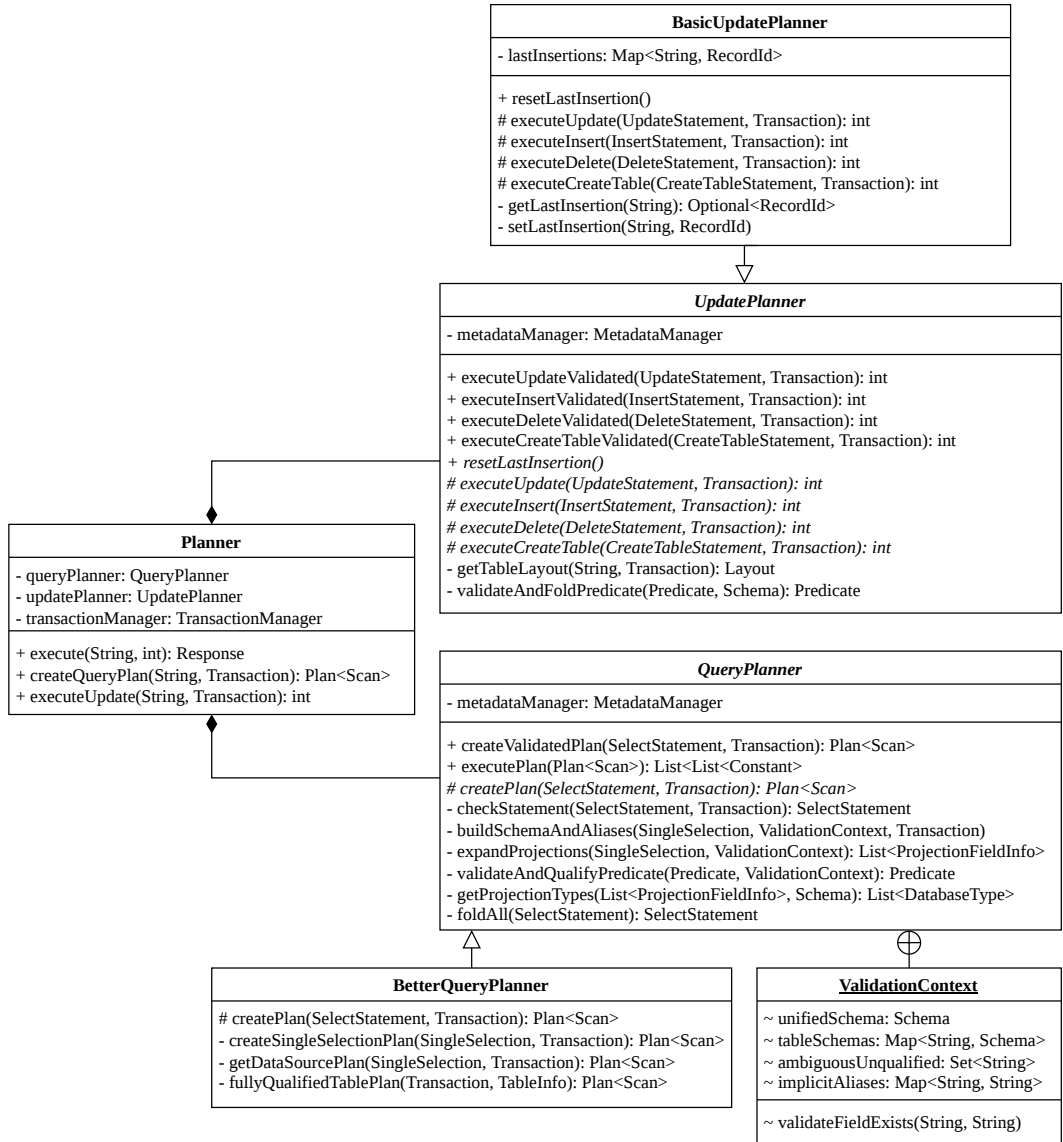
Svaka *SQL* naredba, koja je prvobitno niz karaktera, se prosleđuje *Planner* klasi, koja prvo vrši parsiranje i pretvaranje niza karaktera u objekat iskaza (*Statement*). Klasa *Planner* definiše dve grupe funkcija koje su prilagođene različitim *API* (*Application Programming Interface*) interfejsima:

- *createQueryPlan* i *executeUpdate* koje su prilagođene *JDBC* (*Java Database Connectivity*) *API* interfejsu. *JDBC* definiše generičko ponašanje za interakciju sa

²<https://www.postgresql.org/docs/current/planner-optimizer.html>

³<https://www.postgresql.org/docs/current/planner-stats-details.html>

- sistemima za upravljanje bazama podataka (ne postoji konkretna implementacija za *LBDB*, ali definisanjem ovih metoda ju je lako dodati). `createQueryPlan` kreira plan za *read-only* naredbu, ali ga ne izvršava, dok se `executeUpdate` oslanja na to da su modifikacione naredbe dizajnirane da se odmah izvrše i vraća broj promenjenih slogova,
- `execute` koja je prilagođena klijentsko-serverskoj arhitekturi (poglavlje 8) *LBDB* sistema, u okviru koje se brine o automatskom ili ručnom potvrđivanju transakcija, kreiranju i izvršavanju plana. Vraća instancu `Response` objekta (sekcija 8.1.1), koja predstavlja sve moguće vrste odgovora na neku naredbu.



Слика 22: Структура planera

7.2.3 Planiranje *read-only* naredbi

SQL naredbe se dele na *read-only* i modifikacione. Glavni primer *read-only* naredbe je SELECT naredba, koja služi za struktuirano upitivanje (eng. *query*) baze podataka.

Svaka SELECT naredba prvo mora proći semantičku proveru pre pravljenja samog plana. Semantička provera se sastoji od sledećih koraka:

- provera postojanja fizičkih tabela spomenutih u naredbi
- proširenje zamenskih članova na konkretne kolone
- provera da se zamenski članovi ne koriste u izrazima
- provera postojanja kolona spomenutih u projekcijama i predikatu
- provera dvosmislenih imena kolona (u slučaju da dve tabele imaju isti naziv kolone i ne može da se trivijalno razume na koju kolonu je korisnik mislio)
- provera da li aritmetičke operacije mogu da se izvrše za tip kolone
- provera da li kolone u unijama imaju iste tipove

QueryPlanner apstraktna klasa pruža implementaciju semantičke provere, a konkretni algoritmi planiranja SELECT naredbe koji je nasleđuju mogu da podrazumevaju da su naredbe koje dobiju sigurno semantički validne. QueryPlanner takođe redukuje sve izraze i predikat pomoću PartialEvaluator klase.

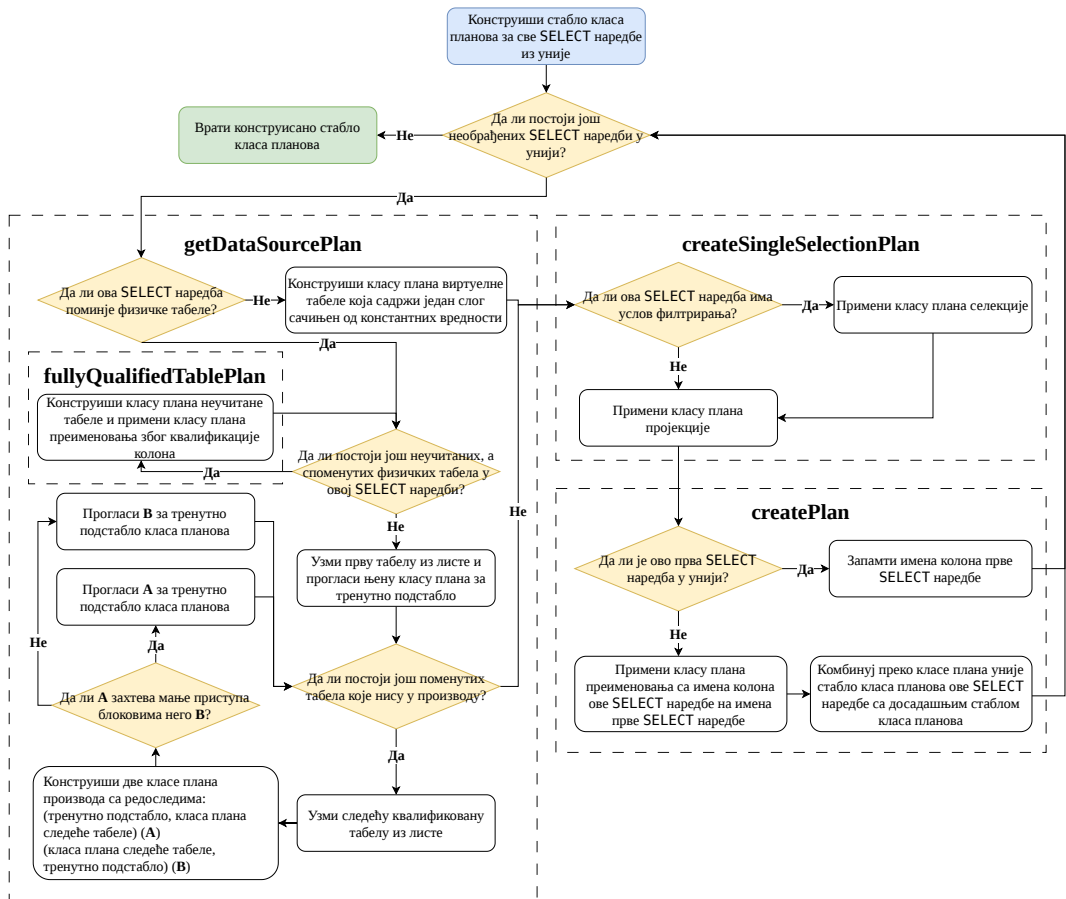
7.2.3.1 Algoritam planiranja SELECT naredbi

BetterQueryPlanner klasa nasleđuje QueryPlanner i predstavlja implementaciju osnovnog planera koji podržava sve alternative SELECT naredbe predstavljene u njenoj gramatici (sekcija 6.2.2.3).

Stablo relacionih operatora je korektno (eng. *sound*), ako svi slogovi koje ono proizvodi ispunjavaju sve uslove relacionih operacija definisanih nekom SELECT naredbom. Stablo planova, koje se prevodi u stablo relacionih operatora, koje BetterQueryPlanner konstruiše je uvek korektno.

Problem BetterQueryPlanner implementacije je niska efikasnost konstruisanih stabala planova, jer ne koristi ni tehnike *RBO* planiranja, ni tehnike *CBO* planiranja, već izvršava samo minimalni skup koraka koji su neophodni da se obezbedi korektnost.

Algoritam planiranja se može videti na figuri 23.

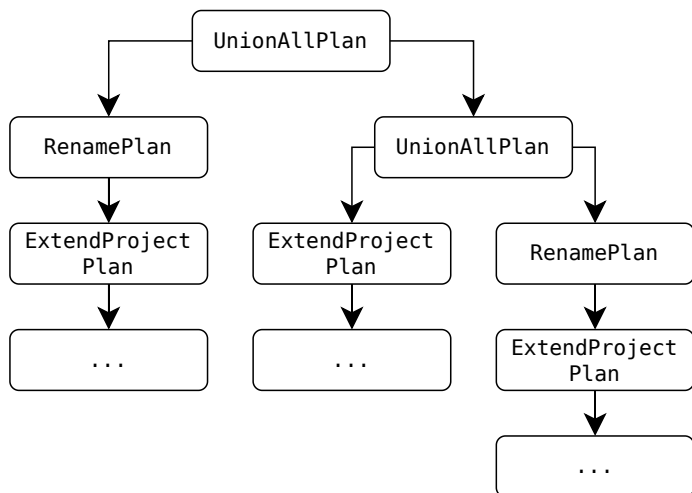


Слика 23: Flowchart дијаграм BetterQueryPlanner алгорита планирања

Kreira finalno stablo planova kroz četiri funkcije koje se međusobno pozivaju, imaju rastući prioritet i zadužene su za različite operacije:

- `createPlan` funkcija je ulazna tačka algoritma, definisana u `QueryPlanner` klasi i ima zaduženje kreiranja unija više `SELECT` naredbi, u slučaju postojanja `UNION ALL` gramatičke alternative.

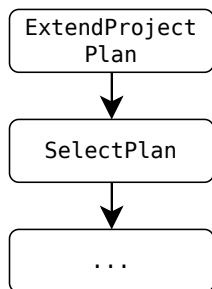
Unije `SELECT` naredbi zahtevaju da se kolonama svake `SELECT` naredbe pristupa pomoću imena datih u prvoj `SELECT` naredbi. Zbog ovoga se dodaje operator preimenovanja ispred svakog podstabla planova svake naredbe u uniji, sem prve. Operacija unije ima najmanji prioritet, pa se poslednja izvršava. Ukoliko ne postoji `UNION ALL` gramatička alternativa, funkcija vraća podstablo planova jedine `SELECT` naredbe.



Слика 24: Primer stabla planova nakon unije 3 naredbe

- `createSingleSelectionPlan` funkcija obezbeđuje korektnost filtriranja i projekcije.

Dodaje operator filtriranja samo ako filter postoji; dodaje nove virtuelne kolone i briše kolone koje nisu spomenute.



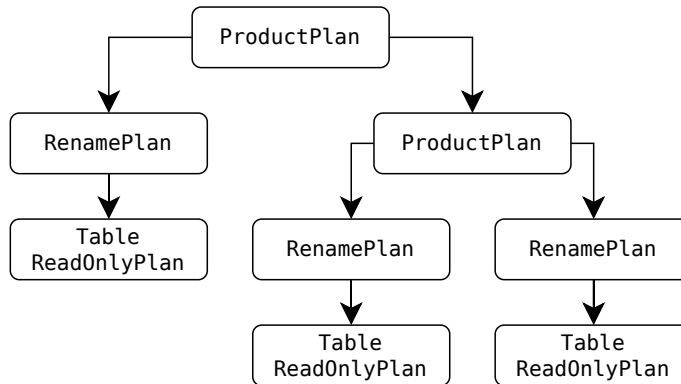
Слика 25: Primer stabla planova nakon filtriranja i projekcije

- `getDataSourcePlan` funkcija se brine o tome odakle će doći slogovi i ima dve putanje izvršavanja.

Prva putanja izvršavanja se dešava kada se u `SELECT` naredbi ne spominje nijedna fizička tabela, već se radi upit virtuelne tabele koja ima jedan slog koji se sastoji samo od konstanti. U tom slučaju, samo vraća `DummyTablePlan` klasu plana.

Druga putanja izvršavanja se dešava kada se spominje jedna ili više fizičkih tabela. Ako se spominje jedna fizička tabela, njen plan biva vraćen. Ako se spominje više od jedne fizičke tabele, potrebno je uraditi operaciju proizvoda. Proizvod tabela se vrši tako što se prva spomenuta tabela proglasi da bude početna, pa se prolazi kroz sve ostale spomenute tabele i ponavlja se postupak: kreiraju se dva proizvod plana, jedan gde je dosadašnje podstablo planova na levom mestu, a plan sledeće tabele na desnom i jedan gde je redosled

obrnut. Plan koji ima manje pristupa blokovima se uzima kao sledeći koren podstabla planova i postupak se ponavlja dok se ne prođe kroz sve spomenute tabele. Time se dobija oformljeno podstablo planova gde su proizvodi tabela zadovoljeni. Ovakva provera broja pristupanih blokova nije optimalna, ali može pomoći u otklanjanju veoma neefikasnih stabala planova i predstavlja jedino mesto gde se primenjuje *RBO* tehnika planiranja.



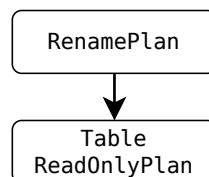
Слика 26: Primer stabla planova nakon proizvoda 3 tabele

- `fullyQualifiedTablePlan` obezbeđuje davanje punokvalifikujućih imena kolonama fizičkih tabela.

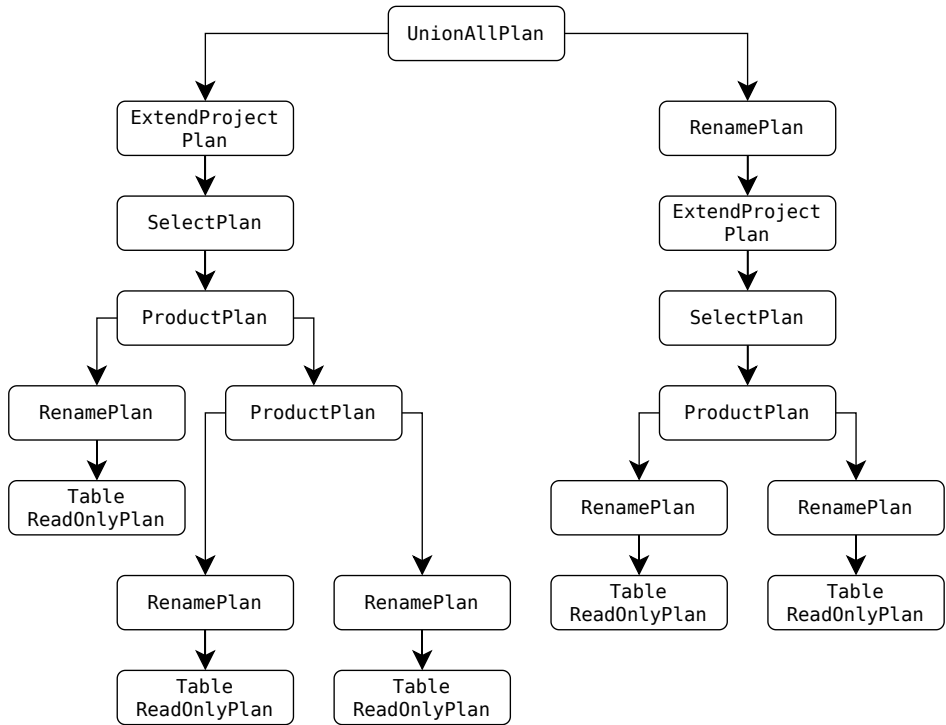
U slučajevima gde se vrši proizvod dve ili više tabele, postoji mogućnost da te tabele imaju istoimenu kolonu. Ovo je čest slučaj jer povećava čitljivost upita i strukture tabela, pa je za njega potrebno pružiti adekvatnu podršku.

U prethodnoj funkciji koja vrši proizvode, rečeno je da se direktno barata sa planovima tabela. Ovo nije precizno, jer se ne barata direktno sa tabelama, već sa podstablom planova koje predstavlja tabelu sa punokvalifikovanim imenama kolona. Kvalifikacija imena kolona se vrši preko operatora preimenovanja, tako što se pre samog imena kolone doda ime tabele i tačka (`ime_kolone` postaje `ime_tabele.ime_kolone`). Dozvoljava i preimenovanje tabela u slučaju da vršimo proizvod dve (ili više) iste tabele.

Ako se proizvodu nalaze samo tabele sa sa punokvalifikovanim kolonama, ne postoji mogućnost da sistem ne može da prepozna kojoj tabeli kolona pripada, sem ako korisnik nije zadao semantički neispravnu naredbu.



Слика 27: Primer stabla planova nakon kvalifikacije kolona tabele



Слика 28: Primer kompletnog stabla planova

7.2.3.2 Automatsko generisanje opisa planova

Drugi primer *read-only* naredbe je EXPLAIN naredba. Njena uloga u sistemu je tabelarno ispisivanje kompletnih stabla planova. EXPLAIN naredba se poziva tako što se doda ključna reč EXPLAIN ispred SELECT naredbe. Nije podržana za modifikacione naredbe.

EXPLAIN naredba je implementirana tako da generiše slogove koji prate šemu specijalne tabele koja ima sledeće kolone: ime relacionog operatora, procena kroz koliko blokova će taj relacioni operator proći da generiše sve slogove, procena broja slogova i specijalni detalji. Svaki slog predstavlja čvor rezultujućeg stabla relacionih operatora. Iako je poenta naredbe tabelarni prikaz stabla, naredba samo generiše ove slogove i ne brine se o formatiranju tabele (sekcija 8.3.1).

Scan	Block est.	Record est.	Details
ExtendProjectScan	5	52	
└─ SelectScan	5	52	student.isactive = TRUE
└─┬─ RenameScan	5	103	
└─└─ TableScan	5	103	'student'

Листинг 4: Primer rezultujuće tabele EXPLAIN naredbe

7.2.4 Planiranje modifikacionih naredbi

Modifikacione naredbe su razne, a UpdatePlanner ima istu ulogu za njih, kao što QueryPlanner ima za SELECT naredbu, a to je samo semantička provera. Konkretni algoritmi planiranja modifikacionih naredbi mogu da podrazumevaju da je naredba semantički validna i da su izrazi i predikati redukovani pomoću PartialEvaluator klase. Za svaku modifikacionu naredbu su opisani koraci za semantičku proveru.

INSERT naredba služi za umetanje novih slogova. Semantička provera INSERT naredbi se sastoji od sledećih koraka:

- da li postoji fizička tabela u koju se umeću novi slogovi
- da li se broj kolona novih slogova podudara sa brojem definisanim u šemi tabele
- proveru tipova kolona novih slogova sa tipovima kolona definisanih u šemi tabele
- da li su dozvoljene *NULL* vrednosti za kolone gde je vrednost novih slogova *NULL*

UPDATE naredba služi za ažuriranje vrednosti postojećih slogova na osnovu nekog uslova filtriranja. Semantička provera UPDATE naredbi se sastoji od sledećih koraka:

- da li postoji fizička tabela čiji se slogovi ažuriraju
- da li postoje kolone spomenute u predikatu i izrazima ažuriranja
- da li su dozvoljene *NULL* vrednosti za kolone gde je nova vrednost *NULL*
- proveru tipova kolona ažuriranih slogova sa tipovima kolona definisanih u šemi tabele

DELETE naredba služi za brisanje postojećih slogova na osnovu nekog uslova filtriranja. Semantička provera DELETE naredbi se sastoji od sledećih koraka:

- da li postoji fizička tabela čiji se slogovi brišu
- da li postoje kolone spomenute u predikatu

CREATE TABLE naredba služi za kreiranje novih tabela. Semantička provera CREATE TABLE naredbi se sastoji od sledećih koraka:

- da li tabela sa tim imenom već postoji
- da li je veličina sloga prešla maksimalnu veličinu (ograničenje opisano u sekciji [11.2.2](#))

7.2.4.1 Algoritam planiranja INSERT naredbe

Stablo planova za INSERT naredbe je uvek isto i sastoji se samo od TablePlan klase plana. Taj čvor se pretvara u svoj prateći relacioni operator nad kojim se vrše umetanja novih slogova. Vraća broj dodatih slogova.

Algoritam umetanja novog sloga implementiran u TableScan operatoru funkcioniše tako što traži prvo slobodno mesto za novi slog, ali počevši od pozicije trenutnog sloga tog TableScan objekta. Nakon što se operator inicijalizuje, pozicioniran je na početku tabele, to jest pre prvog sloga. Ovo znači da će umetanje prvog sloga u listi novih slogova uvek kretati od početka. Prednost ovog pristupa je to što će obrisani slogovi brzo biti ponovo popunjeni, pa se prostor maksimalno iskorišćava. Mana ovog pristupa je to što umetanje prvog novog sloga može da potraje, jer u najgorem slučaju mora da se prođe kroz sve slogove tabele da

se pronade prazno mesto. Drugi način implementacije algoritma je da se umetanje novih slogova uvek vrši na kraju. Prednost je dobra brzina umetanja, jer se preskače pretraga za slobodno mesto. Mana je to što se sve više i više prostora troši na obrisane slogove.

Implementirano rešenje je kompromis ova dva algoritma, gde se za svaku tabelu pamti pozicija poslednjeg umetnutog sloga i novi slogovi se umeću od te pozicije. Pamćenje pozicija poslednje umetnutih slogova važi samo dok je sistem upaljen i resetuje se prilikom gašenja sistema. Zadržava prednost brzine umetanja, a nakon restarta sistema mesta obrisanih slogova mogu ponovo biti popunjena. Takođe, resetuje se pri poništavanju transakcije.

7.2.4.2 Algoritam planiranja UPDATE naredbe

Stablo planova za UPDATE naredbe se može sastojati samo od `TablePlan` klase plana, ali ispred njega može stajati i `SelectReadOnlyPlan` klasa plana u slučaju da slogovi koji trebaju biti ažurirani moraju da ispune neki uslov filtriranja. Ove dve (ili jedan) klase plana se pretvaraju u svoje prateće relacione operatore koji znaju da postavljaju nove vrednosti. Vraća broj ažuriranih slogova.

Za razliku od INSERT naredbe, UPDATE naredba može da sadrži izraze koji nisu konstantni, to jest koji spominju kolone tabele koja se ažurira.

7.2.4.3 Algoritam planiranja DELETE naredbe

Stablo planova za DELETE naredbe se može sastojati samo od `TablePlan` klase plana, ali ispred njega može stajati i `SelectReadOnlyPlan` klasa plana u slučaju da ne trebaju da se obrišu svi slogovi iz tabele već samo oni koji ispunjavaju uslov filtriranja. Vraća broj obrisanih slogova.

Brisanje nekog sloga samo označava mesto gde se taj slog nalazio kao slobodno za umetanje novog sloga, umesto da radi kompresiju datoteke i zapravo izvrši fizičko brisanje.

7.2.4.4 Algoritam planiranja CREATE TABLE naredbe

CREATE TABLE naredba je specijalna vrsta naredbe jer ne modifikuje slogove običnih tabela, već slogove kataloških tabela (sekcija 4.1). Dodaje jedan slog u katalošku tabelu koja pamti sve postojeće tabele. Dodaje slog za svaku kolonu nove tabele u katalošku tabelu koja pamti sve postojeće kolone. Pošto ne utiče na slogove običnih tabela, uvek vraća nula za broj slogova na koje je uticala.

Menadžer metapodataka tabela interno konstruiše tri `TableScan` objekta: prvi koristi da proveriti jedinstvenost imena nove tabele, drugi koristi da umetne slog za novu tabelu u `tablecatalog` katalošku tabelu, a treći koristi da umetne slogove novih kolona u `fieldcatalog` katalošku tabelu. Posledica ovakve implementacije je ta da `UpdatePlanner` apstraktna klasa zapravo ne vrši semantičku proveru pre pozivanja algoritma planiranja CREATE TABLE naredbe, nego reaguje na greške i transformiše ih, ako se dese.

Глава 8

Klijentsko-serverska arhitektura

Postoje dva glavna načina kako sistem upravljanja relacionim bazama podataka može raditi: lokalno, bez mrežne infrastrukture (eng. *embedded connection*) i kao server.

Karakteristike sistema u lokalnom režimu rada:

- radi u istom procesu operativnog sistema kao i aplikacija koja ga koristi,
- samo jedna aplikacija može da koristi tu instancu sistema,
- ne zahteva mrežnu konekciju,
- zahteva manje resursa,
- ponaša se kao biblioteka koja razume unutrašnjosti relacionog modela podataka.

Karakteristike sistema u serverskom režimu rada:

- radi kao zaseban proces operativnog sistema,
- više različitih aplikacija i korisnika može da pristupi toj instanci sistema,
- zahteva mrežnu konekciju,
- zahteva više resursa,
- ponaša se kao pružilac usluge upravljanja relacionim modelom podataka.

LBDB sistem podržava samo serverski režim rada.

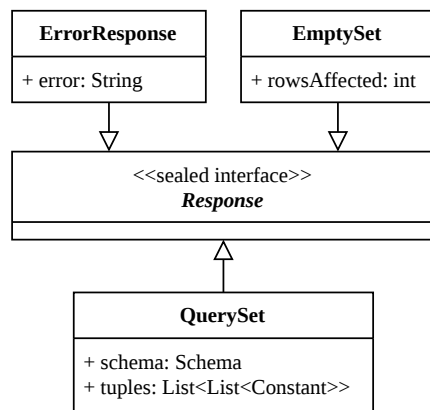
8.1 Komunikacija sa klijentima

Kod serverskog režima rada, pretpostavlja se da je klijent na udaljenom računaru i nema pristup resursima računara na kom se pokreće server. Posledica ovog je da sva komunikacija mora da se vrši kroz mrežu, preko nekog protokola komunikacije.

8.1.1 Odgovori sistema

Sve vrste odgovora koje server može dati za neku naredbu koju je klijent zadao predstavljene su *Java record* konstruktom i implementiraju *Response sealed interface*.

- *QuerySet* predstavlja odgovor na *read-only* naredbe, čiji je rezultat skup slogova. Jedan slog je predstavljen listom konstanti. Da bi sistem znao kako da pošalje slogove preko mreže, a kasnije i kako da napravi tabelarni prikaz, potrebno je poslati i prpratnu šemu koja opisuje poslate slogove.
- *EmptySet* predstavlja odgovor na modifikacione naredbe. Sadrži samo podatak o tome na koliko slogova je uticano.
- *ErrorResponse* se vraća klijentu kada se desi greška prilikom parsiranja, prilikom planiranja ili prilikom izvršavanja naredbe.



Слика 29: Struktura Response hijerarhije

8.1.2 Protokol serijalizacije

Preko mreže je moguće slati samo niz bajtova. Pošto Response objekti nisu podrazumevano predstavljeni nizovima bajtova, za njih se mora definisati način serijalizacije i deserijalizacije.

Protokol serijalizacije Response objekata je inspirisan *RESP (REdis Serialization Protocol)*⁴ protokolom. Svakom tipu se dodeljuje jedan bajt koji ga jedinstveno predstavlja. '*' za QuerySet, '_' za EmptySet i '-' za ErrorResponse.

Brojčane vrednosti se pretvaraju u bajtove byte tipa koji se sastoji od jednog bajta, u bajtove short tipa koji se sastoji od 2 bajta ili u bajtove int tipa koji se sastoji od 4 bajta. String vrednosti se pretvaraju u bajtove uz *UTF-8 (Unicode Transformation Format)* kodiranje.

EmptySet i ErrorResponse je trivijalno serijalizovati i deserijalizovati, jer se sastoje od samo jedne vrednosti.

QuerySet sadrži dosta vrednosti koje imaju specifičan format, pa je algoritam serijalizacije kompleksniji (svaka vrednost je podrazumevano zapisana u bajtovima):

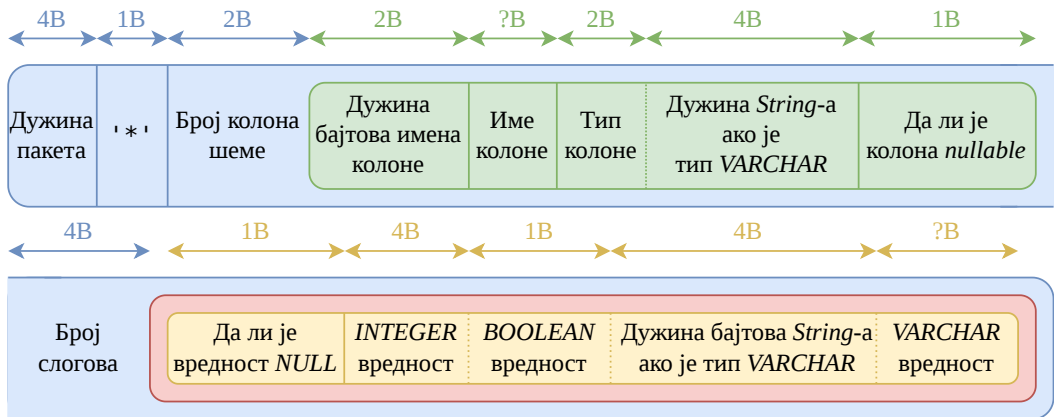
- zapisuje se jedinstveni identifikator QuerySet tipa: '*'
- zapisuje se količina kolona šeme rezultujuće tabele kao short
- za svaku kolonu se zapisuje:
 - dužina bajtova imena, kao short
 - kodirano ime kolone
 - tip kolone, kao short, ako je tip *VARCHAR* zapisuje se i njegova dužina, kao int
 - da li je kolona *nullable*, kao byte
- zapisuje se količina slogova
- za svaku vrednost svakog sloga se zapisuje:

⁴<https://redis.io/docs/latest/develop/reference/protocol-spec/>

- 0 ako je *NULL*, 1 ako nije *NULL*, kao short
- brojnu vrednost, kao int ako je tip *INTEGER*, brojnu vrednost, kao byte ako je tip *BOOLEAN*, dužina *String*-a, kao int tip zajedno sa kodiranim *String*-om ako je tip *VARCHAR*

Na kraju serijalizacije se računa dužina bajtova i ona se stavlja pre svih bajtova, da bi se prilikom deserijalizacije znalo koliko bajtova da se pročita.

Algoritam deserijalizacije za sve tipove isto funkcioniše, ali u suprotnom smeru.



Слика 30: Структура QuerySet paketa

Na slici plavo predstavlja ceo paket, zeleno predstavlja deo paketa koji se ponavlja za svaku kolonu šeme, crveno predstavlja deo paketa koji se ponavlja za svaki slog, a žuto predstavlja deo paketa koji se ponavlja za svaku vrednost sloga.

8.2 Serverski sloj

Klasa *LBDBServer* sadrži main funkciju serverske aplikacije sistema. Čita i validira argumente komandne linije: port na kom će server raditi i putanja gde će se čuvati datoteke sistema. Pokreće instancu servera sa prosleđenim argumentima. Sva logika koja podržava serverske operacije se nalazi u *Server* klasi.

8.2.1 Rukovođenje klijentima

Prva funkcionalnost servera je obrada klijentskih konekcija i naredbi koje one šalju. Prilikom pokretanja servera, instancira se serverski *socket* koji prihvata konekcije na određenom portu i otvara klijentski *socket* za svakog klijenta.

Pošto su klijenti nezavisni, ne bi trebalo da čekaju jedni druge i zbog toga se obrađuju u zasebnim nitima. Zarad lakšeg upravljanja, postoji *ThreadPool* sa 8 fiksnih, stalno postojećih niti, koje čim završe sa obradom jednog zahteva ponovo postaju slobodne.

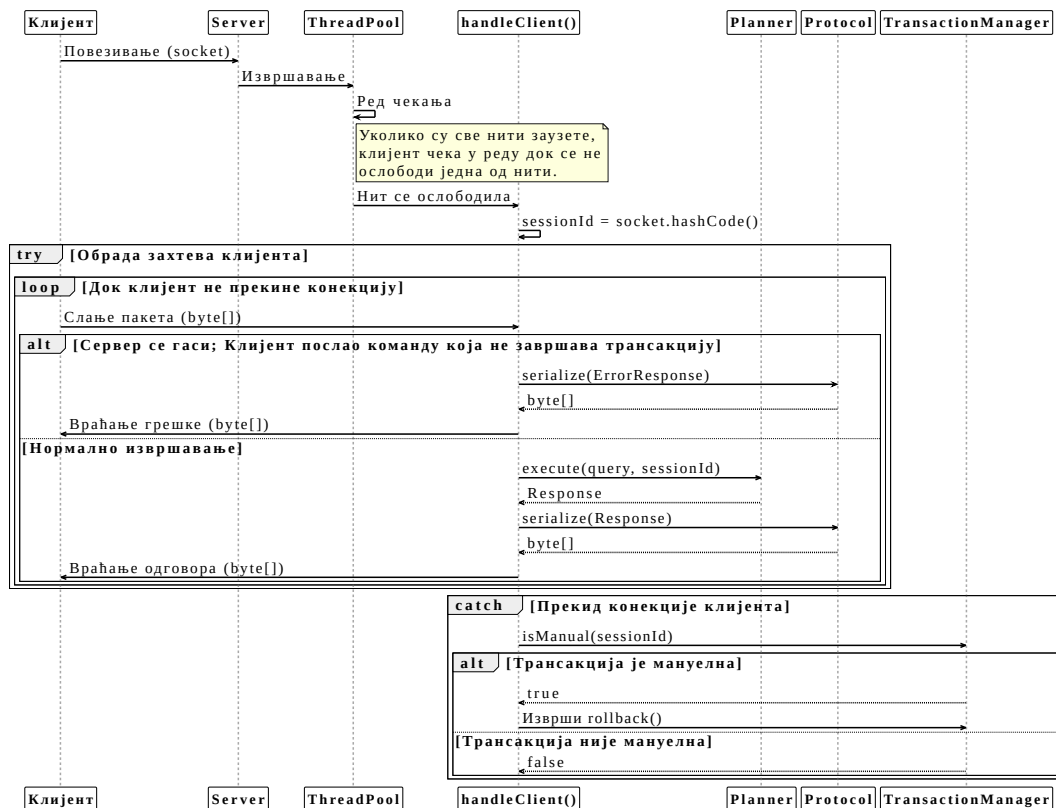
8.2.1.1 Obrada jednog klijenta

Obrada zahteva klijenata se vrši kroz `handleClient()` funkciju. Svaki klijentski *socket* se kodira u jedinstveni broj sesije, tako što se izračuna heš vrednost njegove konekcije. Ovim se omogućava mapiranje klijenta na njegovu trenutnu transakciju. Ako broj sesije klijenta ne postoji u sistemu, dodeljuje se nova (prva) transakcija za tog klijenta.

Klijentske transakcije mogu da rade u režimu gde se sastoje od jedne naredbe (*autocommit*) ili u režimu gde se sastoje od više naredbi. U režimu gde se transakcije sastoje od više naredbi, potrebno je početi ih sa `START TRANSACTION` naredbom, a završiti sa `COMMIT` ili `ROLLBACK` naredbama. Ovo je glavni razlog zašto je potrebno jedinstveno identifikovati sesije klijenata, da bi njihova transakcija mogla da perzistira kroz više naredbi.

U slučaju prekida konekcije, server će izvršiti `ROLLBACK` trenutne transakcije klijenta.

U slučaju da se server gasi, povezani klijenti mogu da pošalju samo naredbe koje završavaju transakcije, ali više o tome u opisu gašenja servera.



Слика 31: Dijagram sekvence obrade klijenta

8.2.2 Pisanje kontrolnih tačaka

Druga funkcionalnost servera je određivanje kada (ali ne i kako) će mirna kontrolna tačka biti pisana. Prilikom pokretanja servera startuje se i nit koja na svakih 10 minuta započinje pisanje mirne kontrolne tačke. Da bi se mirna kontrolna tačka zapisala, nijedna transakcija ne sme biti aktivna u sistemu (sekcija 3.2.1).

Kada prođe 10 minuta od poslednjeg zapisa mirne kontrolne tačke, server poziva algoritam zapisa koji interno čeka da se sve transakcije završe i zaustavlja obradu novih.

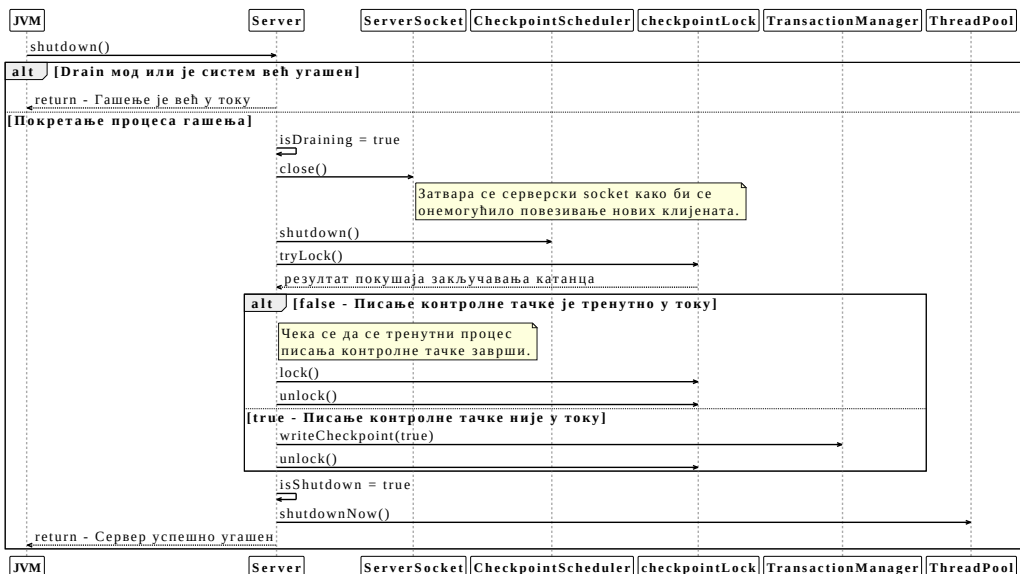
8.2.3 Gašenje sistema

Treća funkcionalnost je bezbedno gašenje sistema. Bezbedno gašenje se inicira slanjem SIGINT signala na Unix operativnim sistemima ili slanjem CTRL_C (ili sličnog) signala na Windows operativnom sistemu. Najčešće, ovo se mapira na gašenje prozora gde je server pokrenut, ili slanjem signala preko CTRL + C prečice.

Bezbedno gašenje se sastoji iz tri koraka:

- prestajanje prihvatanja novih naredbi, sem naredbi završetka transakcije
- čekanje da se završe sve transakcije
- zapisivanje mirne kontrolne tačke

Nakon što je gašenje inicirano, ulazi se u drain mod, gde se ne prihvataju konekcije novih klijenata, ali starim klijentima je dozvoljeno da završe svoje transakcije preko COMMIT ili ROLLBACK naredbi. U drain modu, samo ove naredbe su dozvoljene. Ako je server već u procesu pisanja mirne kontrolne tačke u trenutku slanja zahteva za gašenje, neće se pisati još jedna.



Слика 32: Дијаграм секвенце безбедног гашења сервера

8.3 Klijentski sloj

Klasa `LBDBClient` sadrži `main` funkciju klijentske aplikacije sistema. Čita i validira argument komandne linije: `port` na kojem se nalazi server na koji se klijent povezuje. Sva logika slanja naredbi se nalazi u ovoj klasi.

Klijentska aplikacija pruža korisnicima terminal gde se naredbe mogu upisivati. Terminal podržava automatsko završavanje ključnih reči (eng. *auto complete*) pritiskom `TAB` tastera i navigaciju istorije komandi. Terminal prati i koliko vremena se izvršavala svaka naredba.

Paketi koje šalje serveru su serijalizovani tako da je na prvom mestu dužina teksta naredbe u bajtovima, a zatim i kodirani tekst naredbe. Pakete koje prima od servera deserijalizuje tako što čita prva 4 bajta koja predstavljaju dužinu paketa u bajtovima, a zatim koristi algoritam deserijalizacije koji je opisan u protokolu komunikacije (sekcija 8.1.2).

Podržava ispis sve tri vrste `Response` objekata, gde se `ErrorResponse` i `EmptySet` trivijalno prikazuju jer sadrže samo jednu vrednost. `QuerySet` objekti su tabele i njihovo prikazivanje se radi algoritmom štampanja tabela.

8.3.1 Štampanje tabela

Algoritam štampanja tabela ima zadatak da formatira sva imena kolona i sve vrednosti slogova iz `QuerySet` objekta. U tom objektu stoji šema tabele, pa ovo nije zahtevan zadatak. Koristi `StringBuilder` za efikasno kreiranje *String*-ova. Za lepši prikaz koristi specijalne *Unicode* karaktere kao što su: `▬`, `▮`, `▯`, `▰`, `▱`, `▲`, `△`, `▴`, `▵`, `▶`, `▷`.

tableid	tablename	slotsize
2	tablecatalog	112
3	fieldcatalog	121
5	department	164
6	professor	173
7	student	173
8	course	172
9	enrollment	20

Листинг 5: Primer ispisane tabele `SELECT * FROM tablecatalog;` naredbe

8.3.2 Masovno pokretanje naredbi

`LBDB` paket pruža još jednu vrstu klijentske aplikacije: `BulkExecutor`. Ova klijentska aplikacija funkcioniše slično kao i obična klijentska aplikacija, ali umesto pružanja interakcije sa sistemom preko terminala, redom izvršava sve *SQL* naredbe iz neke datoteke. Ovo radi u ručno započetoj transakciji i ako bar jedna naredba ne uspe sa izvršavanjem, javlja grešku i vrši *rollback*. Korisna je za popunjavanje tabela ili za testiranje sistema.

Pošto je za korektno funkcionisanje sistema potrebno mnogo kompleksnih funkcionalnosti i algoritama, potrebno je izvršiti intenzivno testiranje istih da bi se dokazala pravilna i efikasna implementacija. Slojevi od kojih se sistem sastoji su testirani izolovano, s tim da se slojevi višeg apstrakcionog nivoa ne testiraju odvojeno od slojeva nižeg apstrakcionog nivoa.

9.1 Organizacija testova

Sve testove u sistemu podržava *JUnit*⁵ biblioteka. *JUnit* sadrži razne konfiguracione parametre, a za testiranje *LBDB* sistema su najbitniji parametri koji omogućavaju definisanje čistača i parametri koji omogućavaju paralelno pokretanje testova.

```
junit.jupiter.extensions.autodetection.enabled = true
junit.jupiter.execution.parallel.enabled = true
junit.jupiter.execution.parallel.mode.default = concurrent
```

ЛИСТИНГ 6: Konfiguracioni parametri *JUnit* biblioteke

Sistem prati standardnu definiciju strukture direktorijuma izvornog koda *Maven*⁶ sistema za upravljanje zavisnostima. Više o njemu u o pregledu sistema (sekcija 10.1). Po *Maven*-u, testovi se nalaze unutar `src/test/java` direktorijuma, a konfiguracioni parametri *JUnit* biblioteke se nalaze unutar `src/test/resources` direktorijuma. Testovi su grupisani po istim modulima kao i glavni izvorni kod.

9.1.1 Testno okruženje

Da bi se postiglo korektno i uniformno testiranje svih funkcionalnosti, potrebno je pružiti im odgovarajuće testno okruženje. Pošto većina funkcionalnosti zahteva rad sa datotekama, glavna dužnost testnog okruženja je da izoluje direktorijume gde će se ove datoteke nalaziti. Time se postiže da test *A* koji kreira na primer tri tabele ne može da utiče na test *B* koji kreira dve tabele gde se imena tabela poklapaju.

`TestUtils` je pomoćna klasa koja pruža implementaciju ove izolacije, ali pruža i dodatne pomoćne metode koje olakšavaju testiranje:

- proveru postojanja datoteka,
- dobavljanje privatnih polja putem *Java* refleksije.

⁵<https://junit.org/>

⁶<https://maven.apache.org/>

9.1.2 Sistem izolacije direktorijuma na disku

Prvi od dva načina pokretanja testova je u okviru direktorijuma koji se nalaze na fizičkom disku. Prednosti ovog načina pokretanja su laki pregled generisanih datoteka zarad otklanjanja grešaka i nezahtevno pokretanje. Mana ovog načina pokretanja je brzina jer je pristup fizičkom disku spor.

Da bi se postigla izolacija i kroz iteracije pokretanja istih testova, potrebno je očistiti stare direktorijume. *JUnit* omogućava konfiguraciju čistača, to jest funkcije koja se izvršava pre svih testova. *GlobalCleanup* klasa sadrži ovu logiku.

9.1.3 Sistem izolacije direktorijuma u radnoj memoriji

Drugi od dva načina pokretanja testova je u okviru direktorijuma koji se nalaze u radnoj memoriji. Pošto je *LBDB* sistem kompatibilan sa *java.nio.file API*-jem, rukovođenje direktorijumima u radnoj memoriji se vrši preko *Jimfs* biblioteke. Prednost ovog načina pokretanja je brzina testova. Mane ovog načina pokretanja su težak pristup datotekama zarad otklanjanja grešaka i velika potrošnja radne memorije.

U okviru *TestUtils* klase se podešava način pokretanja testova, gde je pokretanje u radnoj memoriji podrazumevano podešeno. *TestUtils* definiše jednu instancu *Jimfs* implementacije *Filesystem* klase koja se koristi za sve testove. Nema potrebe za čišćenjem preko *GlobalCleanup* klase jer se radna memorija sama čisti kada se proces u kom su pokrenuti testovi završi.

9.2 Funkcionalni testovi

Funkcionalni testovi potvrđuju da li su algoritmi i strukture podataka korektni, to jest da li imaju smisleno i tačno ponašanje.

Za podsistem relacionih operatora, postoji pomoćna klasa *QueryTestUtils* koja pruža dodatne pomoćne metode za testiranje ovog podsistema:

- inicijalizacija i popunjavanje jedne tabele sa 250 slogova, koja ima tri *Integer*, tri *String* i tri *Boolean* kolone,
- inicijalizacija i popunjavanje dve tabele sa po 250 slogova koje imaju istu šemu kao tabela u prethodnoj pomoćnoj metodi.

Za podsistem planiranja, postoji pomoćna klasa *PlanTestUtils* koja pruža dodatne pomoćne metode za testiranje ovog podsistema:

- pokretanje **samo** provere *SELECT* naredbe i vraćanje proširenog *SelectStatement* objekta,
- pokretanje **samo** provere modifikacionih naredbi,
- pravljenje stabla planova za *SELECT* naredbe,
- pokretanje modifikacionih naredbi,
- kreiranje novih transakcija, korisno za proveru izolacije,

- inicijalizacija tri prazne ili tri popunjene tabele, gde prve dve imaju istu šemu kao kod pomoćne klase za testiranje podsistema relacionih operatora, a treća sadrži samo jedno *Integer* polje i korisna je za testiranje spajanja tabela samih sa sobom.

9.3 Testiranje performansi

Testovi performanse imaju zadatak da potvrde pretpostavke iz teksta rada. Za korektno izvršavanje testova performansi, potrebno je pružiti mehanizme pokretanja koji vrše dodatnu izolaciju. Testovi performanse uvek moraju da se izvršavaju serijski, u zasebnim procesima i na disku. *TestUtils* već pruža mogućnost izbora izvršavanja na disku, dok se serijsko izvršavanje i izvršavanje u zasebnim procesima konfiguriše preko *Maven* sistema i *JUnit* biblioteke. Bitna napomena je da je vreme izvršavanja zavisno od hardvera, dok ostale metrike nisu, ali vreme izvršavanja i dalje može pokazati korisne uvide. Svaki test se pokreće deset puta i prikazane vrednosti su medijalne da bi se izbegao šum.

Da bi *Maven* instancirao nov *JVM* proces za svaki test, neophodno je da svaki test stoji u zasebnoj klasi. Pošto svi testovi performanse jednog dela sistema strukturalno izgledaju isto, najbolji način da se ovo obezbedi je da se definiše jedna apstraktna klasa u kojoj stoji logika testa i po jedna klasa naslednica za svaku željenu kombinaciju parametara testa. Listing 7 pokazuje *JUnit* anotacije koje se vezuju za apstraktnu klasu i koji su neophodni za serijsko pokretanje.

```
@Execution(ExecutionMode.SAME_THREAD)
@EnabledIfSystemProperty(named = "benchmark", matches = ".*")
```

Листинг 7: Podešavanja apstraktne klase testa performanse

9.3.1 Testovi performanse algoritama smene bafera

Postoje četiri tipa testova koji imaju funkciju testiranja algoritama smene bafera pod različitim okolnostima. Tipovi su opisani naredbom koja se izvršava i količinom slogova u tabelama. Tipovi testova koji modifikuju podatke (sufiks *_M*) imaju dodatnu logiku pokretanja dve *UPDATE* naredbe koja će učiniti bafere obe tabele “prljavim” i time dati mogućnost izbora na osnovu tog podatka.

```
public enum BufferTestType {
    HOT_BUFFERS("SELECT * FROM table1, table2;", 1000, 50),
    CONTINUOUS_READS("SELECT * FROM table1;", 50000, 0),
    HOT_BUFFERS_M("SELECT * FROM table1, table2;", 1000, 50),
    CONTINUOUS_READS_M("SELECT * FROM table1;", 50000, 0);

    BufferTestType(String query, int numRecords1, int numRecords2) { ... }
}
```

Листинг 8: Tipovi testova za algoritme smene bafera

```
mvn test -Dgroups="[TIP PO ENUMERACIJI]" -Dbenchmark
```

Листинг 9: Pokretanje testova performanse algoritama smene bafera određenog tipa

Табела 5: Rezultati testiranja algoritama smene bafera za *HOT_BUFFERS*

Tip testa	Algoritam	Čitanja	Pisanja	Pogoci	Vreme (ms)
<i>HOT_BUFFERS</i>	<i>LRU</i>	337	1	13007	229
<i>HOT_BUFFERS</i>	<i>FIFO</i>	337	1	13006	219
<i>HOT_BUFFERS</i>	<i>Clock</i>	1792	1	11552	227
<i>HOT_BUFFERS</i>	<i>First unmodified</i>	13342	1	2	238
<i>HOT_BUFFERS</i>	<i>LRM</i>	13342	1	2	252
<i>HOT_BUFFERS</i>	<i>Naive</i>	13342	1	2	236

Табела 6: Rezultati testiranja algoritama smene bafera za *CONTINUOUS_READS*

Tip testa	Algoritam	Čitanja	Pisanja	Pogoci	Vreme (ms)
<i>CONTINUOUS_READS</i>	<i>LRU</i>	16669	1	3	162
<i>CONTINUOUS_READS</i>	<i>FIFO</i>	16669	1	3	162
<i>CONTINUOUS_READS</i>	<i>Clock</i>	16669	1	3	164
<i>CONTINUOUS_READS</i>	<i>First unmodified</i>	16671	1	1	157
<i>CONTINUOUS_READS</i>	<i>LRM</i>	16671	1	1	162
<i>CONTINUOUS_READS</i>	<i>Naive</i>	16671	1	1	161

Табела 7: Rezultati testiranja algoritama smene bafera za *HOT_BUFFERS_M*

Tip testa	Algoritam	Čitanja	Pisanja	Pogoci	Vreme (ms)
<i>HOT_BUFFERS_M</i>	<i>LRU</i>	3505	3317	64991	1046
<i>HOT_BUFFERS_M</i>	<i>FIFO</i>	3488	3291	65010	1082
<i>HOT_BUFFERS_M</i>	<i>Clock</i>	10640	2036	57855	1103
<i>HOT_BUFFERS_M</i>	<i>First unmodified</i>	13412	3542	55083	1118
<i>HOT_BUFFERS_M</i>	<i>LRM</i>	68481	3671	14	1232
<i>HOT_BUFFERS_M</i>	<i>Naive</i>	68481	3671	14	1170

Табела 8: Rezultati testiranja algoritama smene bafera za *CONTINUOUS_READS_M*

Tip testa	Algoritam	Čitanja	Pisanja	Pogoci	Vreme (ms)
<i>CONTINUOUS_READS_M</i>	<i>LRU</i>	166695	161509	45	1751
<i>CONTINUOUS_READS_M</i>	<i>FIFO</i>	166695	162112	45	1709
<i>CONTINUOUS_READS_M</i>	<i>Clock</i>	166695	95856	45	1586
<i>CONTINUOUS_READS_M</i>	<i>First unmodified</i>	166609	174945	131	1814
<i>CONTINUOUS_READS_M</i>	<i>LRM</i>	166735	175019	5	1816
<i>CONTINUOUS_READS_M</i>	<i>Naive</i>	166735	175019	5	1799

U tabelama 5, 6, 7, 8 stoje vrednosti metrika: broja čitanja sa diska, broja upisa na disk, broj pogodaka bafera koji je već u memoriji i vreme izvršavanja.

Rezultati testiranja performansi za tip testa gde se radi upit koji zahteva zadržavanje istih bafera u memoriji su predstavljeni tabelom 5. Ovi podaci jasno pokazuju podelu između algoritama koji uspešno prepoznaju radni skup unutrašnje petlje i onih koji potpuno zakazuju. *LRU* i *FIFO* ostvaruju optimalne rezultate sa minimalnim brojem čitanja i maksimalnim brojem pogodaka jer uspešno zadržavaju blokove manje tabele u memoriji. Nasuprot njima, *Naive*, *LRM* i *First Unmodified* pokazuju najgore performanse jer konstantno izbacuju potrebne podatke pre nego što se oni ponovo iskoriste, dok je *Clock* pozicioniran između njih sa umerenim brojem promašaja.

Rezultati testiranja performansi za tip testa gde radi upit koji zahteva sekvencijalno čitanje su predstavljeni tabelom 6. U ovom scenariju svi ispitivani algoritmi pokazuju praktično identično ponašanje jer veličina tabele drastično prevazilazi kapacitet bafer pula. Nijedna strategija ne može da zadrži podatke za ponovnu upotrebu, pa svi algoritmi završavaju sa istim brojem čitanja i minimalnim brojem pogodaka. Vreme izvršavanja je ujednačeno jer usko grlo u ovom testu diktira isključivo brzina samog diska.

Rezultati testiranja performansi za tip testa gde se prvo modifikuju podaci pa se radi upit koji zahteva zadržavanje istih bafera u memoriji su predstavljeni tabelom 7. Uvođenje modifikacije podataka dodatno naglašava razlike u efikasnosti, gde *LRU* i *FIFO* ponovo postižu najbolje rezultate sa najmanje čitanja jer uspešno čuvaju radni skup unutrašnje petlje. *First Unmodified* donekle uspešno iskorišćava činjenicu da su neki baferi “prljavi”, ali daje lošije rezultate jer ne izbacivanjem “prljavih” bafera previše sužava raspoloživi prostor u memoriji. *Naive* i *LRM* uvek izbacuju pogrešne bafere. *Clock* zauzima sredinu jer uspešno smanjuje broj upisa na disk, ali uz cenu većeg broja čitanja.

Rezultati testiranja performansi za tip testa gde se prvo modifikuju podaci pa se radi upit koji zahteva sekvencijalno čitanje su predstavljeni tabelom 8. Pri masovnom skeniranju izmenjenih podataka broj čitanja ostaje isti za sve strategije jer se svaka stranica mora povući sa diska, ali se ključna razlika uočava u broju upisa. *Clock* algoritam se ovde pokazuje kao najefikasniji jer značajno rasterećuje rad sa diskom u odnosu na *LRU* i *FIFO*.

Generalno, u svim testnim tipovima, broj bafera u listi bafera je izabran da bude 15 jer se time postiže da algoritmi izbora bafera za smenu moraju da rade pod pritiskom i izaberu najbolje bafere za smenu. Jedan slog tabele `table1` je dužine 1240 bajtova, jedan slog tabele `table2` je dužine 916 bajtova, dok je veličina bloka u sistemu podešena na 4096 bajtova. Ovo znači da u jednom bloku staje tri sloga tabele `table1` ili četiri sloga tabele `table2`.

Analizom podataka rezultata testiranja, tvrdnje iz sekcije 2.2.1 su potvrđene. *LRU* je ubedljivo pobednik za generalni slučaj, ali postoje i situacije gde ne mora biti najbrži.

9.3.2 Testovi performanse *RBO* tehnike u planiranju *SELECT* naredbi

Postoje dva tipa testova koji imaju funkciju testiranja algoritma planiranja *SELECT* naredbe. Tipovi su opisani naredbom koja se izvršava i količinom slogova u tabeli. Kod prvog tipa testa, prvo se stavlja tabela čija je veličina sloga veća. Kod drugog tipa testa, prvo se stavlja tabela čija je veličina sloga manja.

```
public enum ProductOrderTestType {
    BIG_SMALL("SELECT * FROM big, small;", 1000),
    SMALL_BIG("SELECT * FROM small, big;", 1000);

    ProductOrderTestType(String query, int numRecords) { ... }
}
```

Листинг 10: Tipovi testova za proizvod velikih i malih tabela

```
mvn test -Dgroups="[TIP IZ ENUMERACIJE]" -Dbenchmark
```

Листинг 11: Pokretanje testova performanse proizvoda velike i male tabele sa *LRU* algoritmom smene bafera

```
mvn test -Dgroups="[TIP IZ ENUMERACIJE]" -Dbenchmark -Dnaive
```

Листинг 12: Pokretanje testova performanse proizvoda velike i male tabele sa *Naive* algoritmom smene bafera

Табела 9: Rezultati testiranja *RBO* tehnike za tip *BIG_SMALL*

Tip testa	Algoritam smene bafera	$B(T_r) = B(T_l) + (RPB(T_l) * B(T_l) * B(T_d))$	Čitanja	Vreme (ms)
<i>BIG_SMALL</i>	<i>LRU</i>	2338	338	4926
<i>BIG_SMALL</i>	<i>Naive</i>	2338	2342	4845

Табела 10: Rezultati testiranja *RBO* tehnike za tip *SMALL_BIG*

Tip testa	Algoritam smene bafera	$B(T_r) = B(T_l) + (RPB(T_l) * B(T_l) * B(T_d))$	Čitanja	Vreme (ms)
<i>SMALL_BIG</i>	<i>LRU</i>	547094	338	5099
<i>SMALL_BIG</i>	<i>Naive</i>	547094	2342	5367

U tabelama 9, 10 stoje vrednosti metrika: predviđeni broj čitanja formulom, zapravo broj čitanja i vreme izvršavanja. Dodatno, stoji i algoritam smene bafera jer korišćenjem *LRU* algoritma se dodatno smanjuje broj pristupa (jer se blokovima manje tabele konstantno pristupa pa ostaju u memoriji), dok se korišćenjem *Naive* algoritma ne postiže ova optimizacija.

Na osnovu izmerenih podataka, tvrdnje iz sekcije 7.2.3.1 su potvrđene. Algoritam planiranja *SELECT* naredbe uvek bira jeftiniju putanju i *RBO* tehnika optimizacije ima vidljiv uticaj.

U okviru ovog poglavlja su objašnjeni raznovrsni detalji sistema koji nisu vezani za same funkcionalnosti sistema.

10.1 Izgradnja i pokretanje

Sistem koristi *Maven* za: automatizaciju kompilacije, izgradnju artefakata (aplikacija koje se pokreću) i za rukovođenje zavisnostima. U *Maven* ekosistemu, izvorni kod prati striktno definisanu strukturu i nalazi se unutar `src/main` direktorijuma.

Za korektno funkcionisanje *Maven* aplikacija, potrebno je definisati `pom.xml` datoteku u kojoj se nalaze sve neophodne instrukcije potrebne *Maven*-u.

Po instrukcijama `pom.xml` datoteke *LBDB* sistema, klijentske aplikacije i serverska aplikacija se grade odvojeno, u tri različita artefakta. Ovo omogućava jednostavno odvojeno pokretanje. Nakon izgradnje, artefakti se mogu pronaći unutar `target` direktorijuma pod imenima: `LBDBServer.jar`, `LBDBClient.jar` i `BulkExecutor.jar`.

```
mvn clean package -Dmaven.test.skip=true
```

Листинг 13: Izgradnja svih artefakata sistema, bez pokretanja testova

```
mvn test
```

Листинг 14: Pokretanje svih funkcionalnih testova u sistemu

10.1.1 Rukovođenje zavisnostima

Klijentske aplikacije i serverska aplikacija dele kod za:

- protokol komunikacije,
- definiciju svih ključnih reči (zbog *auto complete* funkcionalnosti klijentske aplikacije)
- definiciju konstanti,
- definiciju šeme i tipa vrednosti zbog korektnog ispisa.

Preostale zavisnosti i kod nisu deljeni.

10.1.1.1 Zavisnosti servera

Zavisnosti servera su sledeće:

- `datasketches-java` za *Java* implementaciju *HyperLogLog* strukture podataka⁷,
- `annotations` pruža dodatne *Java* anotacije poput `@NotNull`⁸.

⁷<https://datasketches.apache.org/>

⁸<https://github.com/JetBrains/java-annotations>

10.1.1.2 Zavisnosti klijenta

Zavisnosti običnog klijenta su sledeće:

- `jline-reader`, `jline-terminal` i `jline-terminal-jna` pružaju implementaciju terminala i omogućavaju sistemski agnostičnu podršku za *UTF-8* ispis⁹,
- `net.java.dev.jna:jna` za pristup *native* instrukcijama operativnog sistema¹⁰.

Zavisnosti `BulkExecutor` klijenta su iste kao i zavisnosti običnog klijenta, sa time da `jline-terminal` nije iskorišćen.

10.1.1.3 Zavisnosti testnog okruženja

Ove zavisnosti se koriste u testnom okruženju i nisu deo artefakta:

- `junit-jupiter-engine` i `junit-jupiter-params` za pokretanje i definisanje testova,
- `jimfs` je implementacija sistema datoteka u radnoj memoriji¹¹,
- `mockito-core` i `mockito-junit-jupiter` za pravljenje objekata koji imaju praznu implementaciju a neophodni su za pozivanje funkcija i metoda¹².

10.2 Sistemska konfiguracija

U okviru sistema postoji i konfiguraciona klasa `LBDBSettings` preko koje je moguće postaviti parametre izbora algoritama ili vrednosti za određene operacije. Podrazumevane vrednosti su dobar izbor za nenadgledanu inicijalizaciju sistema i mogu se videti u sledećem bloku koda:

```
public class LBDBSettings {
    public boolean UNDO_ONLY_RECOVERY = true;
    public int BLOCK_SIZE = 4096;
    public int BUFFER_POOL_SIZE = 128;
    public BufferStrategy bufferStrategy = BufferStrategy.LRU;
    public String LOG_FILE = "log_file";
    public QueryPlannerType queryPlannerType = QueryPlannerType.BETTER;
    public UpdatePlannerType updatePlannerType = UpdatePlannerType.BASIC;
}
```

Листинг 15: Kod sistemske klase `LBDBSettings`

⁹<https://github.com/jline/jline3>

¹⁰<https://github.com/java-native-access/jna>

¹¹<https://github.com/google/jimfs>

¹²<https://github.com/mockito/mockito>

Redom, parametri označavaju:

1. algoritam oporavka sistema,
2. veličina jednog bloka u bajtovima gde jedan blok predstavlja najmanju jedinicu interakcije sa diskom,
3. količina bafera sa kojim sistem raspolaže,
4. algoritam izbora bafera koji će biti smenjen,
5. putanja do datoteke gde se čuvaju podaci potrebni za oporavak sistema i poništavanje transakcija,
6. implementacija planera za operacije upita; podržana samo BETTER implementacija,
7. implementacija planera za operacije modifikacije; podržana samo BASIC implementacija.

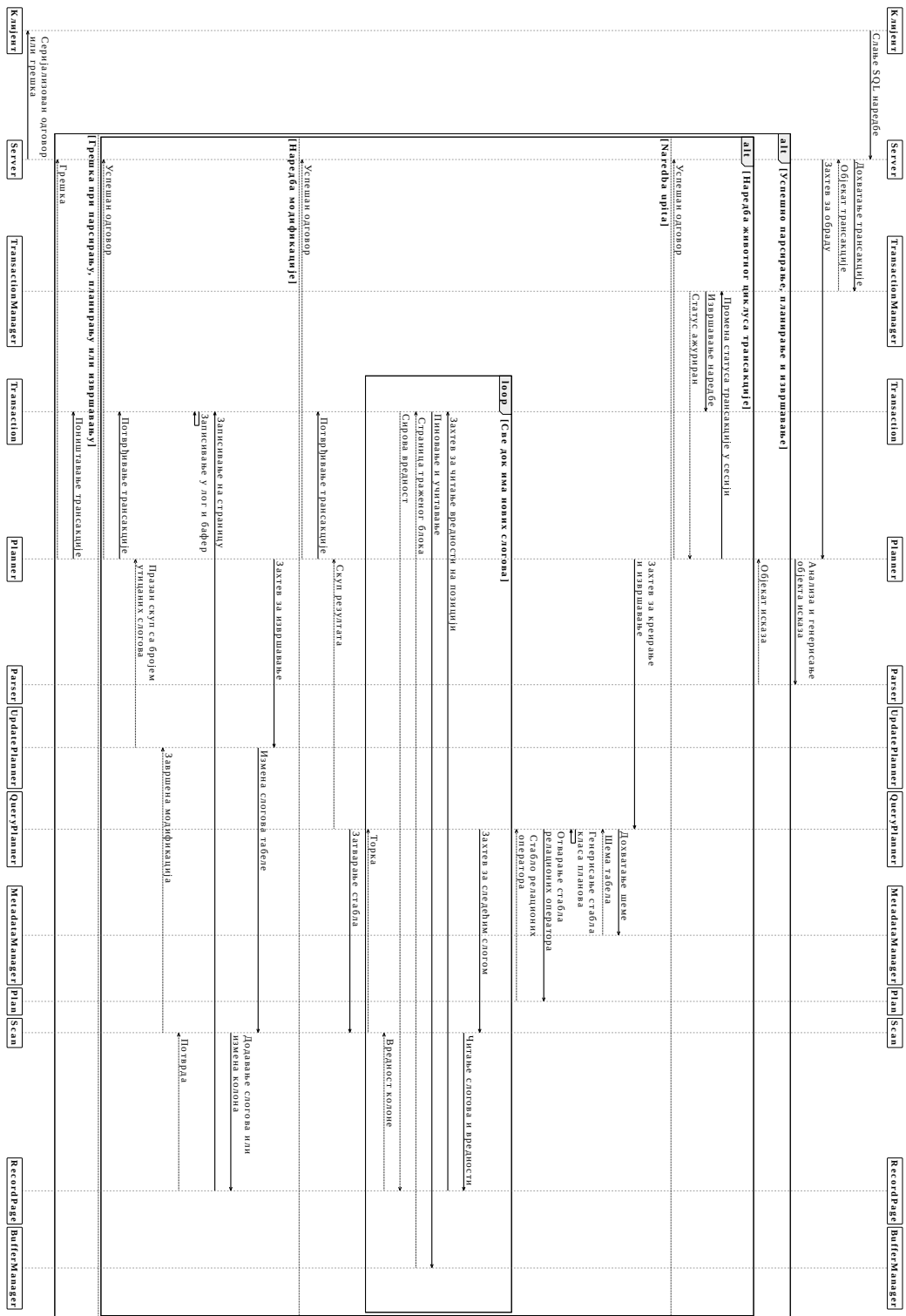
10.3 Integracija sa *GitHub* platformom

*GitHub*¹³ platforma omogućava pokretanje testova (eng. *Continuous Integration, CI*) i izgradnju i objavljivanje aplikacija (eng. *Continuous Delivery, CD*) u okviru njenih servera. *LBDB* sistem koristi ovu mogućnost i definiše specijalnu *GitHub* datoteku za *CI*. U okviru nje se definiše *Windows* i *Ubuntu Linux* okruženje za testiranje. Rezultat testova stoji u okviru bedža u *README.md* datoteci koja se nalazi u *LBDB* repozitorijumu.

10.4 Primer funkcionisanja celokupnog sistema

Dijagram sekvence (slika 33) predstavlja generalno ponašanje svih slojeva *LBDB* sistema. Opisani su slučajevi za *SELECT* naredbu, za naredbe upravljanja životnim ciklusom transakcija i za naredbe modifikacije tabela. Specifičnosti poput algoritma pravljenja stabla planova ili algoritma poništavanja transakcija nisu obrađeni jer bi dijagram bio prevelik, a njihovo objašnjenje je svakako dato u poglavljima gde su definisani.

¹³<https://github.com/>



Слика 33: Dijagram sekvence funkcionisanja celog sistema

11.1 Motivacija

Sistemi za upravljanje bazama podataka (SUBP) su me interesovali od druge godine osnovnih akademskih studija, nakon slušanja predmeta “Baze podataka”. SUBP-ovi su osnova većine informacionih sistema koji se koriste u današnjici i razumevanje samo *SQL* jezika nije dovoljno da bi se shvatilo njihovo interno funkcionisanje. Zbog njihove kompleksnosti, shvatio sam da bi ih najbolje razumeo tako što implementiram svoj SUBP sistem. Naučio sam mnogo iz oblasti upravljanja datotekama, izolacije podataka i obrade deklarativnih programskih jezika kao što je *SQL*.

11.2 Ograničenja sistema

Iako je *LBDB* SUBP funkcionalan i pruža zadovoljavajuću podršku *SQL* jezika, za neke delove implementacije se može reći da im fali još dorade. Odlučio sam da napravim presek kod ovih stvari, da bi postigao funkcionalnu implementaciju u doglednom vremenskom periodu, makar ona ne bila savršena.

11.2.1 Implementiran je samo podskup *SQL*-a

Specifikacija *SQL* jezika definisana *ISO/IEC 9075* standardom¹⁴ pokriva veliku količinu naredbi. *LBDB* sistem implementira samo najosnovnije naredbe potrebne za primitivno upravljanje tabelama i slogovima.

Argumentacija ne implementiranja raznih grupa naredbi je sledeća:

- Iako se brisanje tabela svodi na brisanje slogova u kataloškim tabelama i brisanje datoteka tih tabela, sistem oporavka i potvrde transakcija nije dovoljno napredan da prati promene koje se dešavaju van bafera: kreiranje i brisanje datoteka. Na primer, ako se u istoj transakciji kreira i obriše tabela, nakon potvrde transakcije datoteka tabele će ostati na disku. Ovo se dešava jer proces potvrde zapisuje sve bafere (sekcija 3.1.2.2.2), a bar jedan bafer će biti dodeljen novoj obrisanoj datoteci i njegovo pisanje na disk će uvek rekreirati datoteku.

Dodavanje novih blokova na kraju datoteka je iste prirode kao i kreiranje i brisanje datoteka, ali poništavanje te akcije je lakše implementirati pošto će datoteka i dalje postojati nakon oporavka.

¹⁴https://en.wikipedia.org/wiki/ISO/IEC_9075

- Ne postoje operacije nad celim bazama podataka (`CREATE DATABASE ...`) jer nisu neophodne za funkcionisanje implementacije relacionog modela.
- Ne postoje pogledi (eng. *views*). Iako u *Database Design And Implementation* [1] knjizi postoji opis implementacije pogleda, odlučio sam da ih izbacim jer se nisu slagali sa svim semantičkim proverama planera. Potrebno je produbiti i eventualno promeniti načine na koji planer proverava upite da bi se lako proveravali i pogledi.
- `GROUP BY`, `DISTINCT`, `ORDER BY` i ostali delovi `SELECT` naredbe koji zahtevaju agregaciju podataka nisu podržani jer ne postoji implementacija materijalizovane obrade. *Database Design And Implementation* [1] u poglavlju 13 opisuje materijalizovanu obradu, pa ću je istražiti za sledeću iteraciju *LBDB* sistema.
- Ostatak *SQL* jezika je previše kompleksan za implementaciju u okviru ovakvog projekta, ali je vredno istražiti ga da bi se shvatilo kako moderni SUBP-ovi funkcionišu [9].

11.2.2 Ograničenje na slogove fiksne dužine

LBDB sistem za upravljanje datotekama i slogovima ograničava slogove na fiksnu, unapred definisanu veličinu i ređa ih serijski jedne do drugih. Prednost ovog načina upravljanja slogovima je jednostavnost implementacije i lako računanje pozicija slogova (što dalje omogućava lako brisanje, umetanje, ...). Uz neke promene (omogućavanje *NULL* vrednosti), preuzet je direktno iz *Database Design And Implementation* [1] knjige.

Prezentuju se dve velike mane:

- čuvanje vrednosti različitih dužina u okviru iste kolone (*String*, to jest ***VARCHAR*** tip) je nemoguće, jer je uvek potrebno znati unapred veličinu svih vrednosti. Da bi se mogle smestiti sve vrednosti do te dužine, finalna veličina kolone će uvek biti maksimalna.
- ukupna veličina sloga (u bajtovima) ne sme da bude veća od sistemski definisane veličine jednog bloka (isto u bajtovima) jer slogovi ne mogu da se prostiru kroz više od jednog bloka.

Slotted Page Architecture (*SPA*) predstavlja arhitekturu koja rešava ove probleme i moderni SUBP-ovi je intenzivno koriste¹⁵. Koncepti na koje se oslanja su prvi put uvedeni u okviru *SystemR* RSS (*Relational Storage System*) sistema [10].

U okviru *SPA*, vrednosti slogova, pa ni sami slogovi, nemaju predefinisanu poziciju. Blokovi se isto mapiraju na stranice. Jedna stranica se sastoji od dve komponente: zaglavlje i vrednosti. U zaglavlju stoje specijalni brojevi koji označavaju pozicije (eng. *slots*) vrednosti. Zaglavlje se uvek nalazi na početku stranice i raste u desno, a vrednosti se uvek nalaze na kraju stranice i rastu u levo. Stranica se smatra popunjenim ako se zaglavlje preklopi sa vrednostima.

Ako se stranica popuni na taj način da svi slogovi koji joj logički pripadaju nemaju dovoljno prostora za sve svoje vrednosti, toj stranici se dodeljuje stranica preliivanja (eng. *overflow*

¹⁵<https://www.postgresql.org/docs/current/storage-page-layout.html>

page) i time se izbegava ograničenje veličine sloga na veličinu bloka. Pozicija stranice prelivanja se isto nalazi u zaglavlju. Moguće je uvezivati više stranica prelivanja. Ako se stranica popuni i treba da se doda novi slog koji joj ne pripada logički, pravi se nova stranica umesto stranice prelivanja.

U zaglavlju takođe stoji i pozicija sledećeg logičkog sloga. Slogovi se identifikuju preko *slot* vrednosti pa je reorganizacija, brisanje i dodavanje slogova moguća manipulacijom samo *slot* vrednosti. Ovo znatno olakšava probleme stvorene stranicama prelivanja, jer se slogovi ne prate preko fizičke pozicije, već preko logičke pozicije. Rupe i fragmentacija stranica se rešavaju kompakcijom (*PostgreSQL VACUUM* naredba¹⁶).

11.2.3 Sporo računanje statističkih metapodataka

Statistički metapodaci sistema se čuvaju u memoriji i pristup njima je brz. Problemi ovog načina su to što se ne skalira efikasno sa brojem tabela i to što se računanje metapodataka mora raditi iznova (pri svakom pokretanju sistema, na svakih 100 poziva). Bolji pristup je čuvanje statističkih metapodataka u zasebnim kataloškim tabelama. *LBDB* ne podržava ovaj način zato što je:

- održavanje ažurnosti tih tabela zahtevno
- čitanje iz tih tabela potrebno raditi brzo što dalje zahteva implementaciju *read uncommitted* izolacionog nivoa transakcija

Moderni SUBP-ovi implementiraju i ceo deo spomenutog *SQL* standarda u kom su definisani specijalni pogledi i tabele metapodataka [1]. Pored toga postoje i kompleksni histogrami koji sadrže razne podatke iz više tabela i dodatno ubrzavaju upite [11].

11.2.4 Nedostajuća implementacija indeksnih struktura podataka

Indeksi predstavljaju specijalnu strukturu podataka koja omogućava znatno bržu pretragu podataka. Realizuju se preko jedne od *BTree* varijanti ili kao *Hash* indeks.

Iako su indeksi krucijalni za efikasan SUBP, odlučio sam da ih ne implementiram jer želim da im se posvetim u okviru sledeće iteracije *LBDB* sistema. Ipak, podržano je kreiranje metapodataka vezanih za indekse u okviru menadžera metapodataka, ali smatram da ovo nije vredno spominjati van ovog poglavlja jer ne utiče na dalji sistem.

11.2.5 Neoptimalni planer

Planer *LBDB* sistema ne koristi skoro nijednu naprednu tehniku planiranja. Brzina izvršavanja upita dosta zavisi od redosleda tabela u upitu, uslov filtriranja se primenjuje nakon ulančavanja svih tabela umesto izolovano po tabeli, selektivnost i procene broja *NULL* i jedinstvenih vrednosti se ne koriste, itd.

¹⁶<https://www.postgresql.org/docs/current/sql-vacuum.html>

Poglavlja 14 i 15 iz [1] opisuju implementaciju efikasnijeg planera koji intenzivno koristi *RBO* tehnike planiranja, ali je ovo ostavljeno za sledeću iteraciju *LBDB* sistema.

11.2.6 Ulančavanje članova predikata je moguće samo konjunkcijom

Predikati se mogu sastojati samo od članova ulančanih logičkom operacijom konjunkcije (AND). Negacija izraza (NOT) i ulančavanje operacijom disjunkcije (OR) nisu podržani iako je lako dodati obradu logičkih operacija jer nisam bio siguran kako se uklapaju u napredne tehnike planiranja. Kada završim sa istraživanjem naprednog planera, biće mi lakše da ubacim i nedostajuće logičke operacije.

11.3 Razlike između osnovne implementacije i *LBDB* sistema

U tabeli 11 se mogu videti razlike između osnovne implementacije i *LBDB* sistema.

Табела 11: Najznačajnija proširenja u implementaciji *LBDB* sistema

Osnovna implementacija iz [1]	Implementacija <i>LBDB</i> sistema
<i>Integer</i> i <i>String</i> tipovi	Podržan i <i>Boolean</i> tip
<i>Naive</i> algoritam smene bafera	Podržani i <i>FIFO</i> , <i>LRU</i> , <i>Clock</i> , <i>First unmodified</i> , <i>LRM</i> algoritmi smene bafera
<i>Undo only</i> algoritam oporavka	Podržan i <i>undo redo</i> algoritam oporavka
-	Podržane <i>NULL</i> vrednosti
-	Aproksimacija broja jedinstvenih vrednosti kolone tabele preko <i>HyperLogLog</i> probabilističke strukture
Konstantni i identifikacioni izrazi	Podržani i aritmetički izrazi i članovi sa svim operatorima poređenja
Samo operatori selekcije, projekcije i proizvoda	Podržani i operatori generalizovane projekcije, preimenovanja i unije
Prosto računanje redukcionog faktora	Računanje redukcionog faktora na osnovu algoritma istraživačkog rada <i>SystemR</i> [7]
-	Izraz zamenskog člana
-	Detaljna semantička provera svih naredbi
-	Podržana <i>EXPLAIN</i> naredba
-	Redukcija izraza za vreme planiranja
-	Protokol komunikacije, serverski i klijentski sloj, kontrolisano gašenje sistema i periodično pisanje mirne kontrolne tačke
-	Implementacija sistema za testiranje funkcionalnosti
Starija <i>Java</i> verzija	Moderna <i>Java</i> verzija i njene mogućnosti poput <i>sealed interface</i> , <i>record</i> , <i>Optional</i> , <i>stream API</i> -ja, ...
-	<i>Maven</i> za automatizaciju kompilacije

Spisak slika

Слика 1	Uprošćeni dijagram slojevite arhitekture sistema	1
Слика 2	Izgled <i>log</i> datoteke i smer iteracije	4
Слика 3	Izgled jednog bloka popunjenog slogovima	8
Слика 4	Operacije, njihovi <i>log</i> tipovi i struktura <i>log</i> tipova	11
Слика 5	Hijerarhija konstanti	19
Слика 6	Hijerarhija izraza	20
Слика 7	Hijerarhija implementacije relacionih operatora	23
Слика 8	Primer naredbe u <i>SQL</i> jeziku gde se pokazuju sve komponente virtuelne mašine	26
Слика 9	Primer poziva <code>getValue()</code> metode u konkretnom stablu relacionih operatora	27
Слика 10	Dijagram sekvence <code>next()</code> metode u konkretnom stablu relacionih operatora	28
Слика 11	Gramatika <i>Parse</i> sintaksičke kategorije	30
Слика 12	Gramatika <i>ParseUpdate</i> sintaksičke kategorije	31
Слика 13	Gramatika <i>ParseSelect</i> sintaksičke kategorije	31
Слика 14	Gramatika <i>ParseInsert</i> sintaksičke kategorije	32
Слика 15	Gramatika <i>ParseDelete</i> sintaksičke kategorije	32
Слика 16	Gramatika <i>ParseCreateTable</i> sintaksičke kategorije	33
Слика 17	Gramatika <i>ParsePredicate</i> sintaksičke kategorije	33
Слика 18	Gramatika <i>ParseExpression</i> sintaksičke kategorije	34
Слика 19	Gramatika <i>ParseEvaluable</i> sintaksičke kategorije	36
Слика 20	Hijerarhija implementacije planova	38
Слика 21	<i>Flowchart</i> dijagram računice redukcionog faktora člana	40
Слика 22	Struktura planera	45
Слика 23	<i>Flowchart</i> dijagram <i>BetterQueryPlanner</i> algoritma planiranja	47
Слика 24	Primer stabla planova nakon unije 3 naredbe	48
Слика 25	Primer stabla planova nakon filtriranja i projekcije	48
Слика 26	Primer stabla planova nakon proizvoda 3 tabele	49
Слика 27	Primer stabla planova nakon kvalifikacije kolona tabele	49
Слика 28	Primer kompletnog stabla planova	50
Слика 29	Struktura <i>Response</i> hijerarhije	54
Слика 30	Struktura <i>QuerySet</i> paketa	55
Слика 31	Dijagram sekvence obrade klijenta	56
Слика 32	Dijagram sekvence bezbednog gašenja servera	57

Слика 33	Dijagram sekvence funkcionisanja celog sistema	68
----------	--	----

Spisak listinga

Листинг 1	Podržani tipovi kolona	7
Листинг 2	Evaluatable interfejs	21
Листинг 3	Prioriteti aritmetičkih operacija u sistemu	35
Листинг 4	Primer rezultujuće tabele EXPLAIN naredbe	50
Листинг 5	Primer ispisane tabele SELECT * FROM tablecatalog; naredbe	58
Листинг 6	Konfiguracioni parametri <i>JUnit</i> biblioteke	59
Листинг 7	Podešavanja apstraktne klase testa performanse	61
Листинг 8	Tipovi testova za algoritme smene bafera	61
Листинг 9	Pokretanje testova performanse algoritama smene bafera određenog tipa .	61
Листинг 10	Tipovi testova za proizvod velikih i malih tabela	64
Листинг 11	Pokretanje testova performanse proizvoda velike i male tabele sa <i>LRU</i> algoritmom smene bafera	64
Листинг 12	Pokretanje testova performanse proizvoda velike i male tabele sa <i>Naive</i> algoritmom smene bafera	64
Листинг 13	Izgradnja svih artefakata sistema, bez pokretanja testova	65
Листинг 14	Pokretanje svih funkcionalnih testova u sistemu	65
Листинг 15	Kod systemske klase <i>LBDBSettings</i>	66

Spisak tabela

Табела 1	Različiti izolacioni nivoi transakcija	14
Табела 2	Šema <i>tablecatalog</i> tabele	17
Табела 3	Šema <i>fieldcatalog</i> tabele	17
Табела 4	Primer karakteristika tabela za računanje broja blokova plana proizvoda . . .	42
Табела 5	Rezultati testiranja algoritama smene bafera za <i>HOT_BUFFERS</i>	62
Табела 6	Rezultati testiranja algoritama smene bafera za <i>CONTINUOUS_READS</i> . . .	62
Табела 7	Rezultati testiranja algoritama smene bafera za <i>HOT_BUFFERS_M</i>	62
Табела 8	Rezultati testiranja algoritama smene bafera za <i>CONTINUOUS_READS_M</i> .	62
Табела 9	Rezultati testiranja <i>RBO</i> tehnike za tip <i>BIG_SMALL</i>	64
Табела 10	Rezultati testiranja <i>RBO</i> tehnike za tip <i>SMALL_BIG</i>	64
Табела 11	Najznačajnija proširenja u implementaciji <i>LBDB</i> sistema	72

Spisak korišćenih skraćenica

Skraćenica	Značenje
SUBP	Sistem za upravljanje bazama podataka
SQL	<i>Structured Query Language</i>
AST	<i>Abstract Syntax Tree</i>
ACID	<i>Atomicity, Consistency, Isolation, Durability</i>
MVCC	<i>Multi Version Concurrency Control</i>
FIFO	<i>First In First Out</i>
LRU	<i>Least Recently Used</i>
LRM	<i>Least Recently Modified</i>
RAII	<i>Resource Acquisition Is Initialization</i>
RBO	<i>Rule-Based Optimisation</i>
HBO	<i>Heuristics-Based Optimisation</i>
CBO	<i>Cost-Based Optimisation</i>
JDBC	<i>Java Database Connectivity</i>
SPA	<i>Slotted Page Architecture</i>
RSS	<i>Relational Storage System</i>
CI	<i>Continuous Integration</i>
CD	<i>Continuous Delivery</i>
U/I	Ulaz/Izlaz
API	<i>Application Programming Interface</i>
EOF	<i>End Of File</i>
UTF-8	<i>Unicode Transformation Format</i>

Skraćenica**Značenje***RESP**REdis Serialization Protocol*

Spisak korišćenih pojmova

Pojam	Objašnjenje
Blok	Najmanja jedinica interakcije sa datotekom i diskom.
Stranica	Sirovi bajtovi jednog bloka učitani u radnu memoriju.
Bafer	Oмотаč oko stranice u memoriji koji sadrži dodatne metapodatke za upravljanje stanjem.
Pinovanje	Označavanje bafera od strane transakcije kako se on ne bi izbacio iz radne memorije dokle god je u upotrebi.
Log sekvenca	Unikatni identifikator zapisanog <i>log</i> -a na osnovu koga se održava redosled operacija.
Mirna kontrolna tačka	Oznaka u <i>log</i> datoteci nakon koje sistem za oporavak ne mora da gleda u prošlost jer nema aktivnih transakcija
<i>Deljeni</i> katanac	Mehanizam zaključavanja bloka koji omogućava višenitno čitanje istih podataka bez modifikacija.
<i>Ekskluzivni</i> katanac	Mehanizam zaključavanja bloka koji garantuje isključivi pristup podacima zarad modifikacije.
Prljivo čitanje	Problem konkurentnosti gde jedna transakcija čita nepotvrđene izmene druge transakcije.
Fantomsko čitanje	Problem konkurentnosti gde se tokom obrade opsega pojave novi slogovi umetnuti od strane druge transakcije.
Slog	Najmanja instanca podataka koja opisuje jedan red iz tabele.
Šema	Definicija atributa, tipova, dužina i ograničenja kolona za jednu tabelu.
Kataloška tabela	Sistemska tabela baze u kojoj se perzistiraju metapodaci potrebni za funkcionisanje sistema.
<i>HyperLogLog</i>	Probabilistička struktura podataka iskorišćena za procenu broja jedinstvenih vrednosti.
Virtuelna tabela	Skup rezultujućih slogova nakon primene relacionih operatora koji nisu nužno perzistirani na disku.
Izraz	Aritmetička kombinacija konstanti, operatora i kolona čijom evaluacijom nad nekim slogom dobijamo konstantnu numeričku vrednost.

Član	Objekat koji poredi dva izraza na osnovu nekog operatora poređenja. Evaluacijom dobijamo da li je rezultat poređenja istinit ili ne.
Predikat	Logička kombinacija članova i čijom evaluacijom dobijamo konstantnu logičku vrednost.
Protočna obrada	Izvršavanje stabla operatora obradom slogova jedan po jedan, bez čuvanja međurezultata u memoriji.
Materijalizovana obrada	Čuvanje svih (ili delimičnih) izračunatih međurezultata na disku ili u memoriji, potrebno za operacije sortiranja i agregacije.
Redukcioni faktor	Statistički broj koji označava za koliko će se puta smanjiti količina izlaznih slogova pod dejstvom određenog predikata.
Iskaz	Najviša sintaktička kategorija koja opisuje potpunu <i>SQL</i> operaciju i sadrži sve neophodne podatke da bi se ista izvršila (eng. <i>Statement</i>).
<i>Pratt parsing</i>	Tehnika <i>recursive descent</i> parsiranja za efikasnu validaciju izraza i obradu aritmetičkih prioriteta.
<i>Generics</i>	Mehanizam <i>Java</i> programskog jezika za pisanje agnostičnog koda kojeg različiti tipovi koriste na isti način
Punokvalifikujuće ime	Ime kolone sastavljeno od imena tabele i same kolone kako bi se izbegla dvosmislenost.
Parcijalni evaluator	Komponenta koja statički svodi izraze za trivijalnim operacijama na konstante već u trenucima kreiranja plana upita.
Protokol serijalizacije	Skup pravila konvertovanja struktura podataka u linearan niz bajtova za prenos kroz mrežu.
Ugrađeni režim	Režim rada baze podataka gde ona postoji u okviru istog operativnog procesa kao i klijentska aplikacija koja je koristi.
Serverski režim	Režim rada gde se baza podataka ponaša kao server, primajući naredbe sa mreže od više nezavisnih klijenata.
<i>Socket</i>	Par <i>IP</i> adrese i porta koji, uz dodatnu logiku, omogućava komunikaciju preko mreže
Zavisnost	Softverska biblioteka od koje zavisi izgradnja i uspešan rad sistema.
<i>Native</i> instrukcije	Instrukcije specifičnog operativnog sistema

Литература

- [1] E. Sciore, *Database Design and Implementation: Second Edition*. in Data-Centric Systems and Applications. Springer Cham, 2020.
- [2] T. Haerder and A. Reuter, “Principles of transaction-oriented database recovery,” *ACM computing surveys (CSUR)*, vol. 15, no. 4, pp. 287–317, 1983.
- [3] P. A. Bernstein and N. Goodman, “Multiversion concurrency control—theory and algorithms,” *ACM Transactions on Database Systems (TODS)*, vol. 8, no. 4, pp. 465–483, 1983.
- [4] P. Flajolet, É. Fusy, O. Gandouet, and F. Meunier, “Hyperloglog: the analysis of a near-optimal cardinality estimation algorithm,” *Discrete mathematics & theoretical computer science*, no. Proceedings, 2007.
- [5] E. F. Codd, “A relational model of data for large shared data banks,” *Communications of the ACM*, vol. 13, no. 6, pp. 377–387, 1970.
- [6] V. R. Pratt, “Top down operator precedence,” in *Proceedings of the 1st annual ACM SIGACT-SIGPLAN symposium on Principles of programming languages*, 1973, pp. 41–51.
- [7] P. G. Selinger, M. M. Astrahan, D. D. Chamberlin, R. A. Lorie, and T. G. Price, “Access path selection in a relational database management system,” in *Proceedings of the 1979 ACM SIGMOD international conference on Management of data*, 1979, pp. 23–34.
- [8] E. Wong and K. Youssefi, “Decomposition—a strategy for query processing,” *ACM Transactions on Database Systems (TODS)*, vol. 1, no. 3, pp. 223–241, 1976.
- [9] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed. McGraw-Hill Education, 2020.
- [10] M. M. Astrahan *et al.*, “System R: relational approach to database management,” *ACM Trans. Database Syst.*, vol. 1, no. 2, pp. 97–137, June 1976.
- [11] P. B. Gibbons, Y. Matias, and V. Poosala, “Fast incremental maintenance of approximate histograms,” *ACM Trans. Database Syst.*, vol. 27, no. 3, pp. 261–298, Sept. 2002.